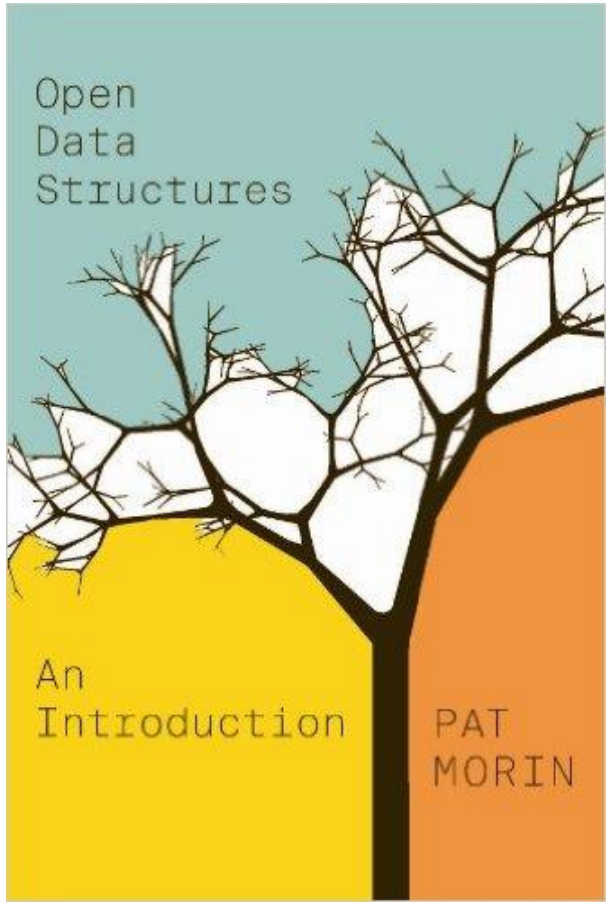
Heaps

COMP 2402
Abstract Data Types & Algorithms

Readings

Binary Heap

- Chapter 10.1



Readings

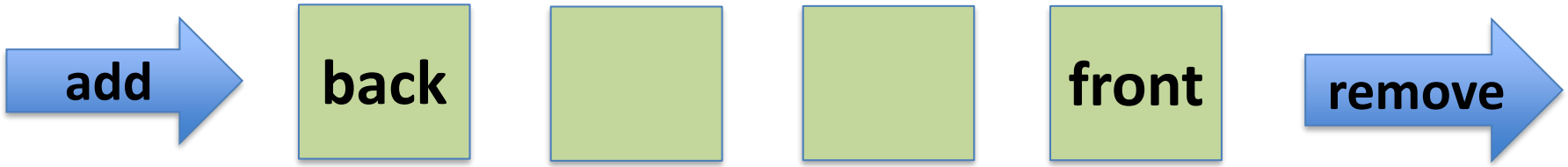
Binary Heap

- Chapter 10.1

Heaps

Recall the **FIFO Queue** abstract data type.

A FIFO queue simulates waiting in a line (queue)

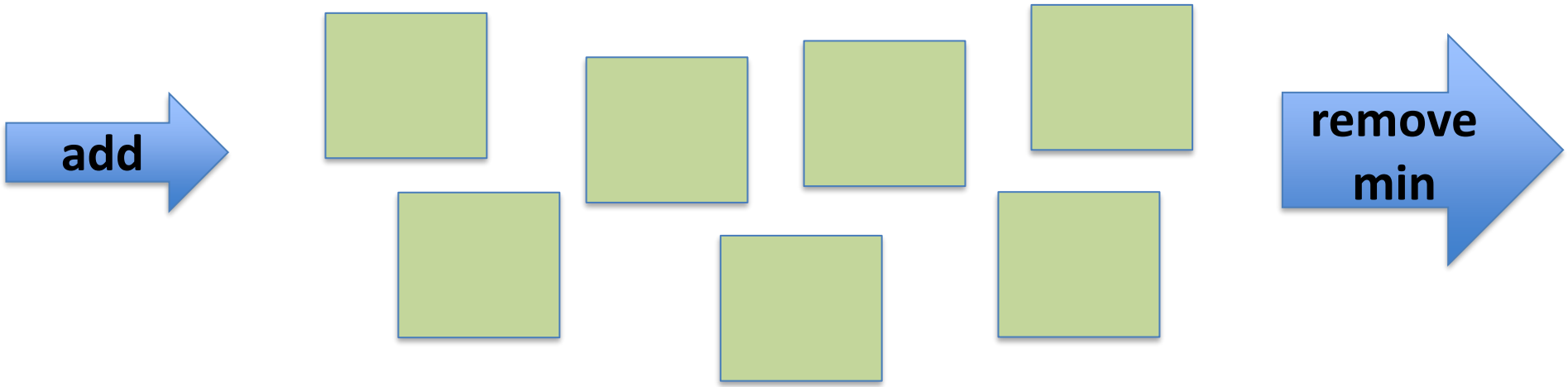


A FIFO queue has a sequential structure.

Heaps

The **Priority Queue** abstract data type.

A Priority Queue simulates an emergency room waiting list

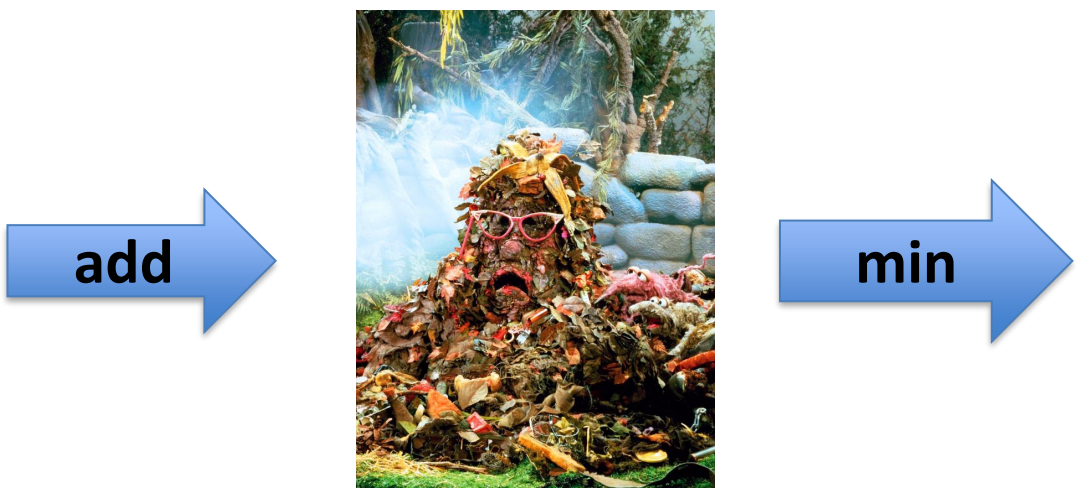


The queueing discipline is independent of items' values. It removes an item with **minimal priority** first

Heaps

A **heap** is a disorganized pile.

The **heap** data structure is an implementation of the priority queue interface. (We will focus on the **binary heap**.)



Heaps

A **binary heap** is a disorganized binary tree that maintains the **heap property**.

Recall that a **Random Binary Search Tree** was a strictly organized tree that maintained both the binary search tree property and the **heap property**.

Heaps

A **binary heap** is a disorganized binary tree that maintains the **heap property**.

For every node u, except the root, the heap property states that

$$u.parent.priority < u.priority$$

Or, a parents priority is always less than its childrens'.

Heaps

A **binary heap** is a disorganized binary tree that maintains the **heap property**.

What do we want?

1. Return an item with minimal priority in $O(\log n)$ time.
   <<remove() should be fast.>>

Heaps

A **binary heap** is a disorganized binary tree that maintains the **heap property**.

What do we want?

2. Add an item in guaranteed* $O(\log n)$ time
   <<add() should be fast.>>

Heaps

A **binary heap** is a disorganized binary tree that maintains the **heap property**.

What do we want?

1. Return an item with minimal priority in $O(\log n)$ time.
   <<remove() should be fast.>>
2. Add an item in guaranteed* $O(\log n)$ time
   <<add() should be fast.>>

How do we do that in a binary tree?

Heaps

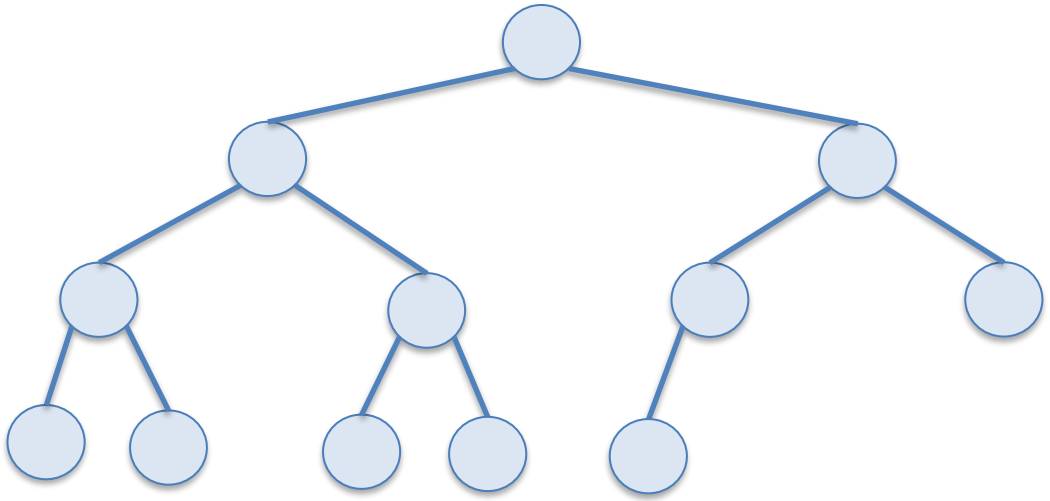
A **binary heap** is a disorganized binary tree that maintains the **heap property**.

Let’s give some more structure to our tree.

Heaps

Consider a **complete** binary tree with height h.

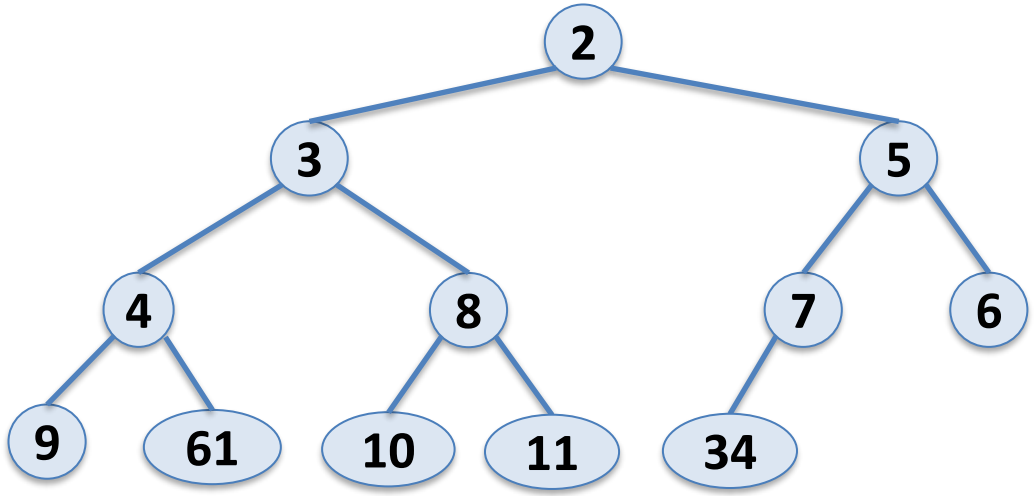
- Every level of the tree is full except possibly the last level (h)
- All nodes in last level are as far left as possible.



What is the height of a complete binary tree?

Heaps

A **binary heap** is a **complete** binary tree that maintains the **heap property**.



Heaps

`remove()`

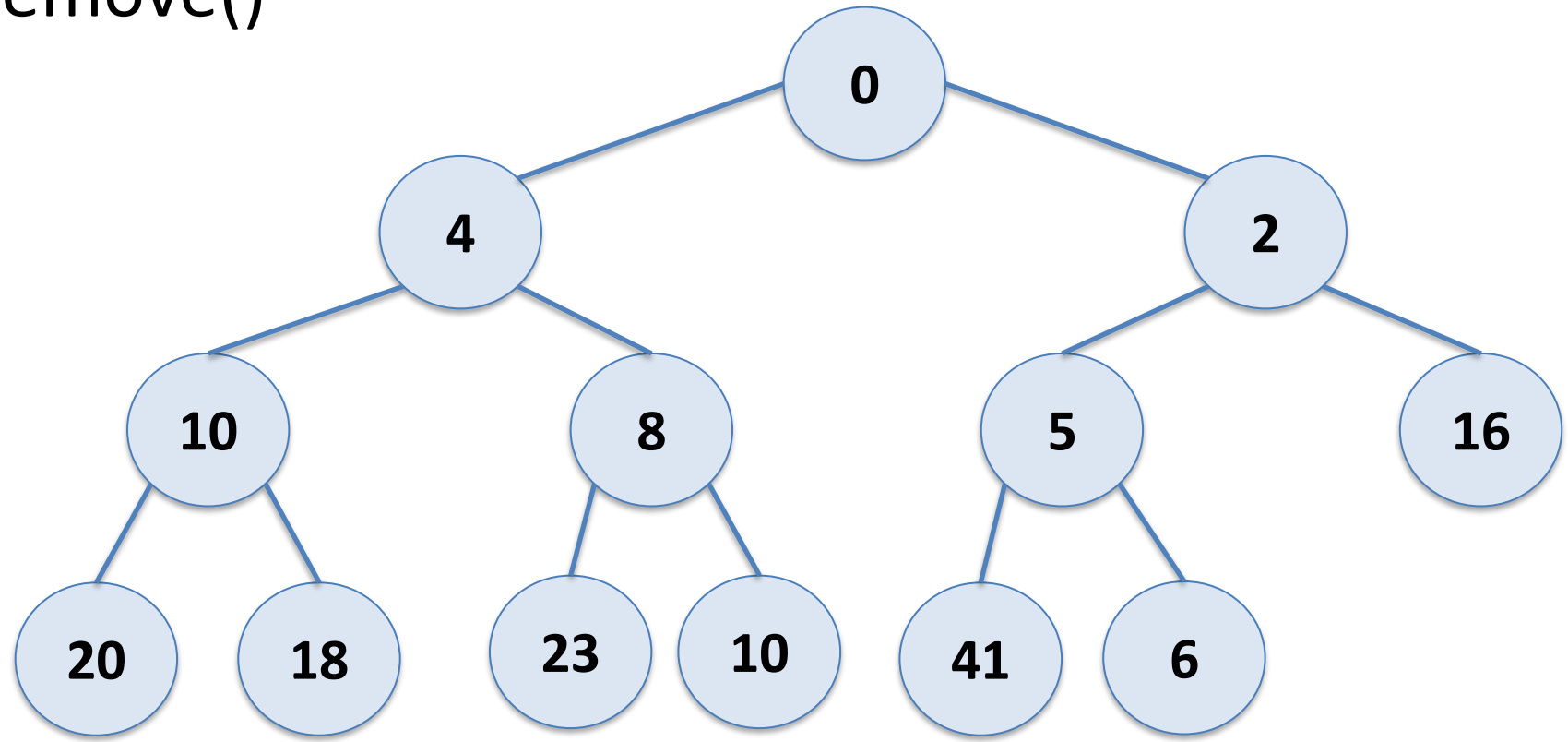
- Swap root element (the min element to return) with the rightmost element in the last level

- **trickle down** the new root until heap property is satisfied
  - Repeatedly swap the node with its child with least priority

- Return the min element

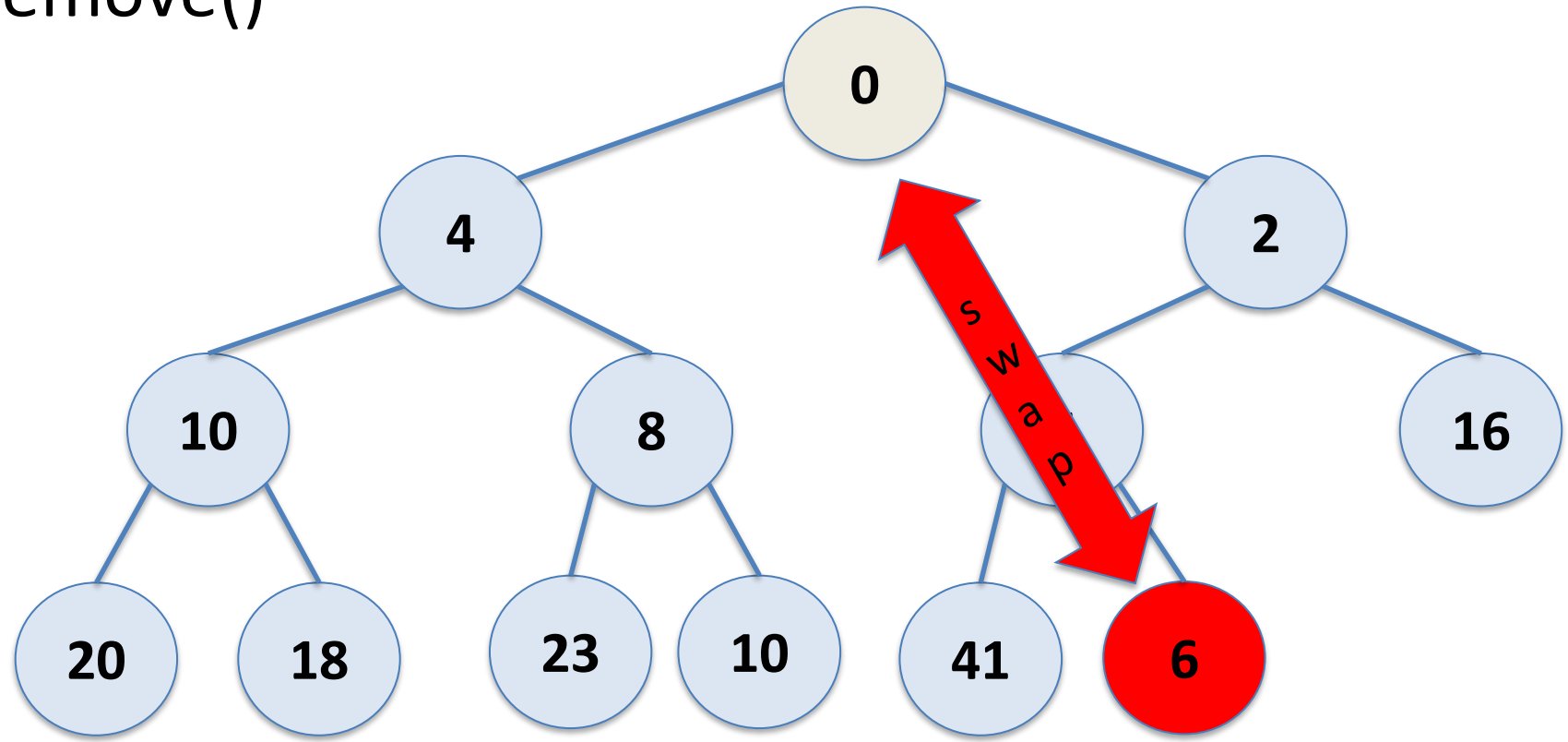
Binary Heap

remove()



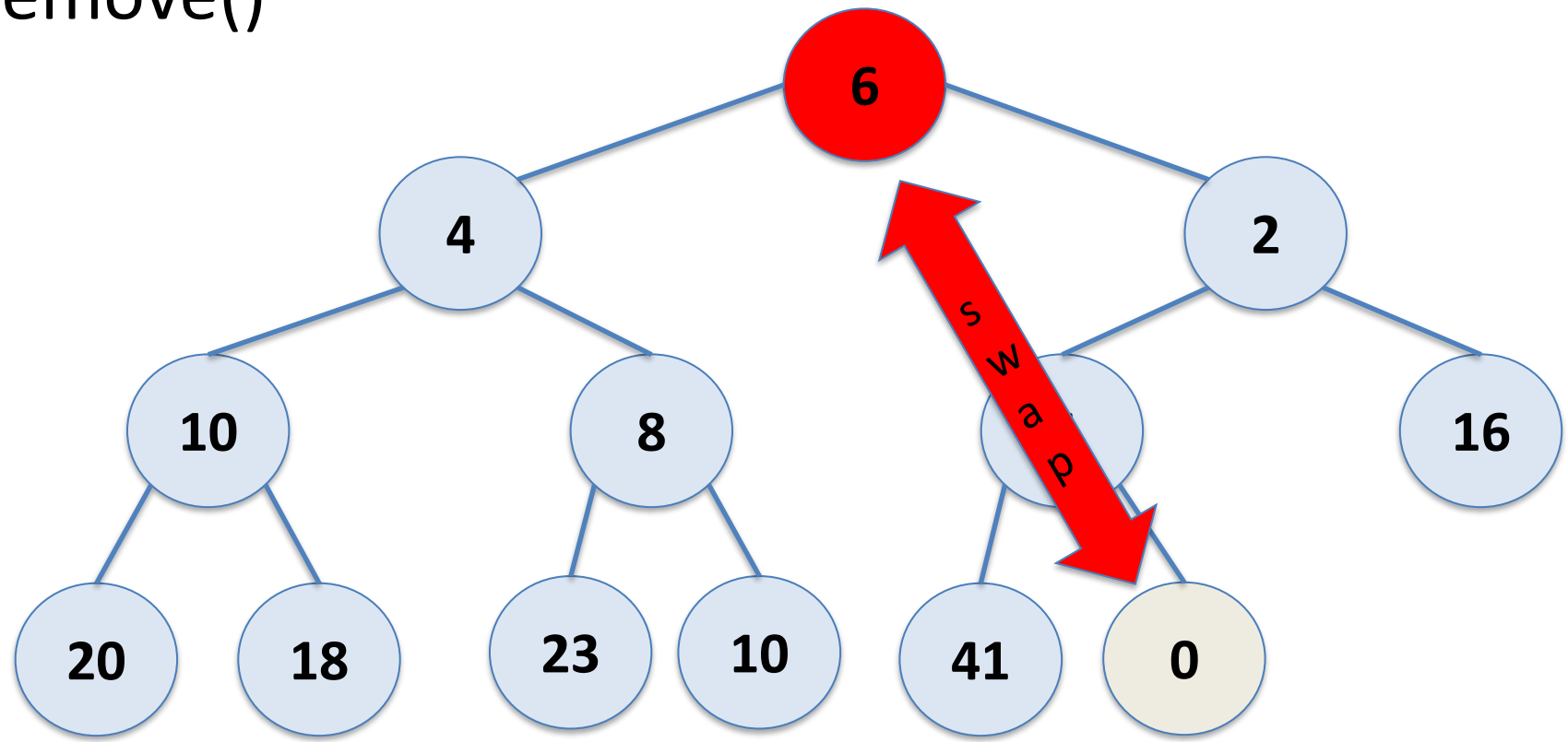
Binary Heap

remove()



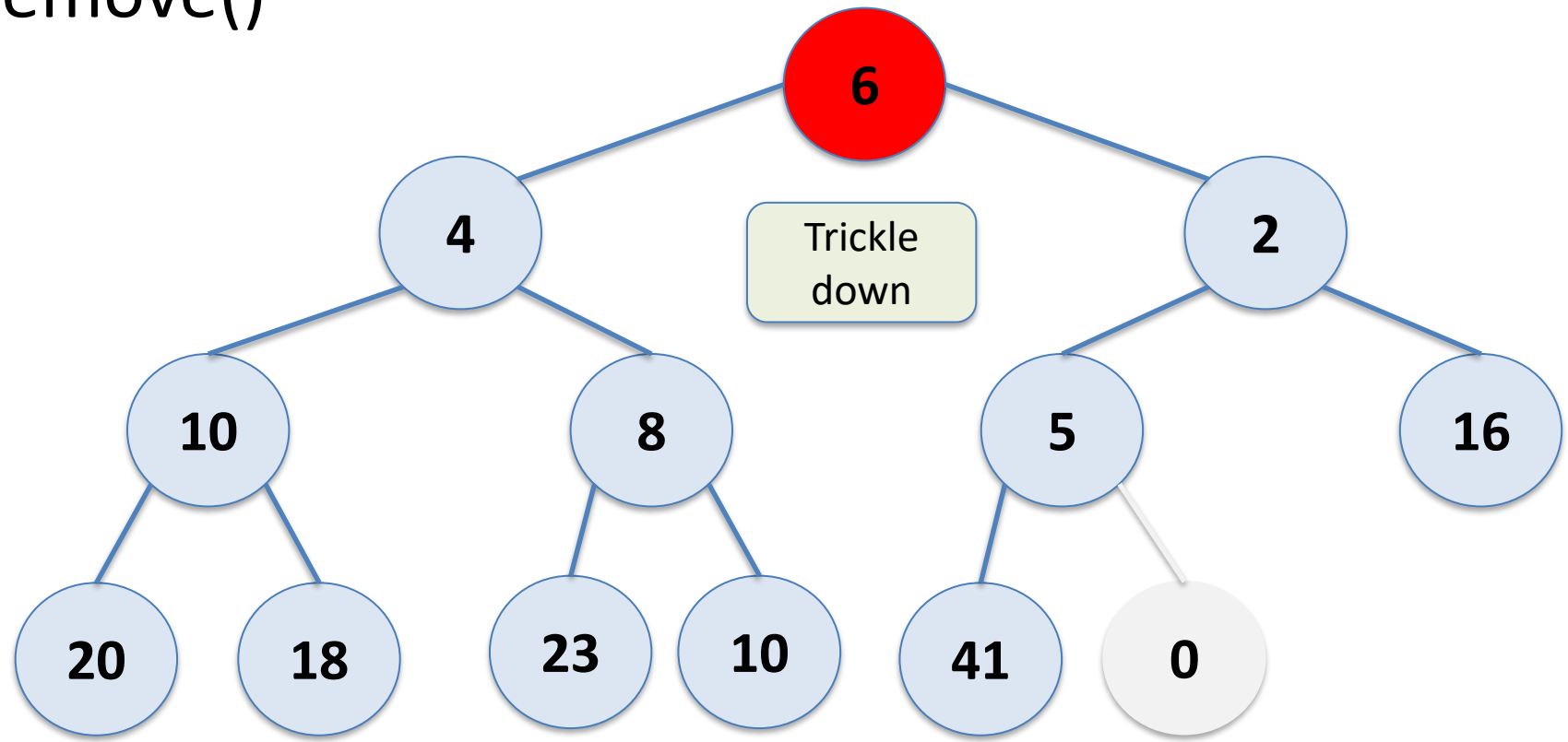
Binary Heap

remove()



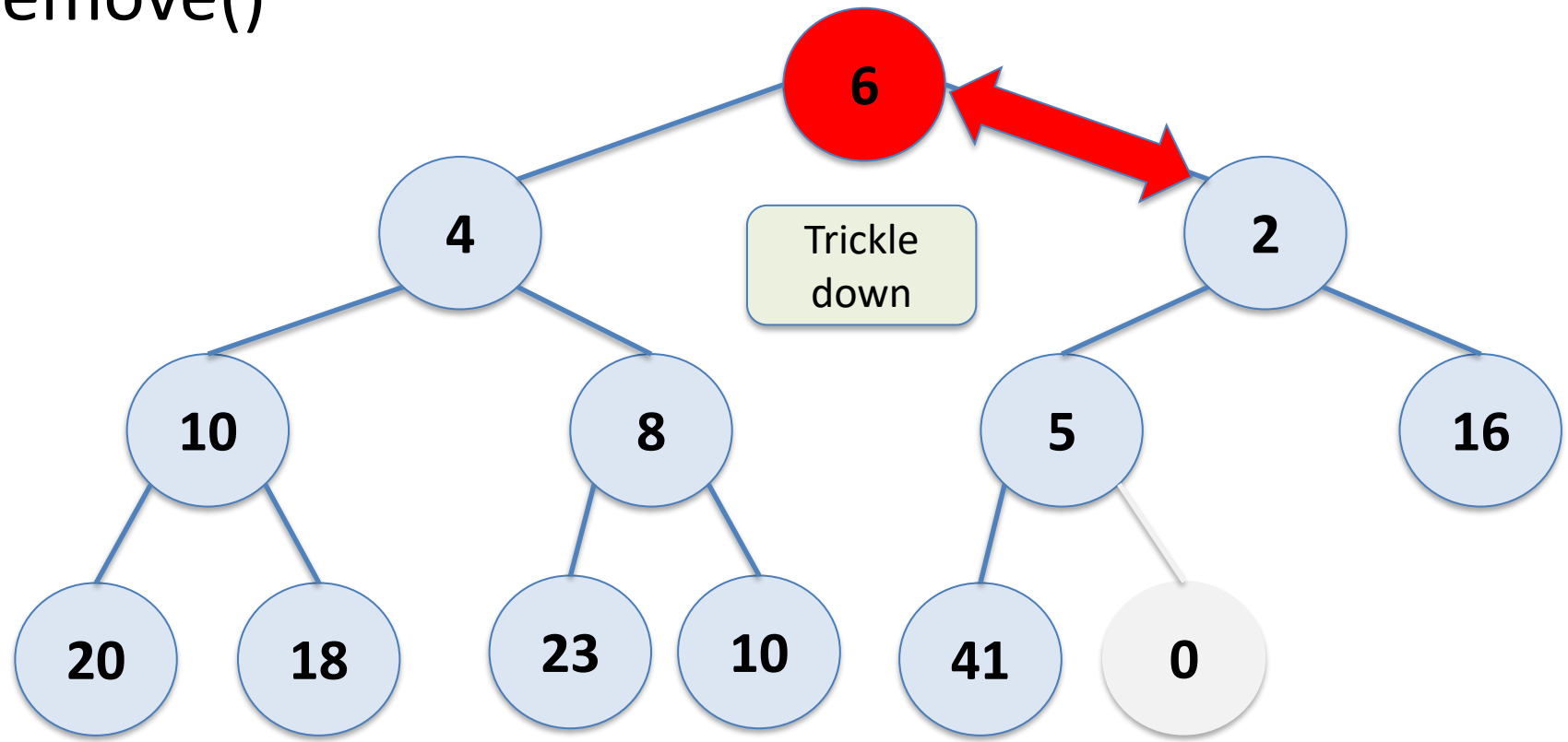
Binary Heap

remove()



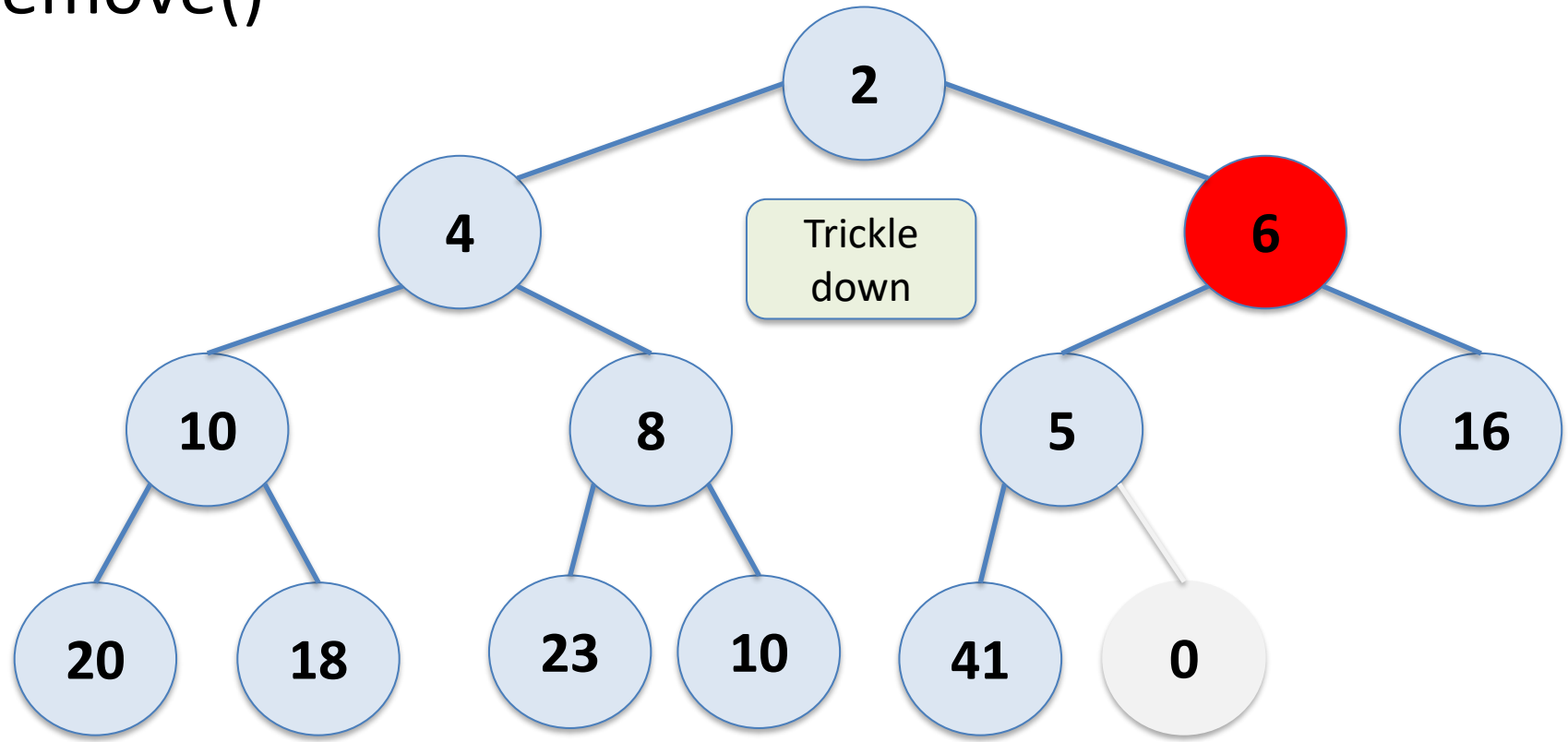
Binary Heap

remove()



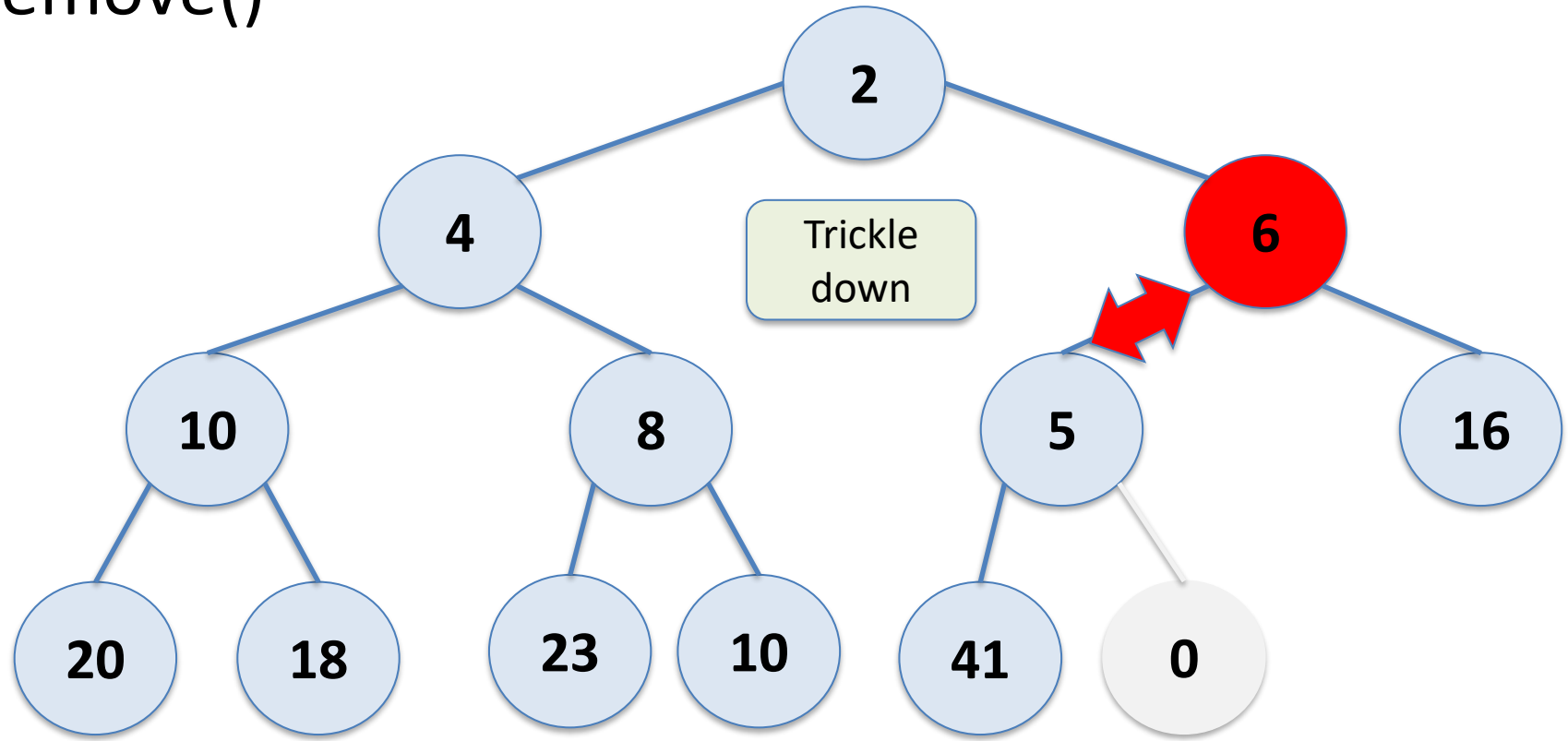
Binary Heap

remove()



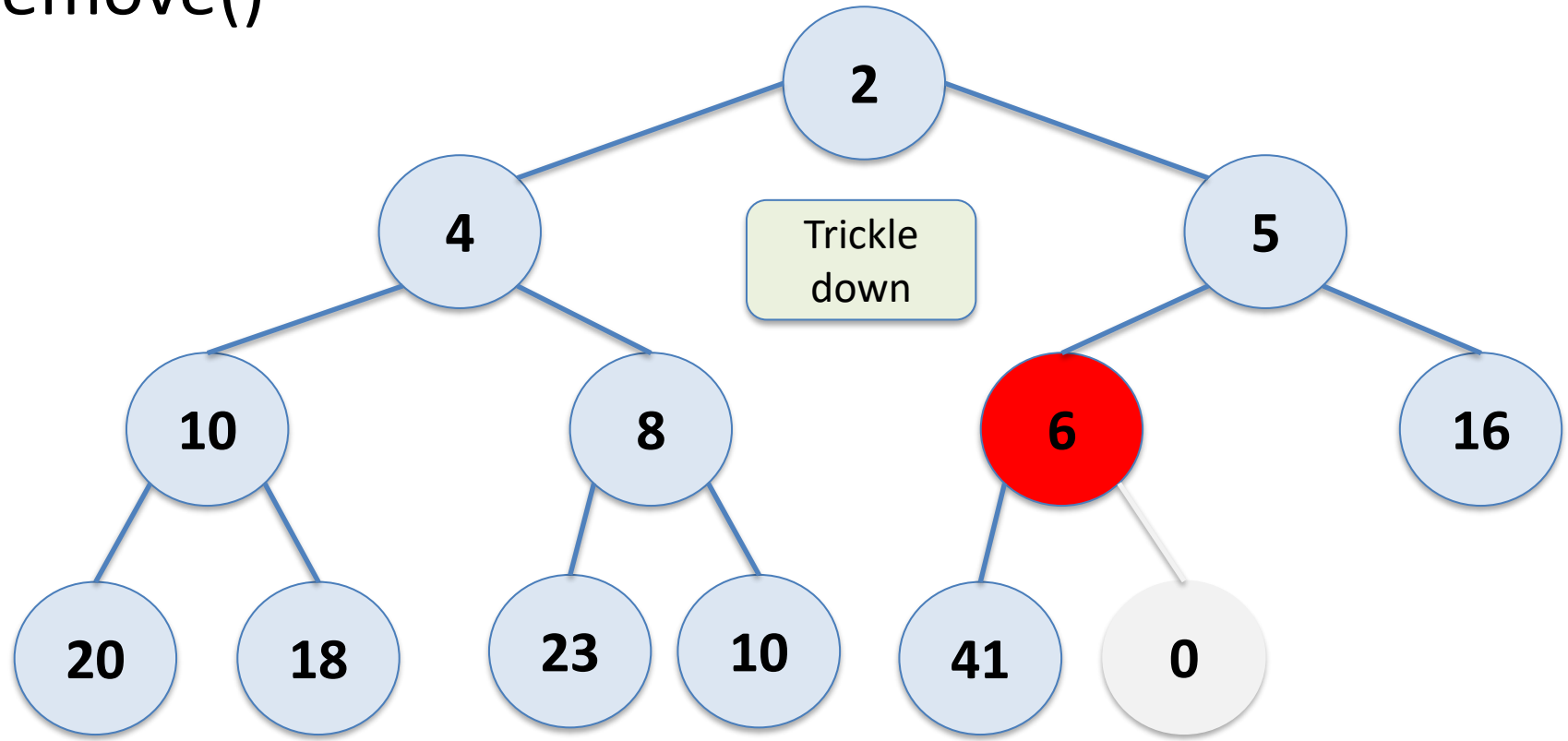
Binary Heap

remove()



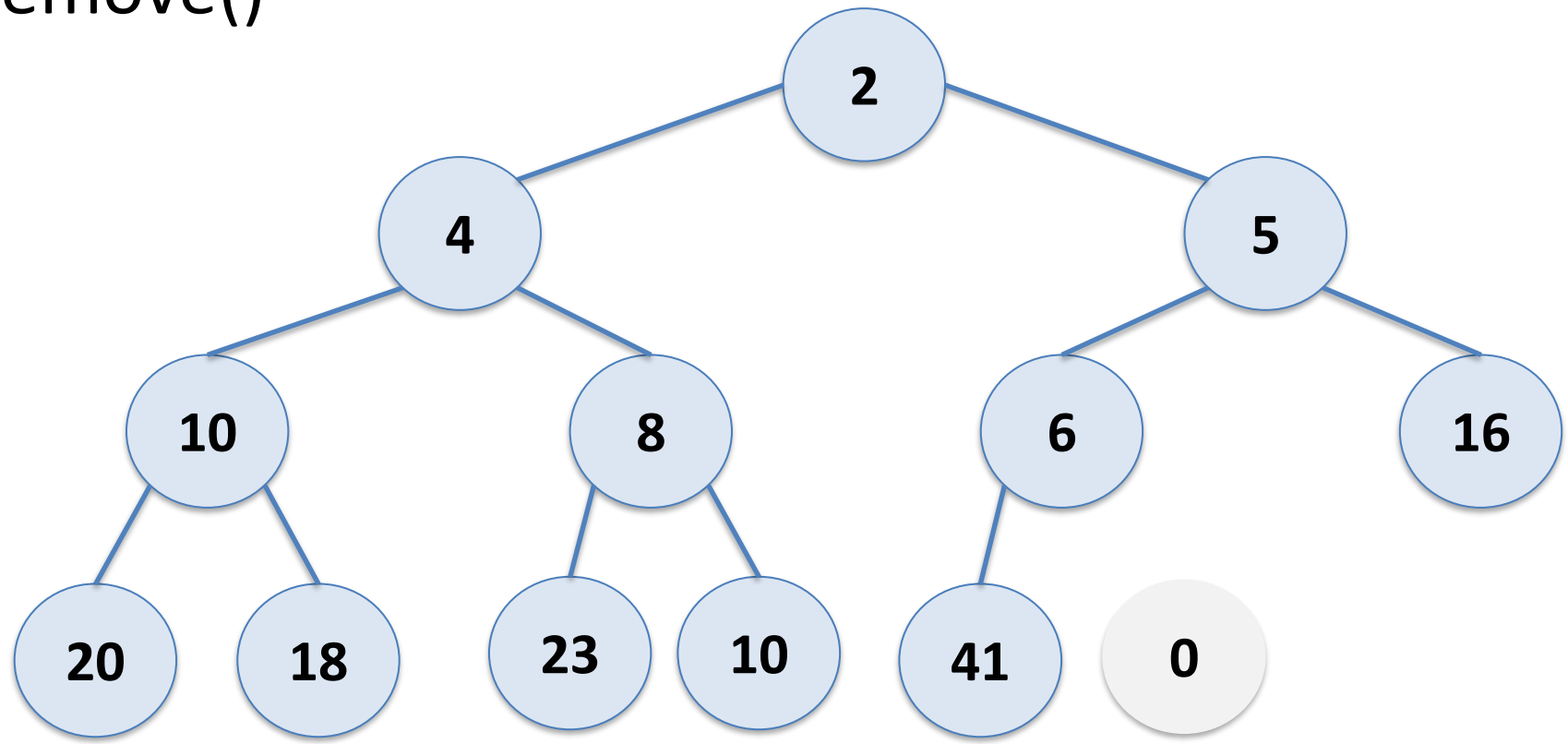
Binary Heap

remove()



Binary Heap

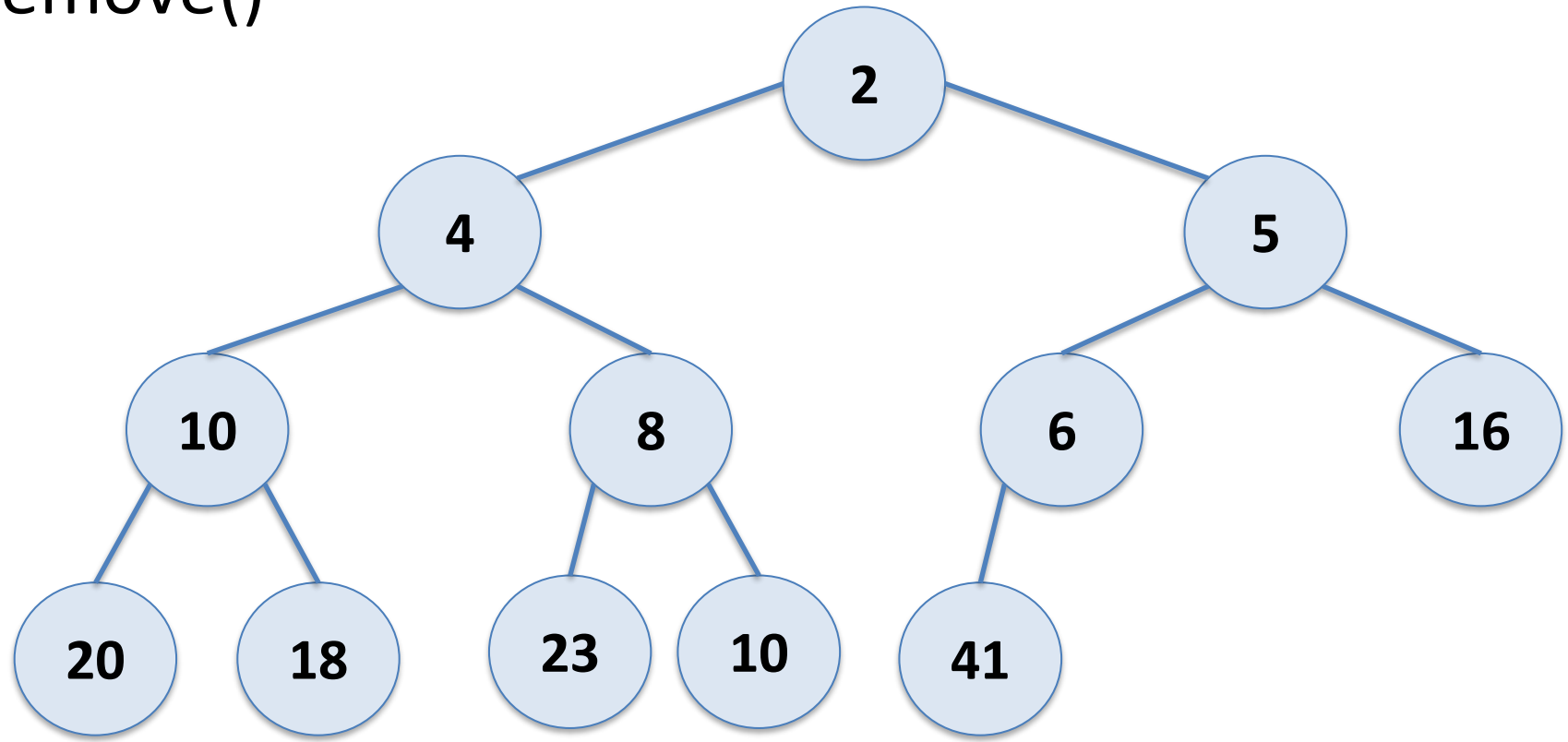
remove()



Return data from node 0

Binary Heap

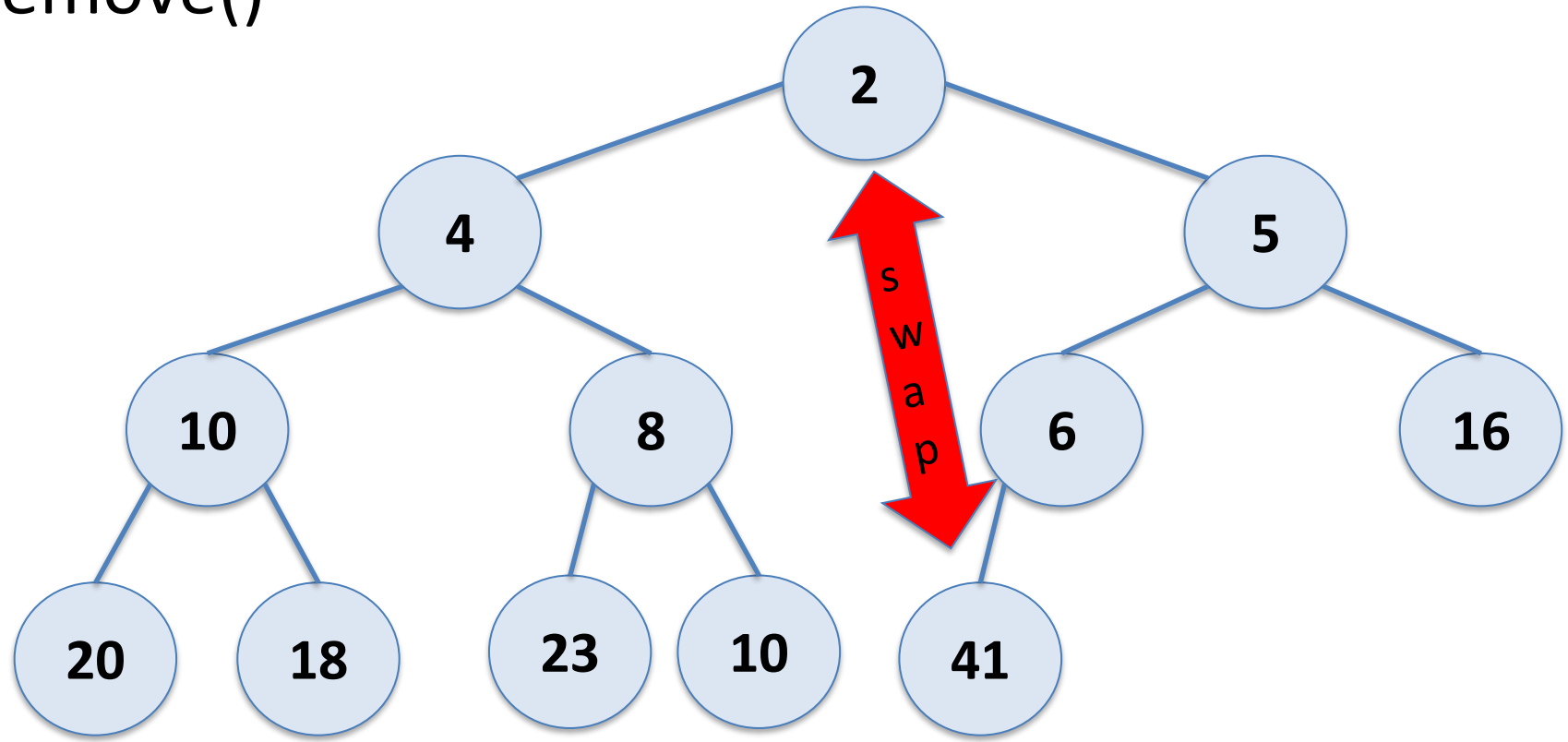
remove()



Let’s remove again

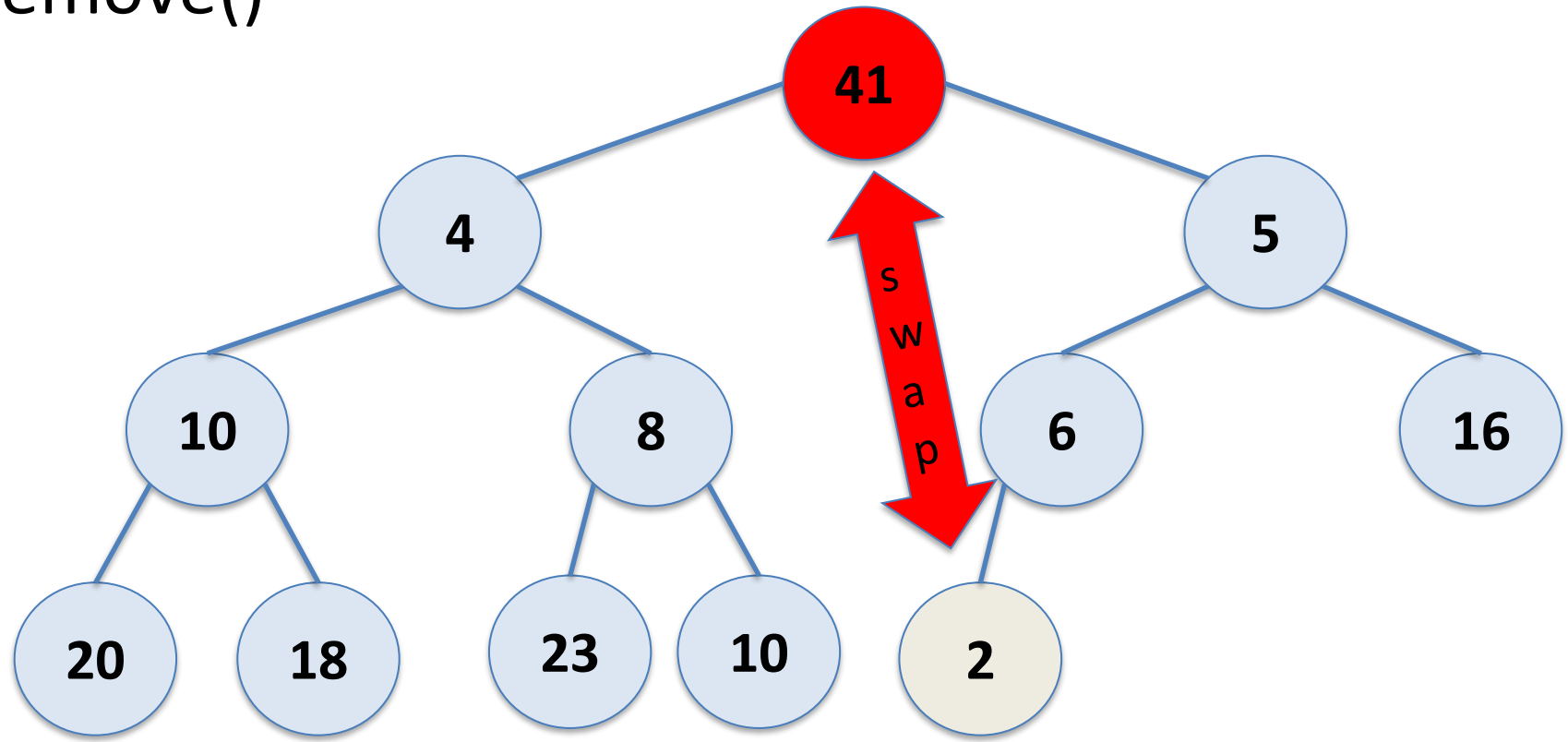
Binary Heap

remove()



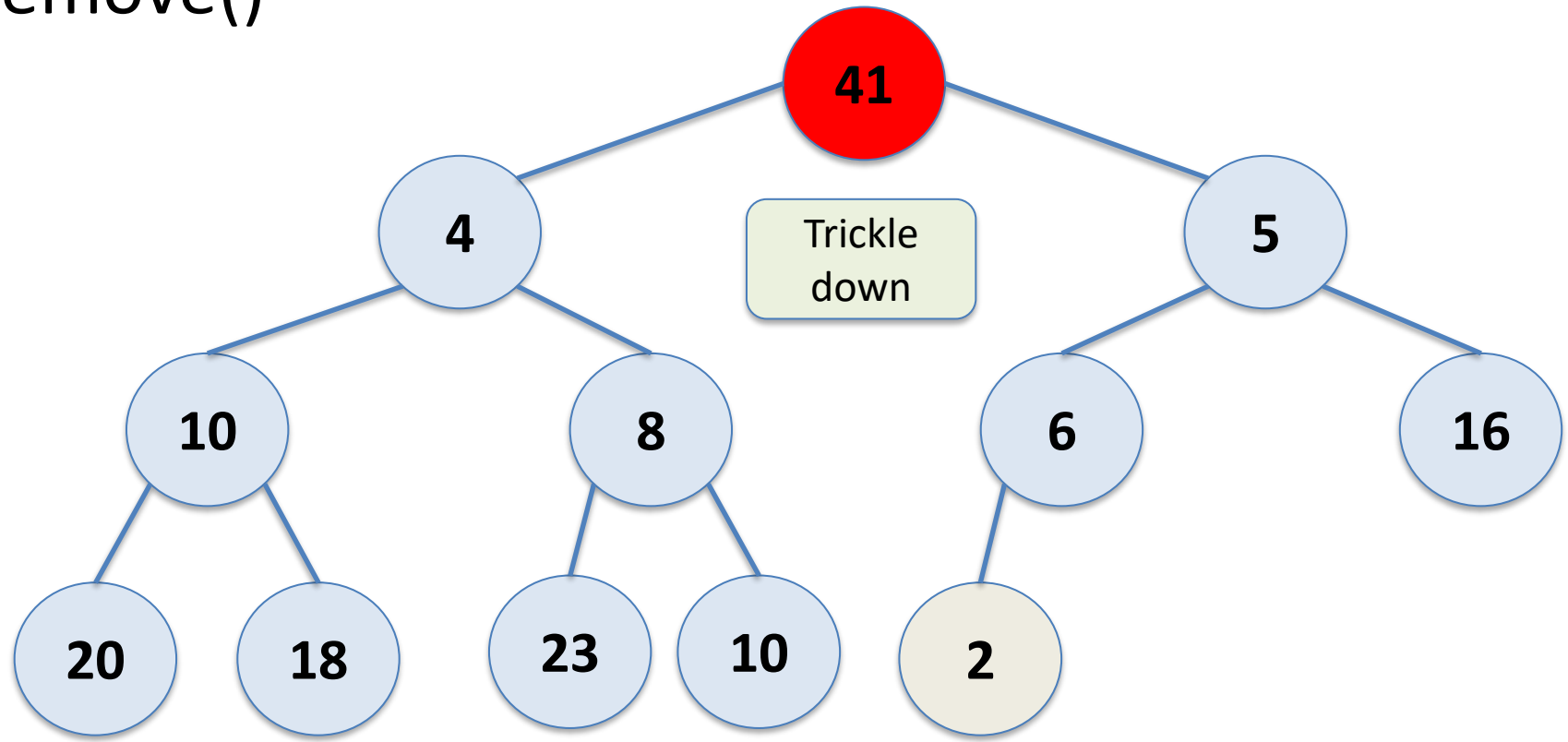
Binary Heap

remove()



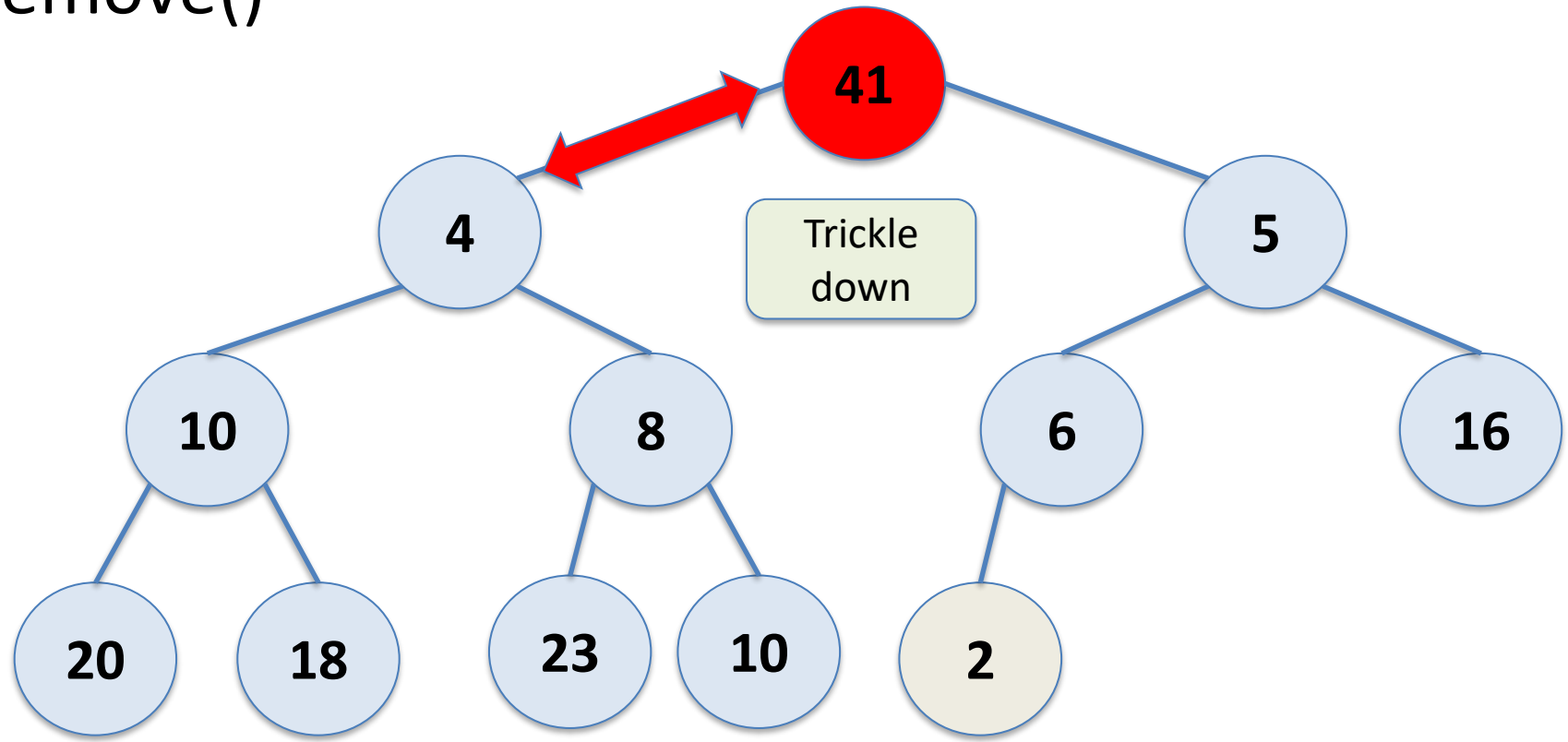
Binary Heap

remove()



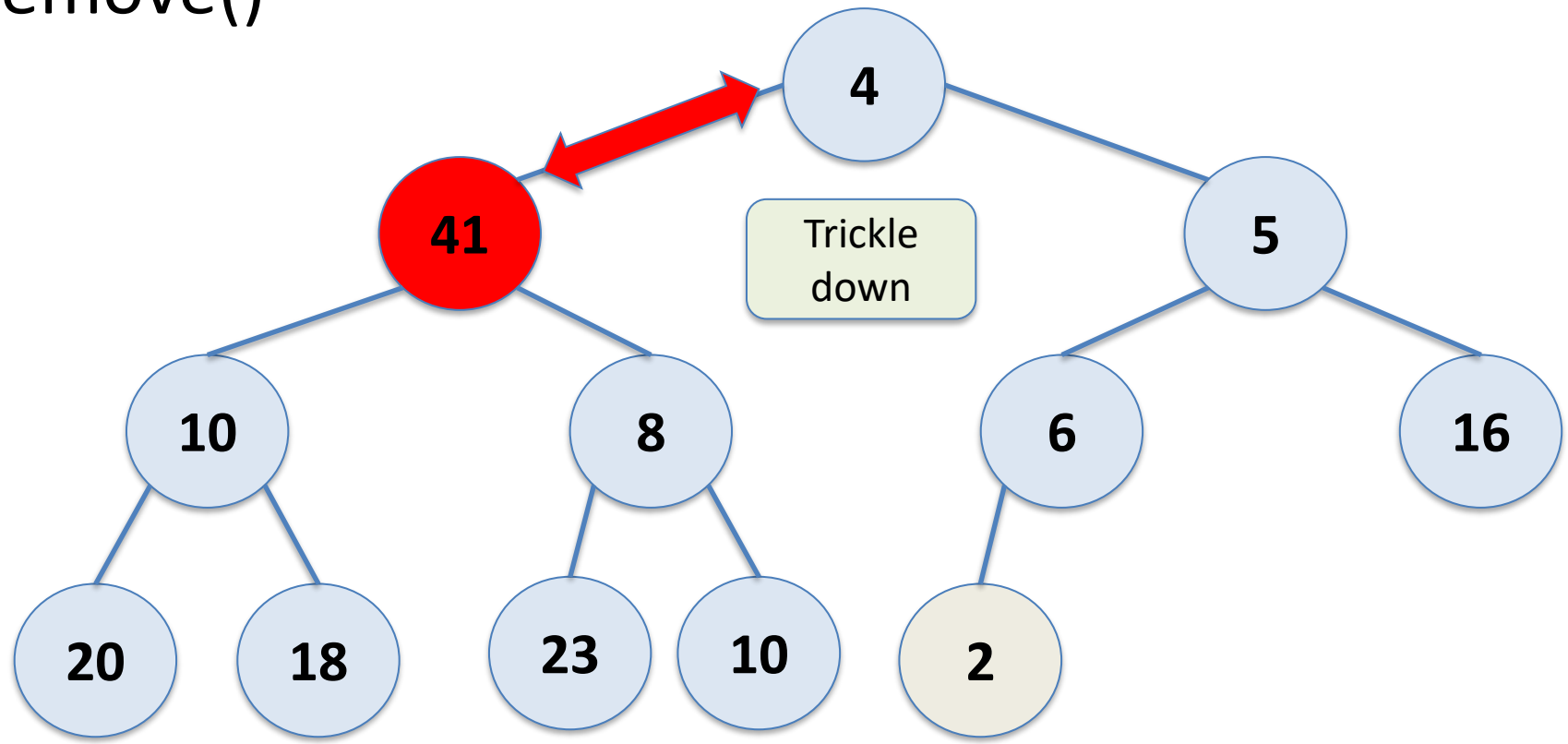
Binary Heap

remove()



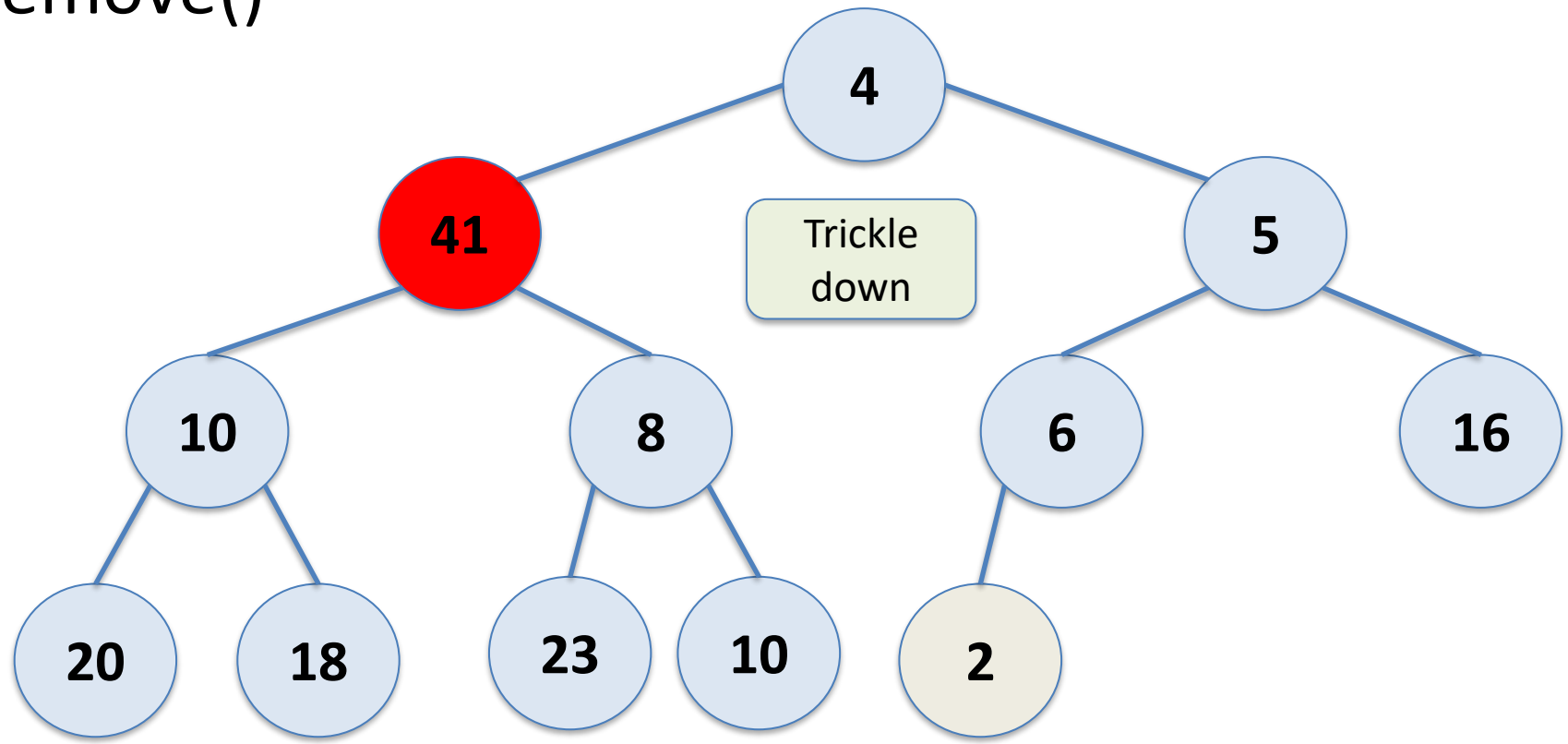
Binary Heap

remove()



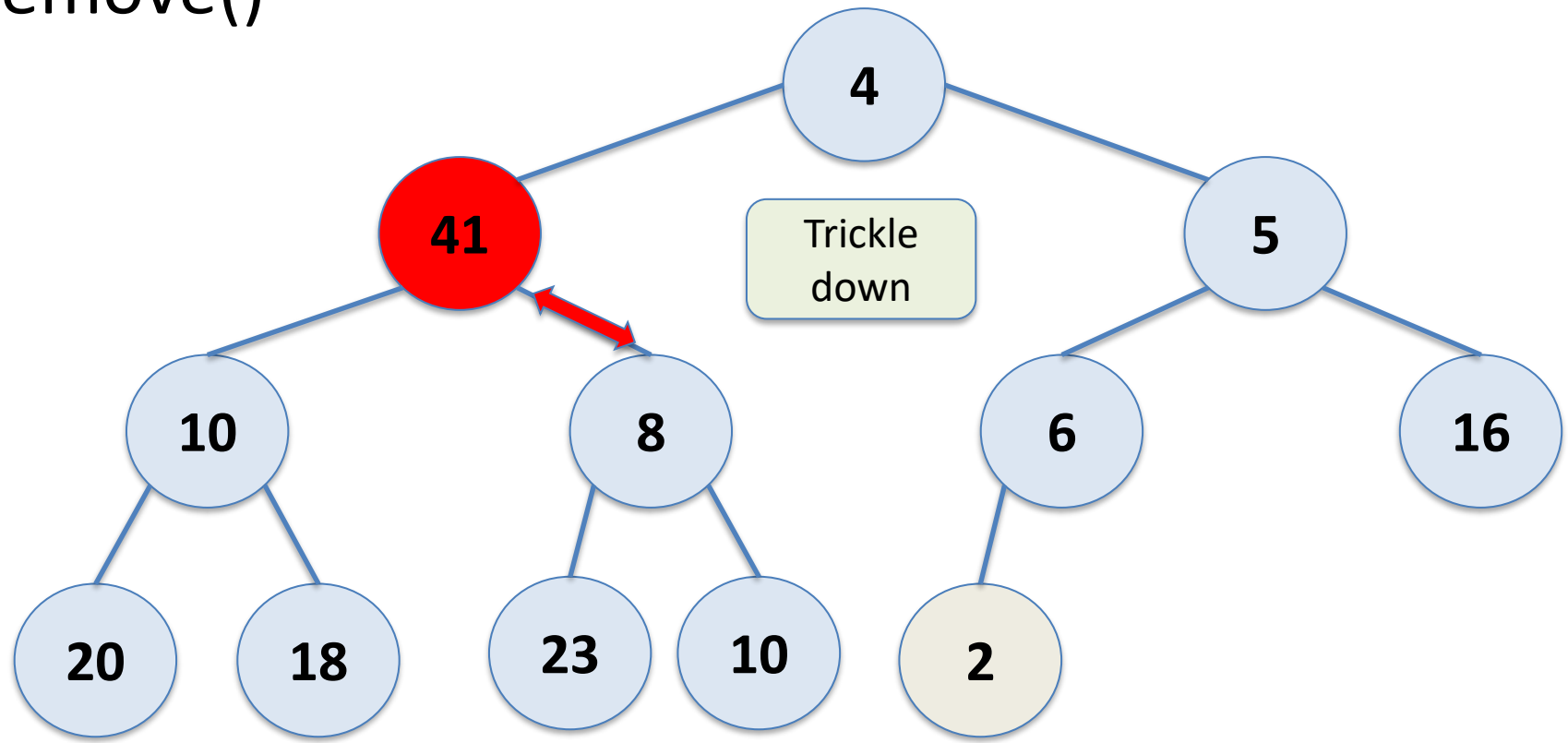
Binary Heap

remove()



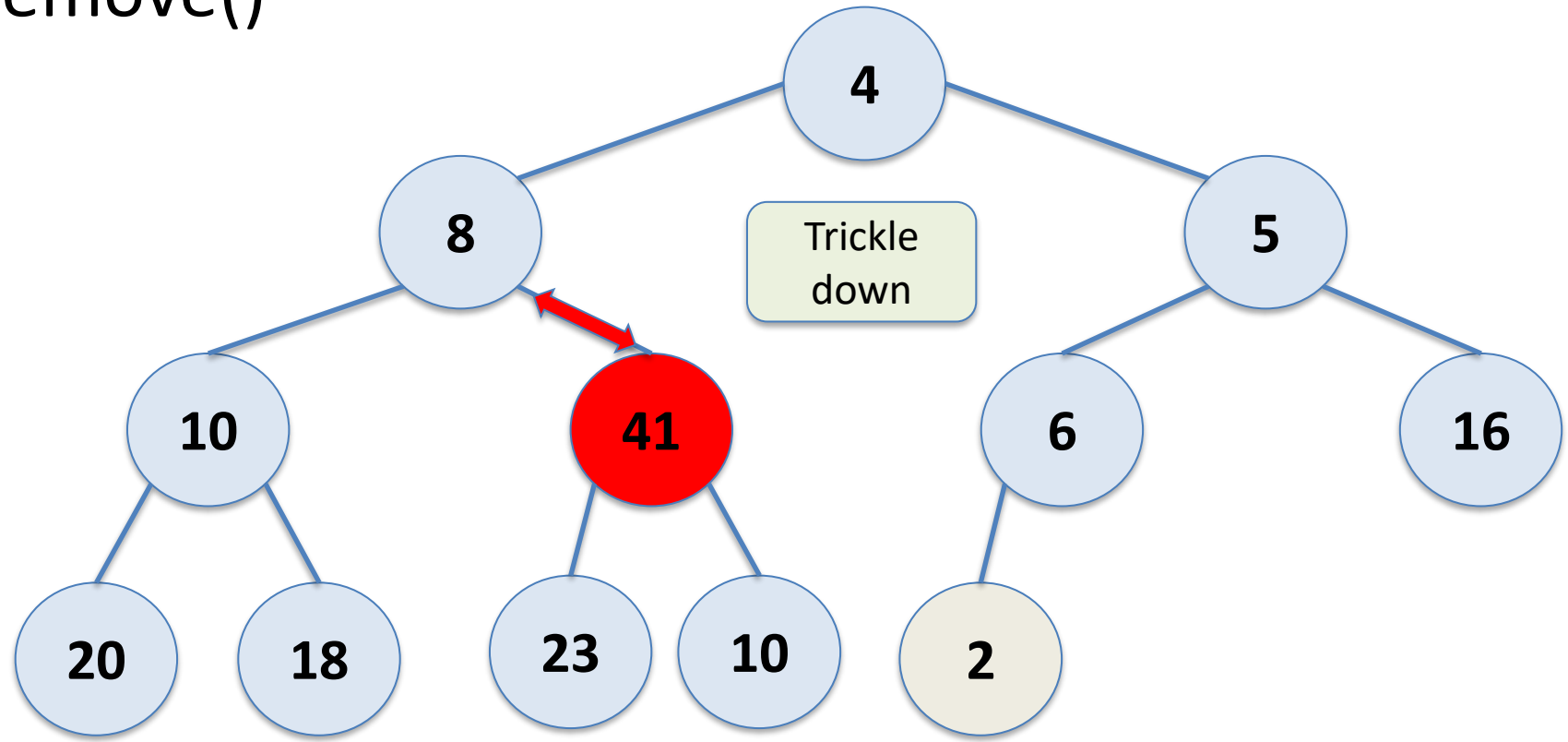
Binary Heap

remove()



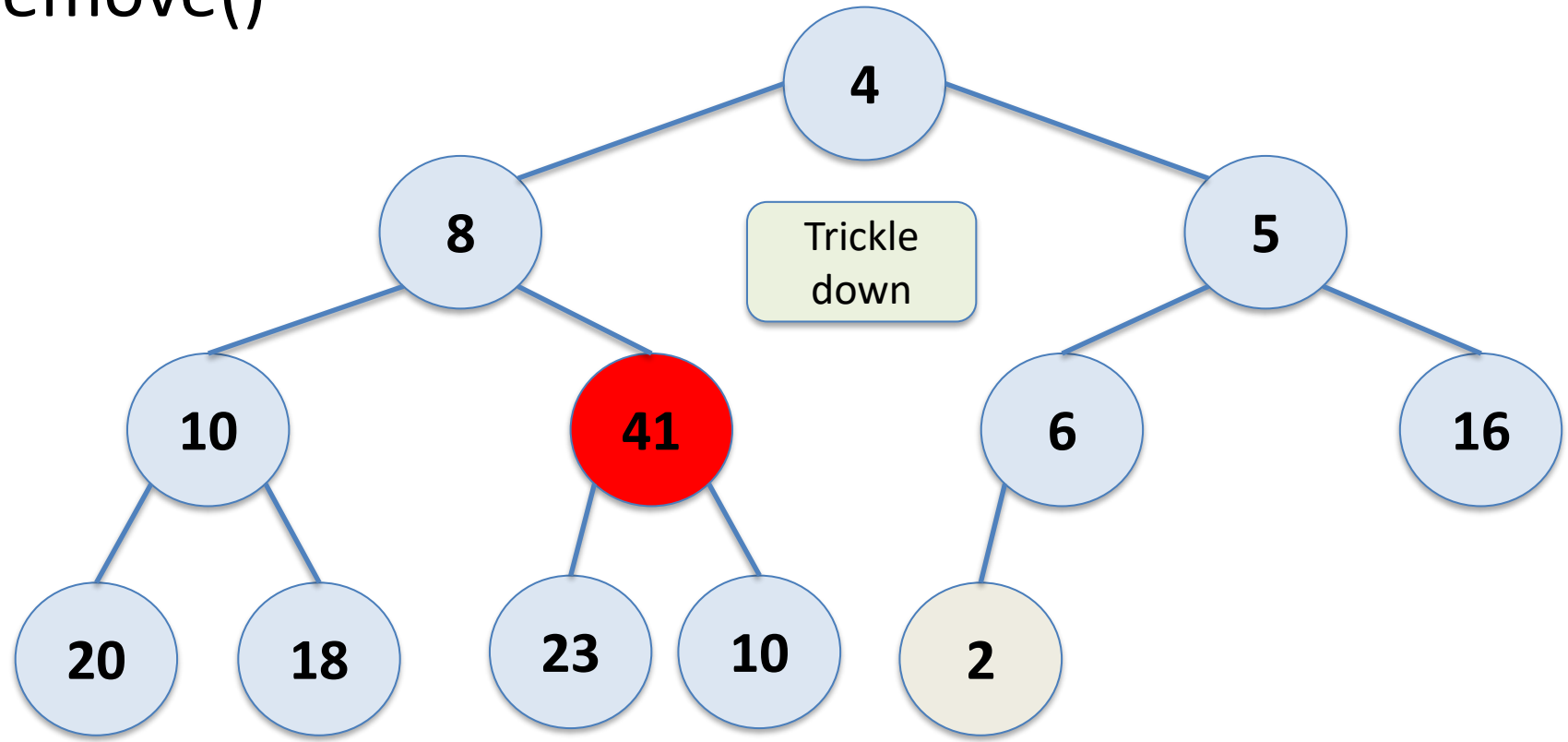
Binary Heap

remove()



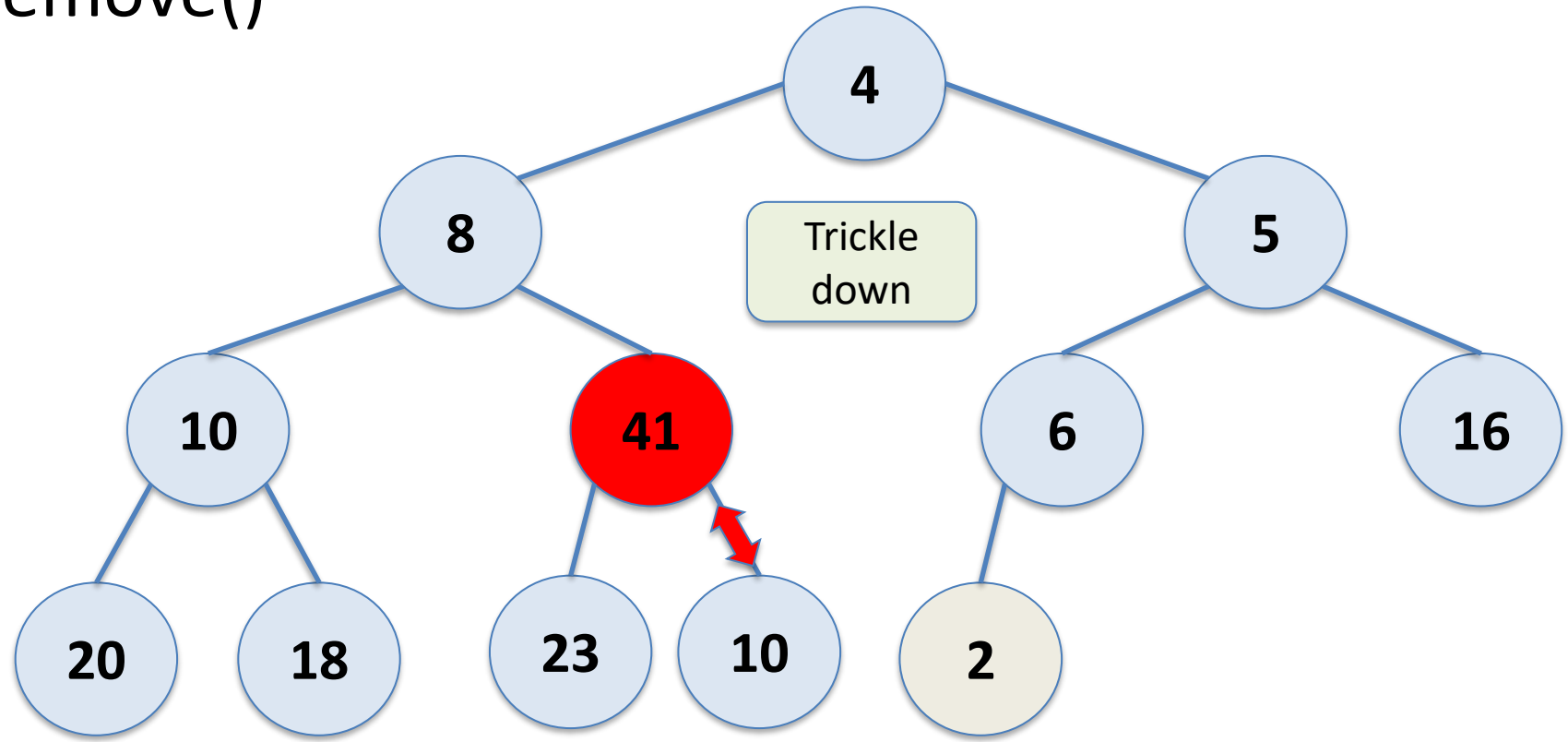
Binary Heap

remove()



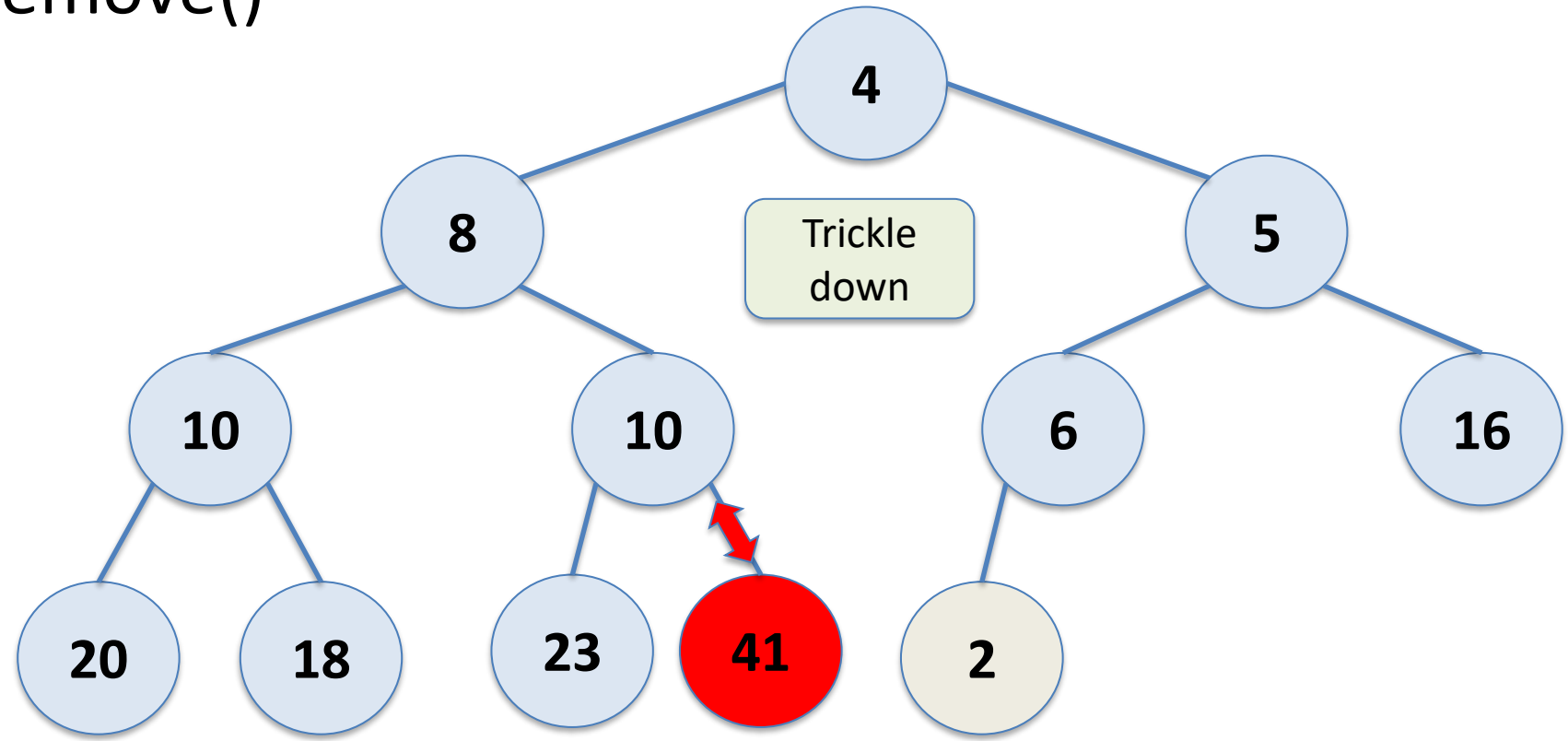
Binary Heap

remove()



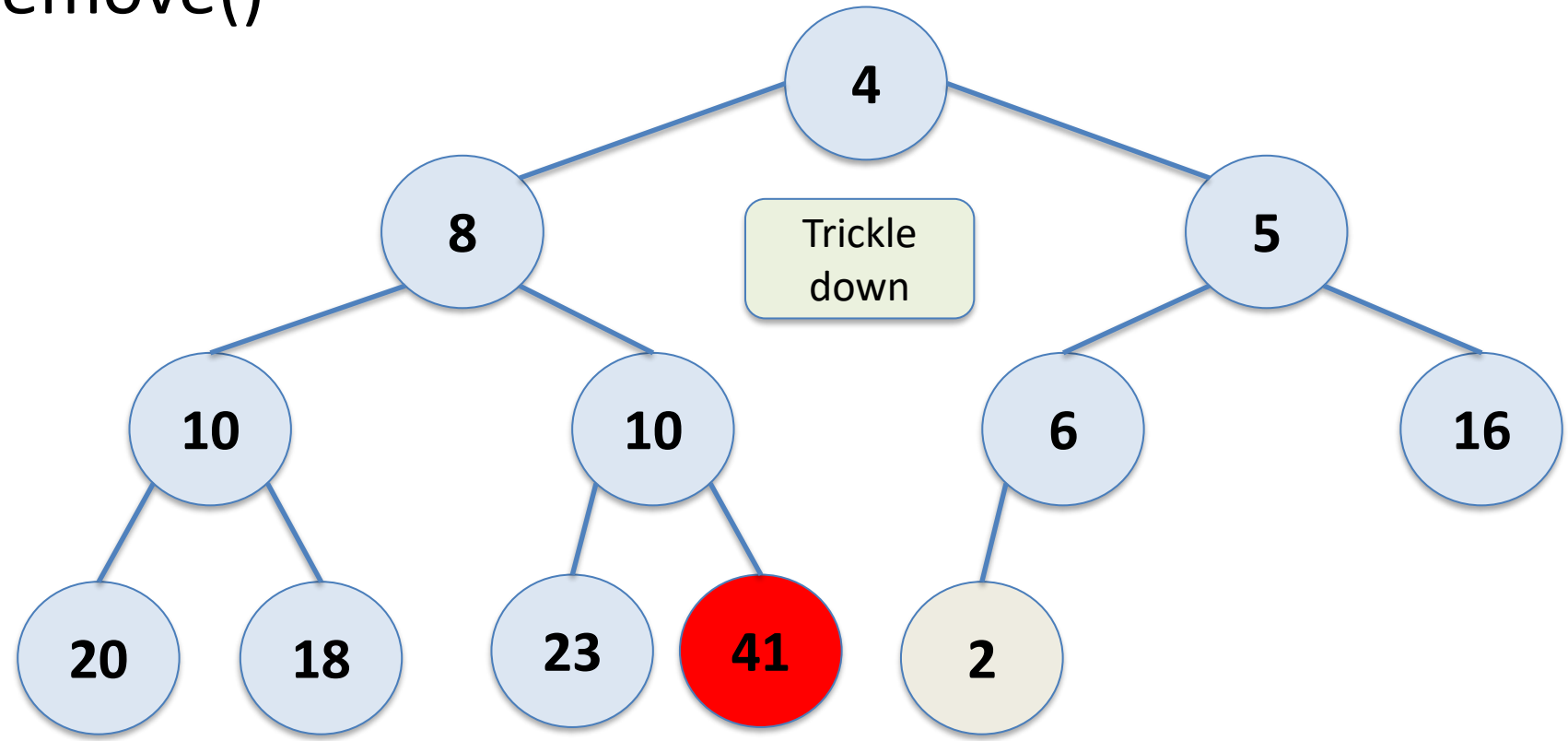
Binary Heap

remove()



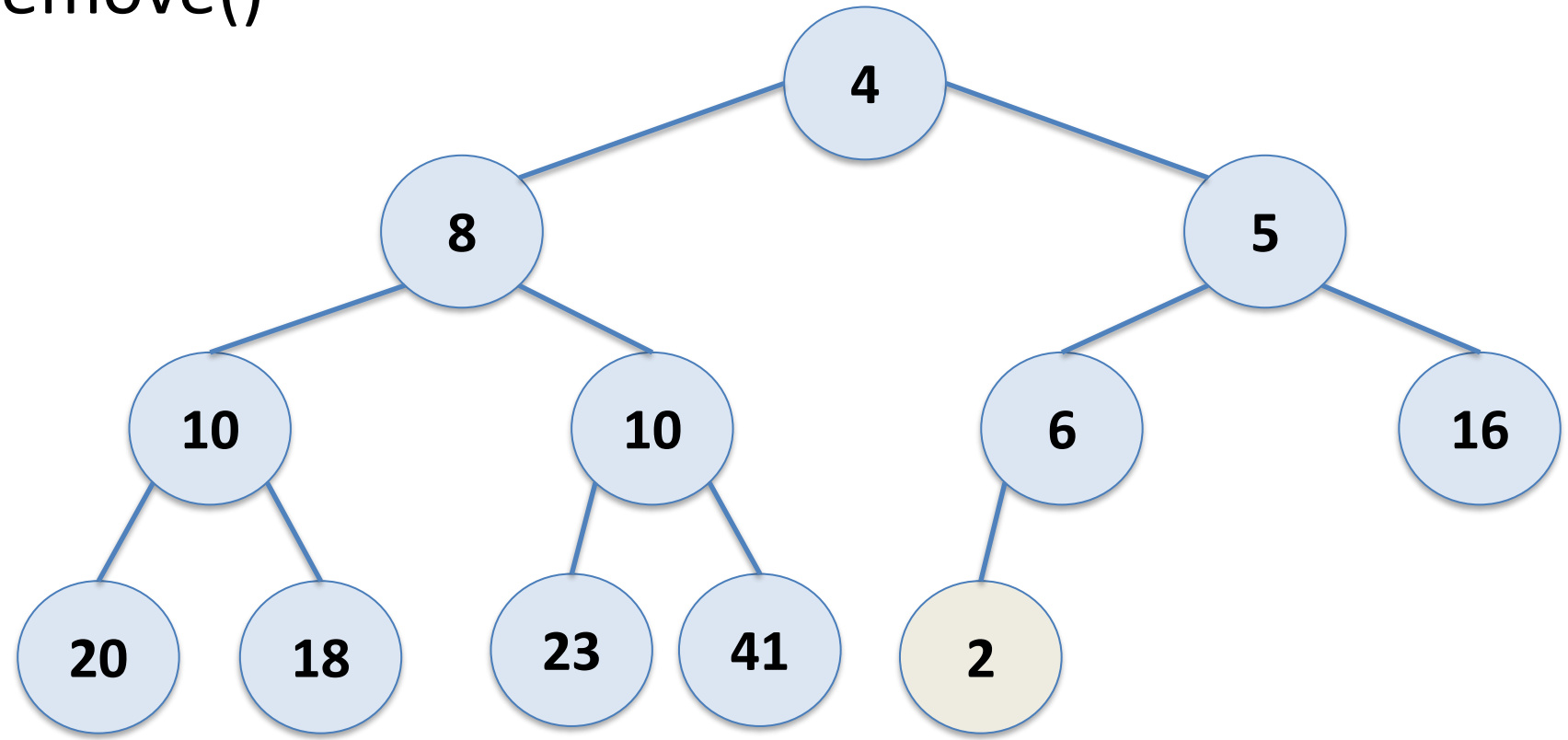
Binary Heap

remove()



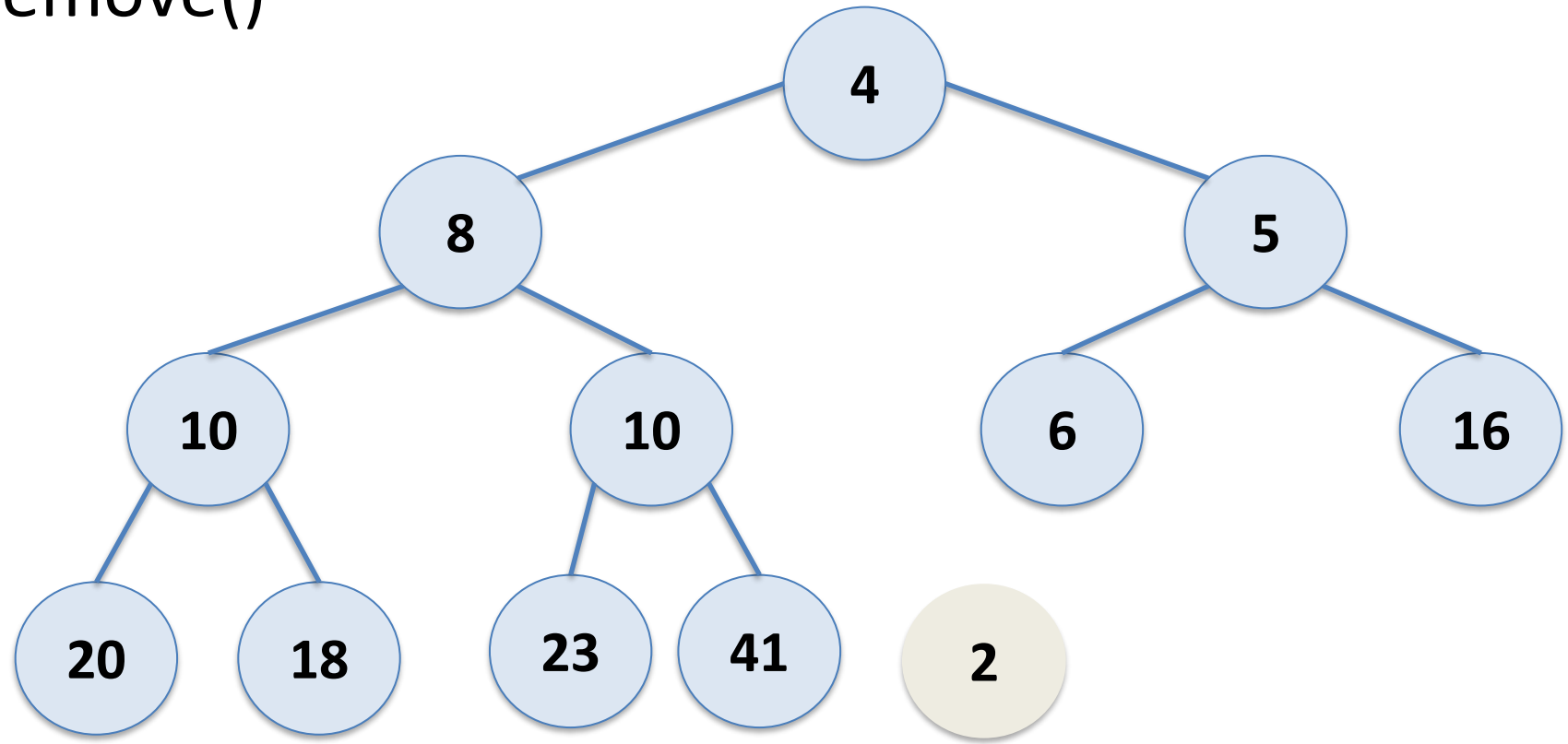
Binary Heap

remove()



Binary Heap

remove()



Return data from node 2

Heaps

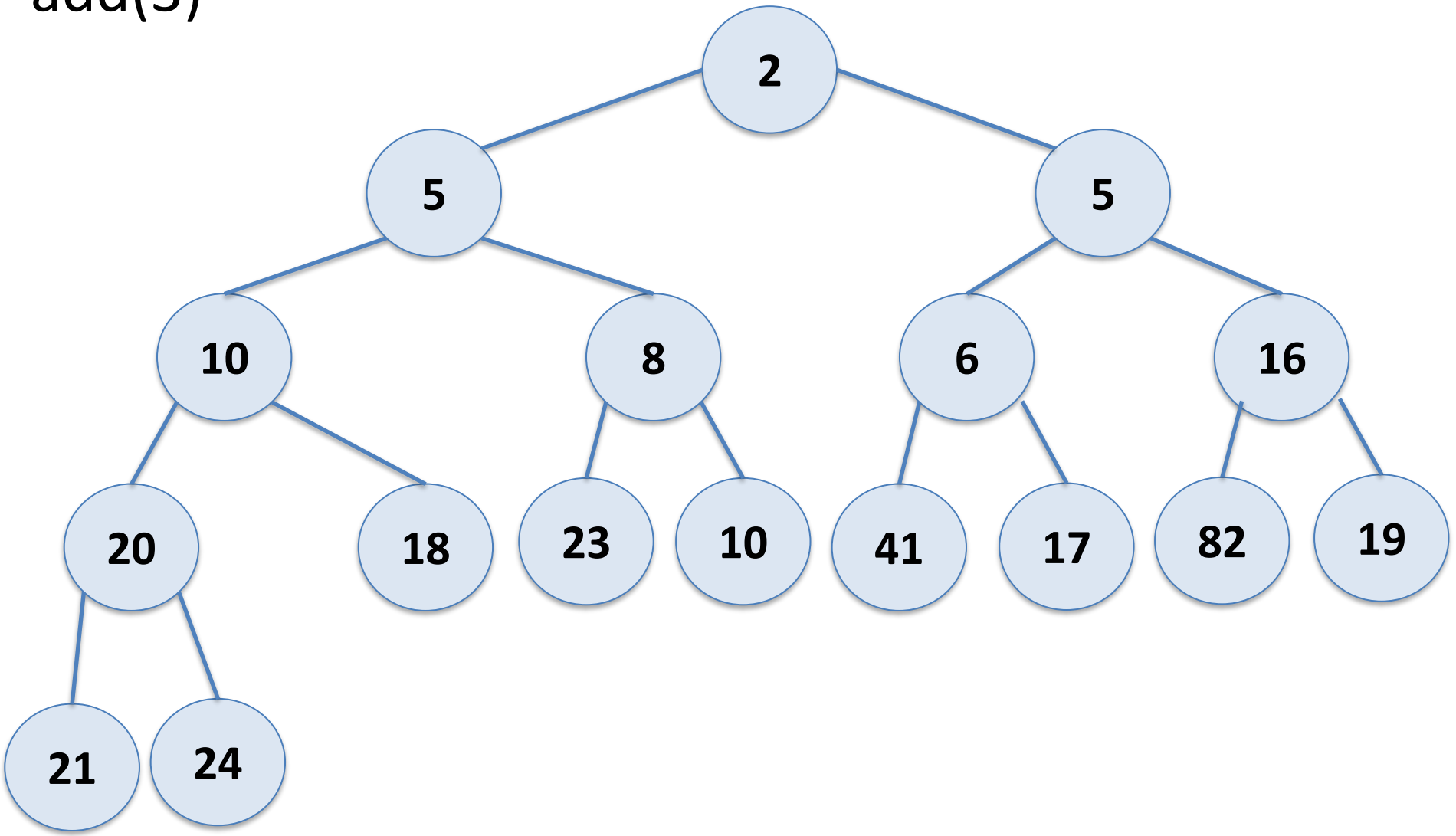
`add(value, priority)`

- add a new node in next available place in the tree

- **bubble up** until heap property is satisfied
  - Repeatedly swap the node its parent if needed to maintain heap property

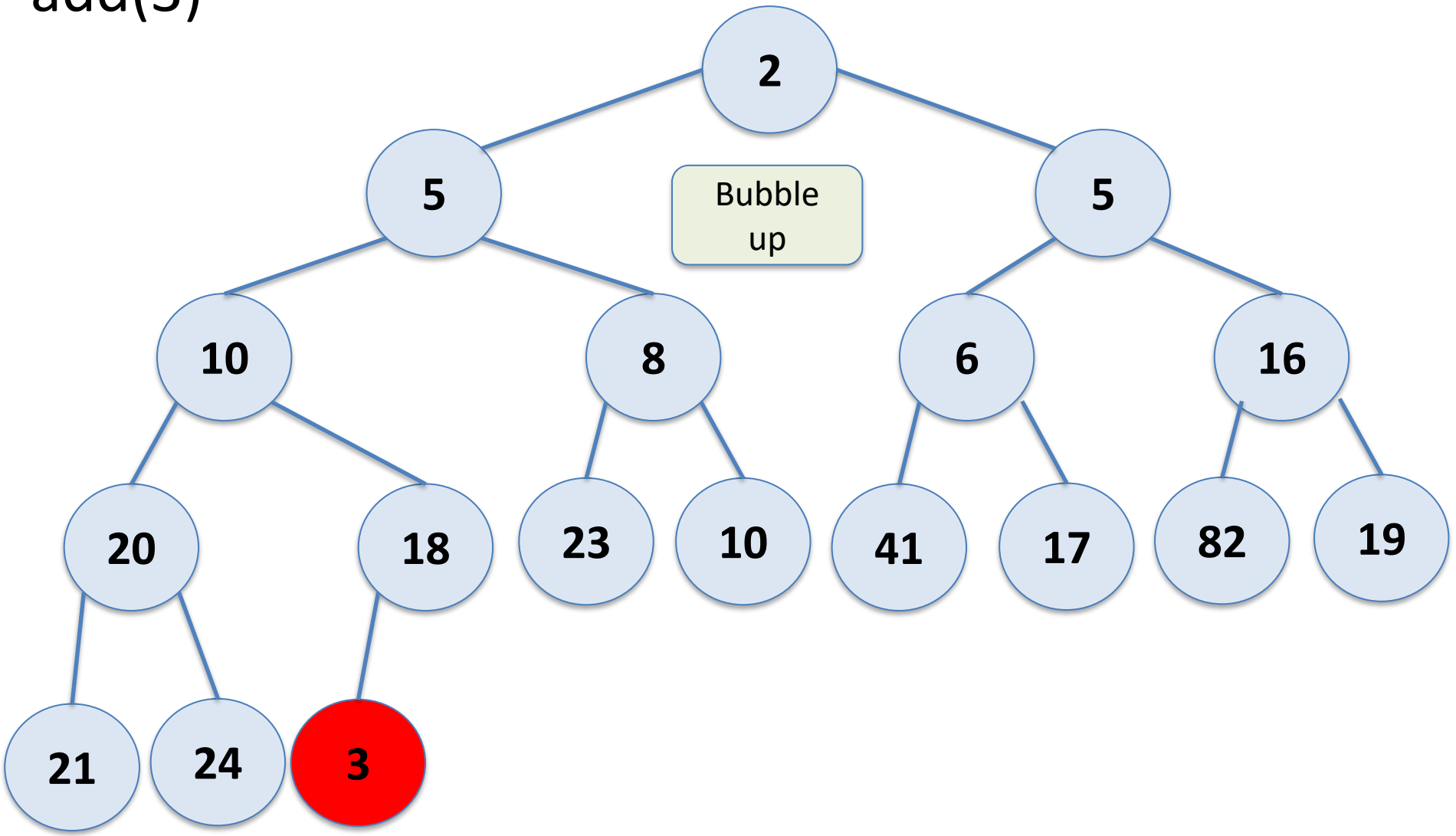
Binary Heap

add(3)



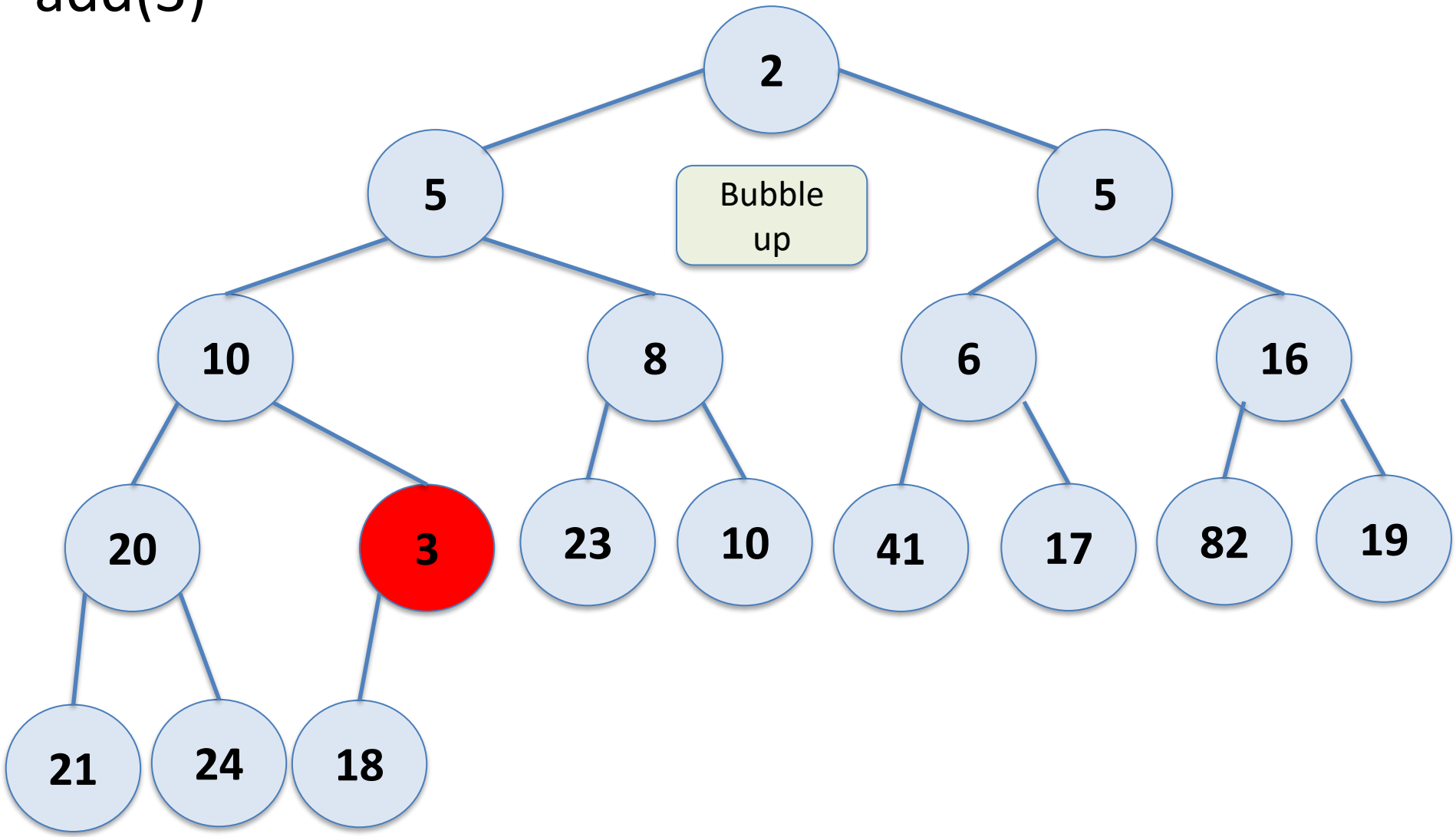
Binary Heap

add(3)



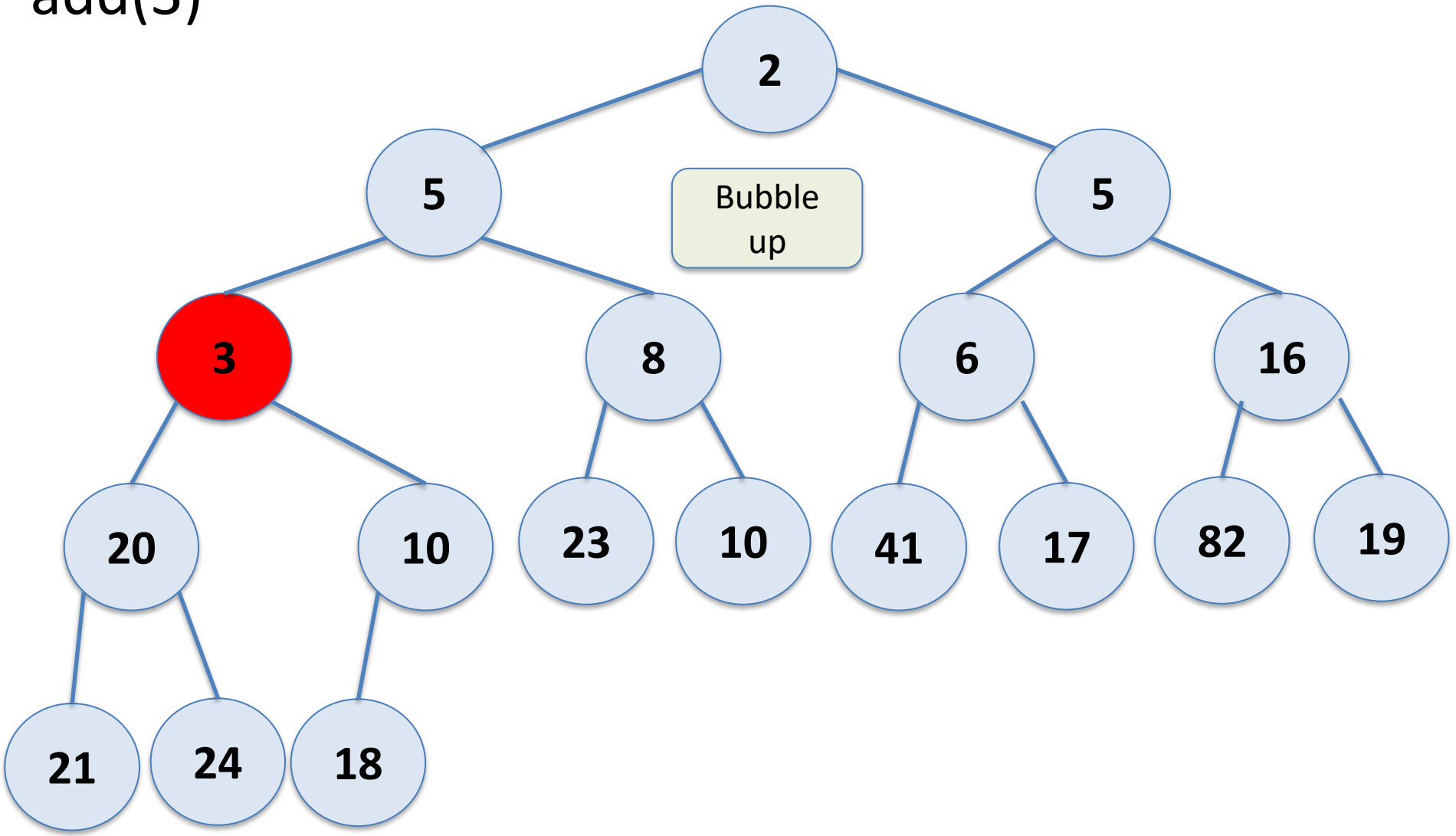
Binary Heap

add(3)



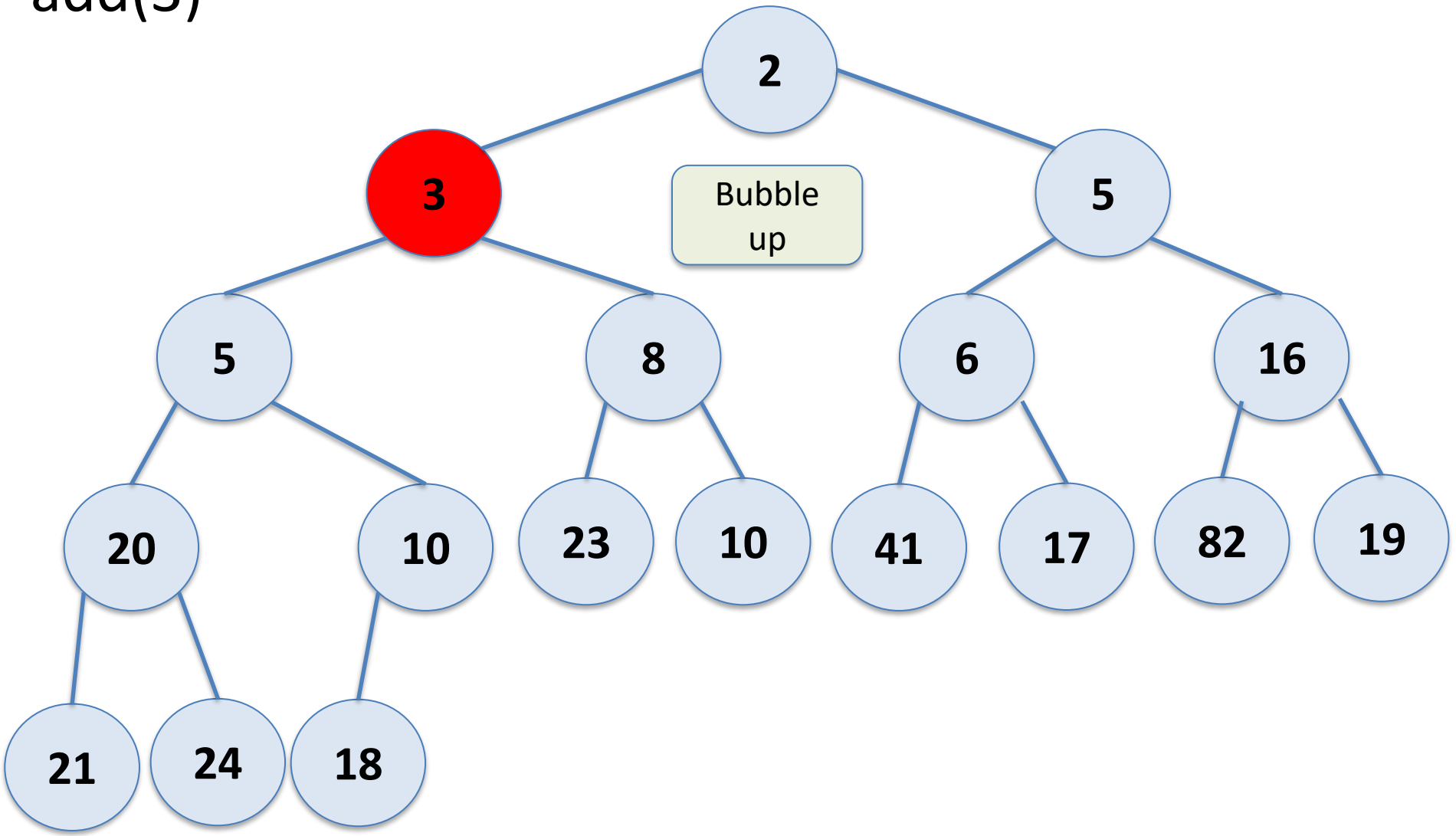
Binary Heap

add(3)



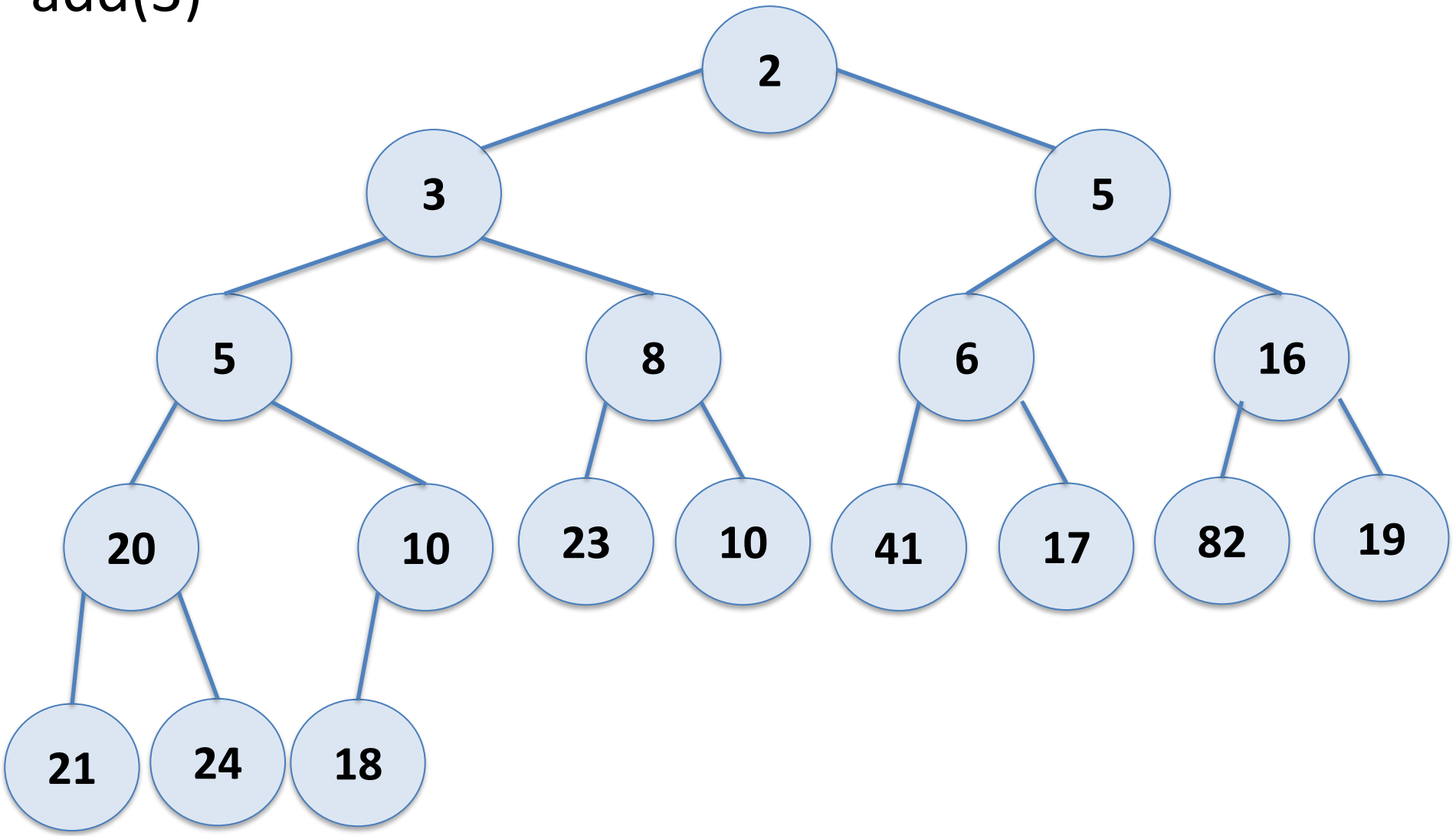
Binary Heap

add(3)



Binary Heap

add(3)



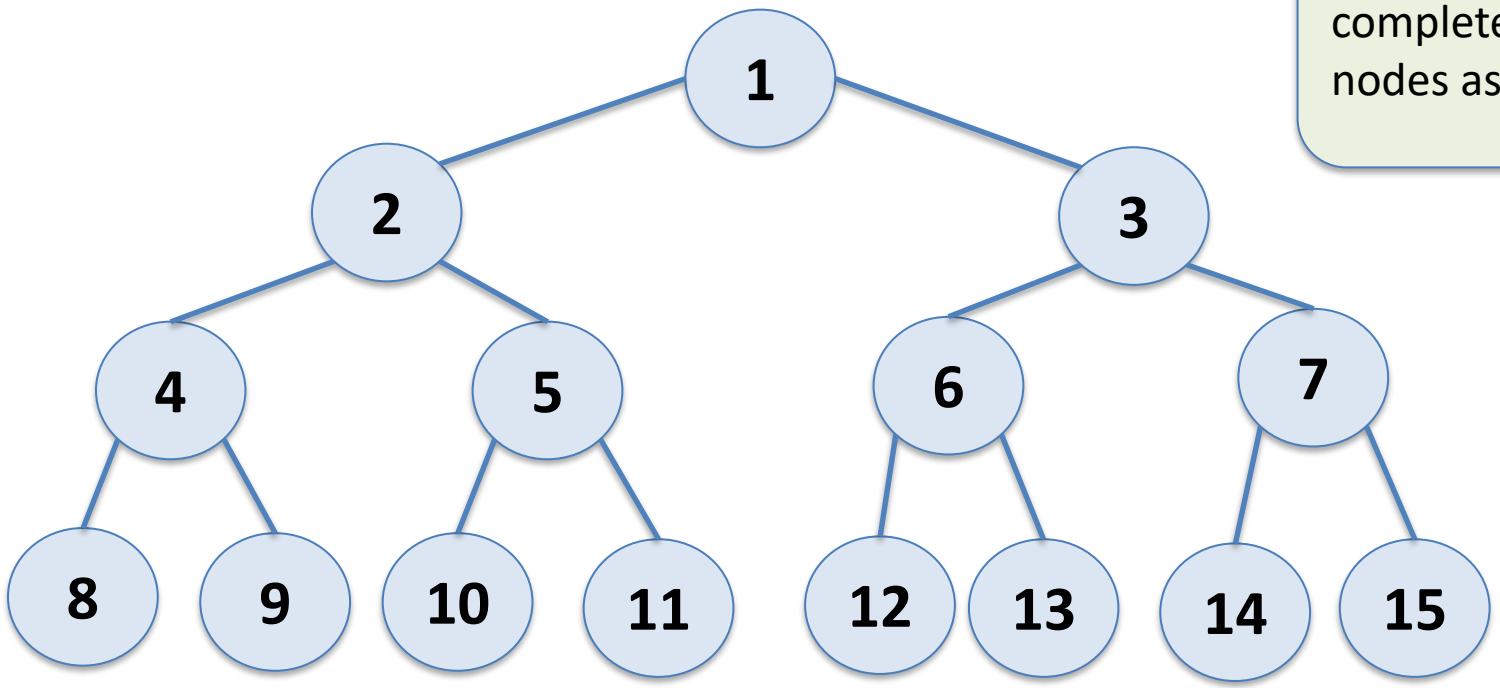
Binary Heap

How do we find the last node in the last level?

Binary Heap

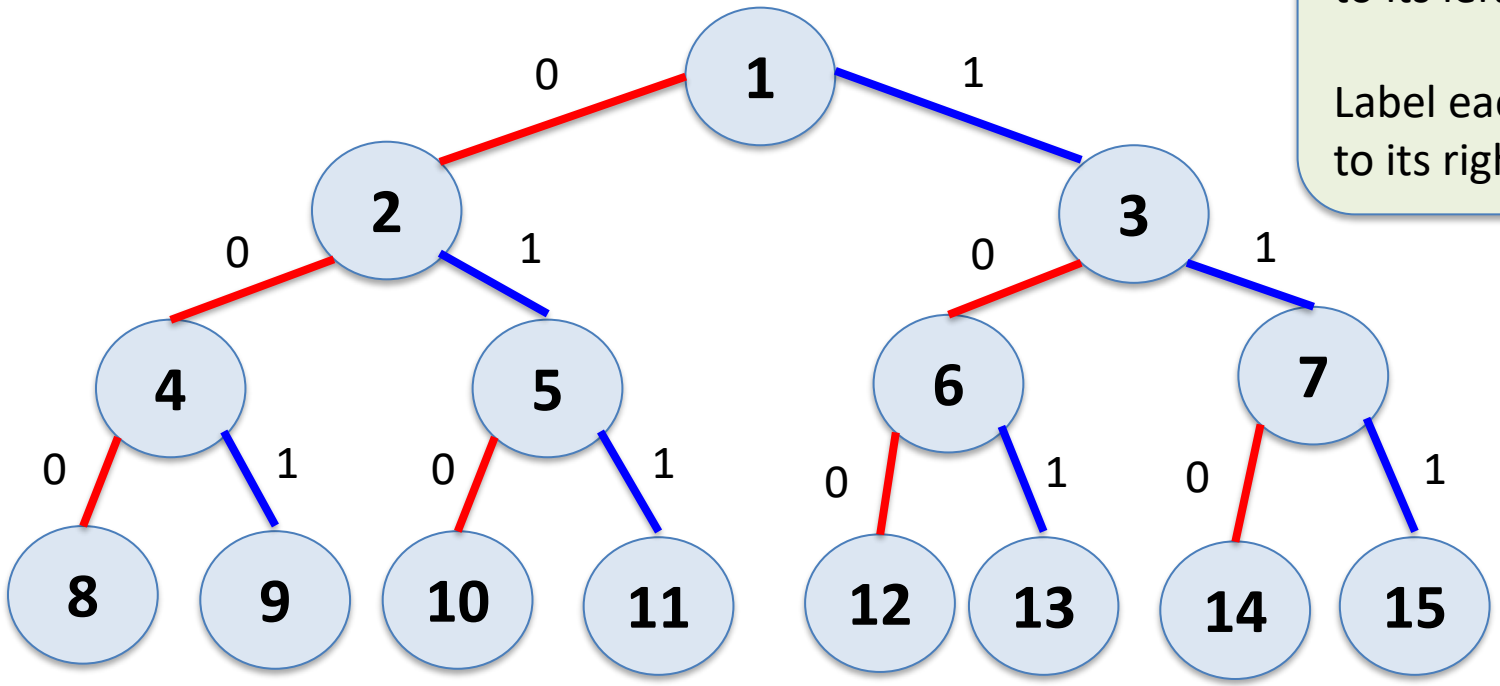
How do we find the last node in the last level?

Label the nodes 1 to n for a complete binary tree with n nodes as shown. (BFS order)



Binary Heap

How do we find the last node in the last level?

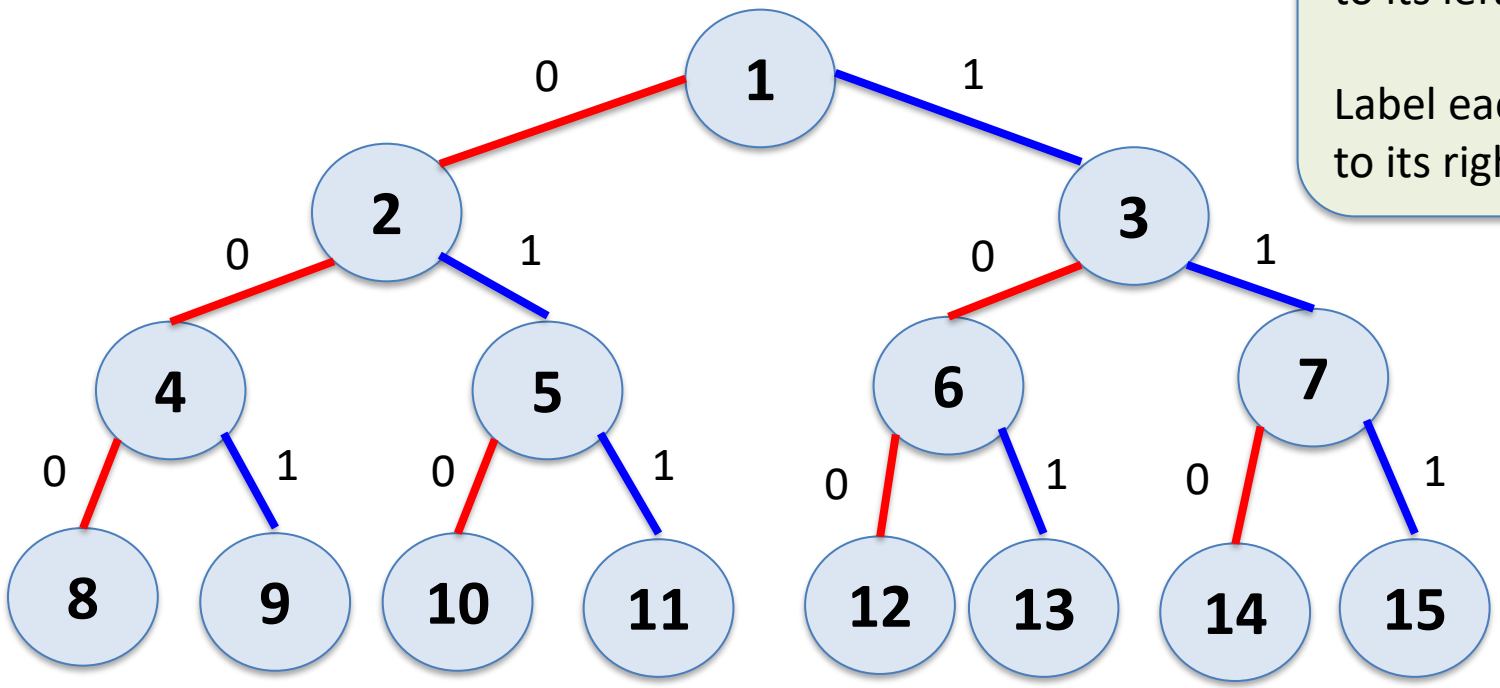


Label each edge from a node to its left child with 0

Label each edge from a node to its right child with 1

Binary Heap

How do we find the last node in the last level?



Label each edge from a node to its left child with 0

Label each edge from a node to its right child with 1

To go from the root to any node follow the binary representation of that node!

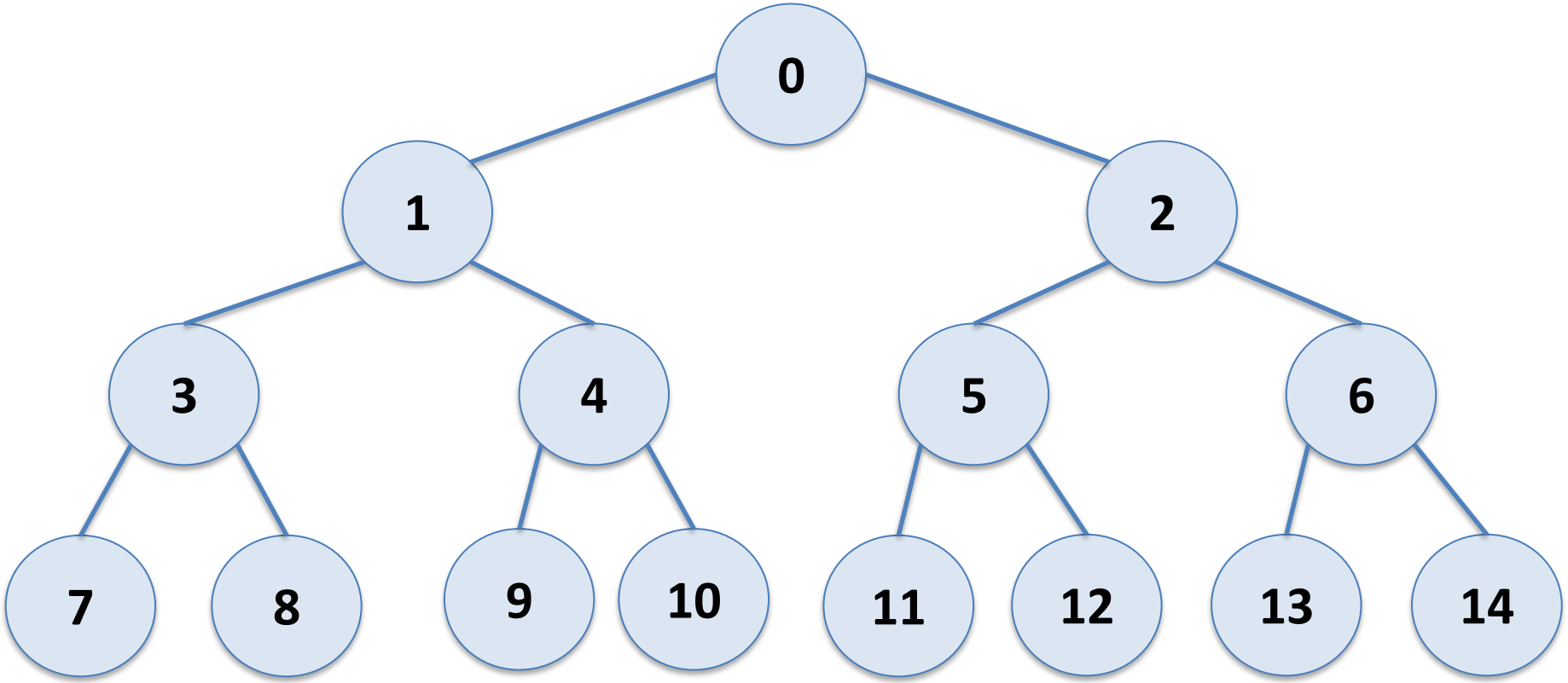
Eytzinger's Method

Eytzinger's method allows us to represent a complete binary tree using an array.

Label the nodes by their BFS (breadth first search) order starting with 0 now.

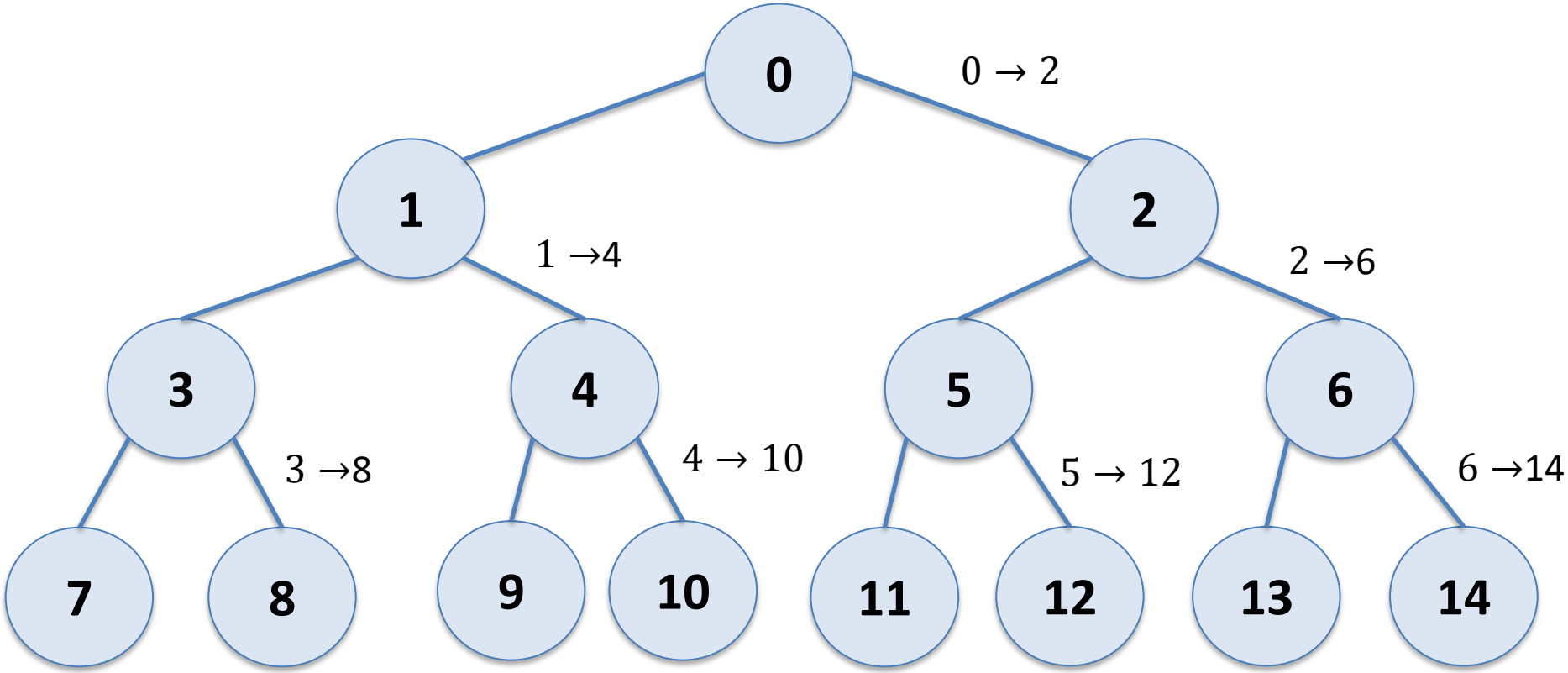
Eytzinger's Method

Eytzinger's method allows us to represent a complete binary tree using an array.



Eytzinger's Method

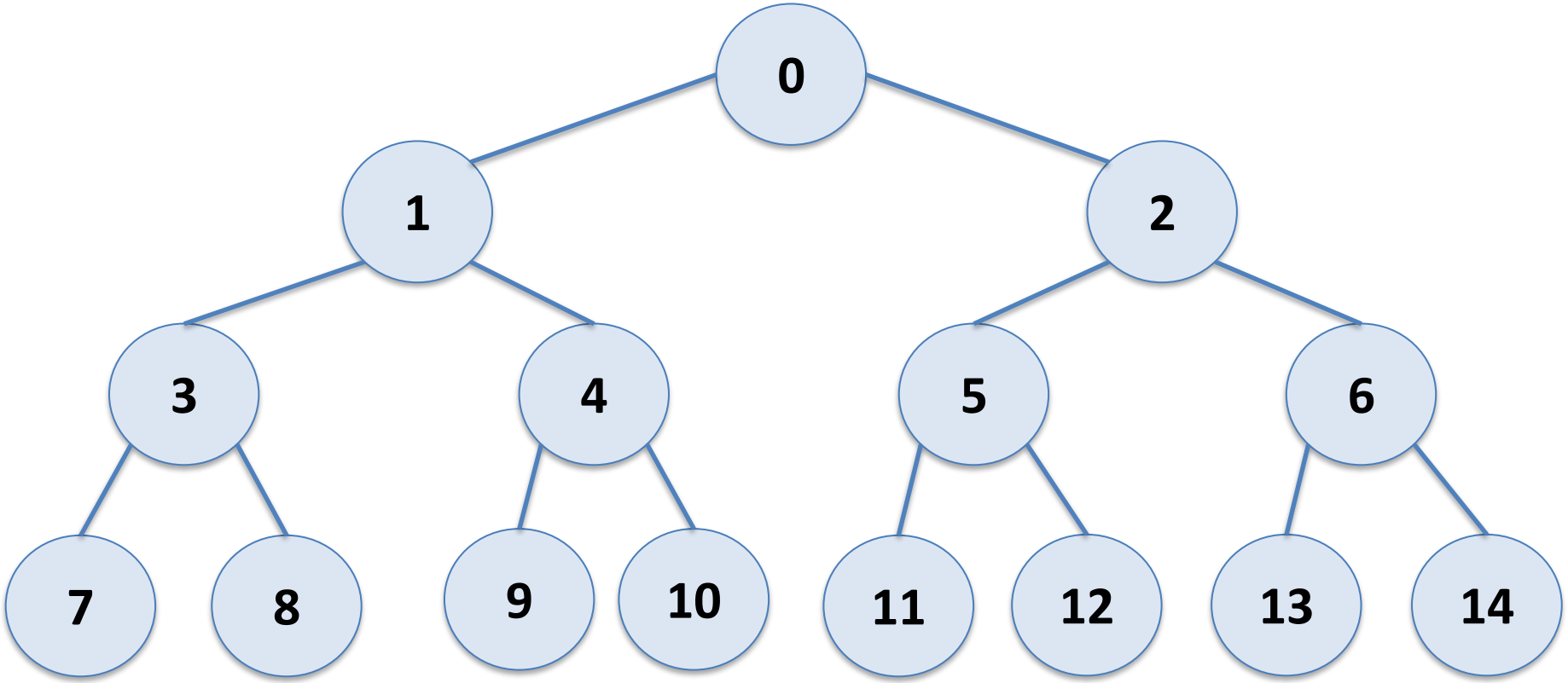
Indices: what is right child of node x?



Eytzinger's Method

Indices: what is right child of node x? 2x+2

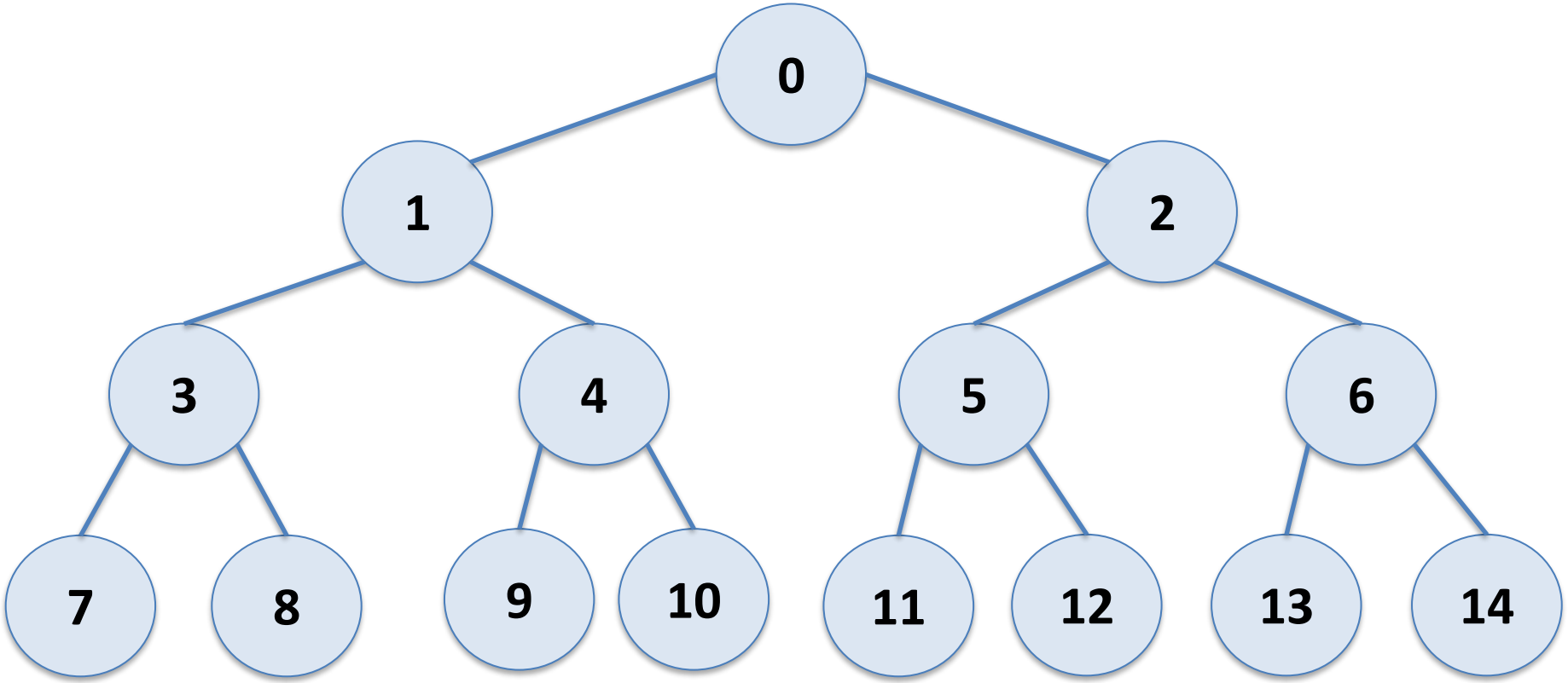
Indices: what is left child of node x?



Eytzinger's Method

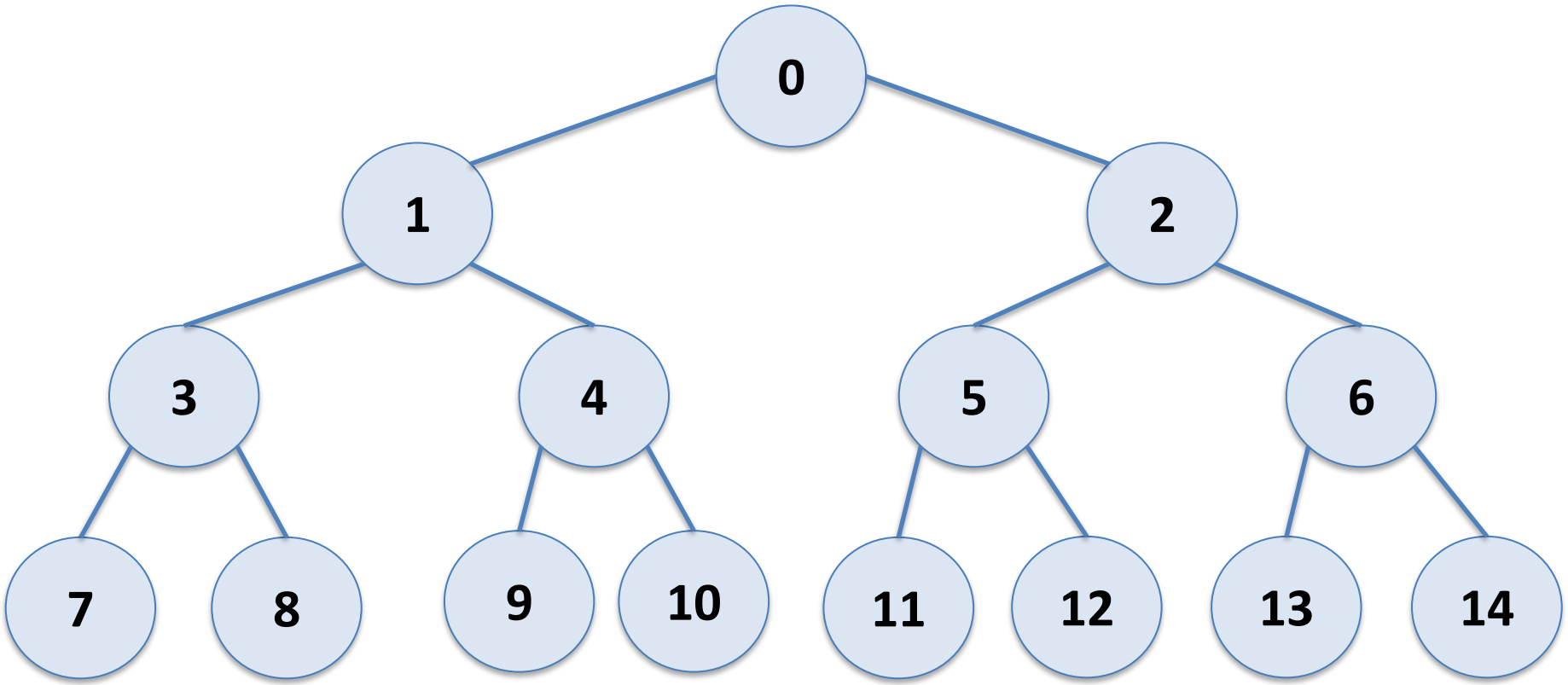
Indices: what is right child of node x? 2x+2

Indices: what is left child of node x? 2x+1



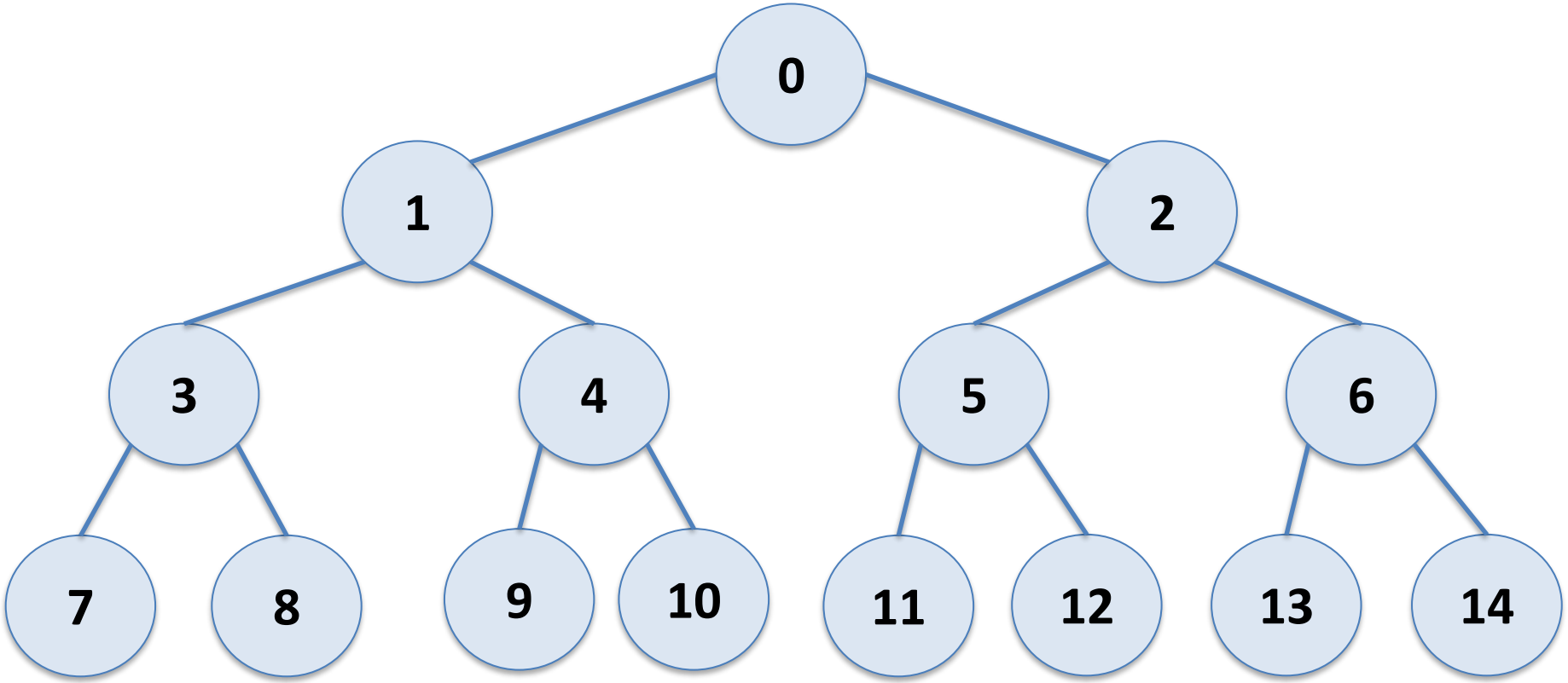
Eytzinger's Method

Indices: what is parent of node x?



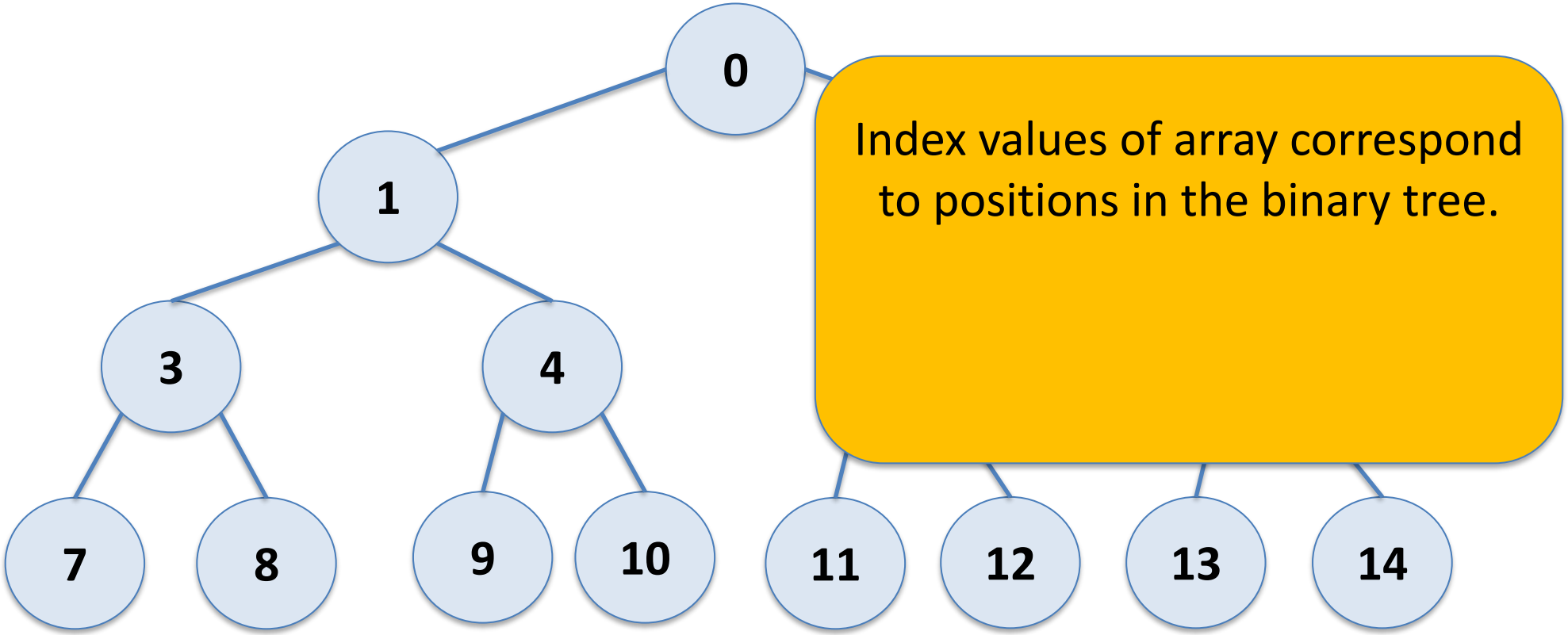
Eytzinger's Method

Indices: what is parent of node x? floor of (x-1)/2



Eytzinger's Method

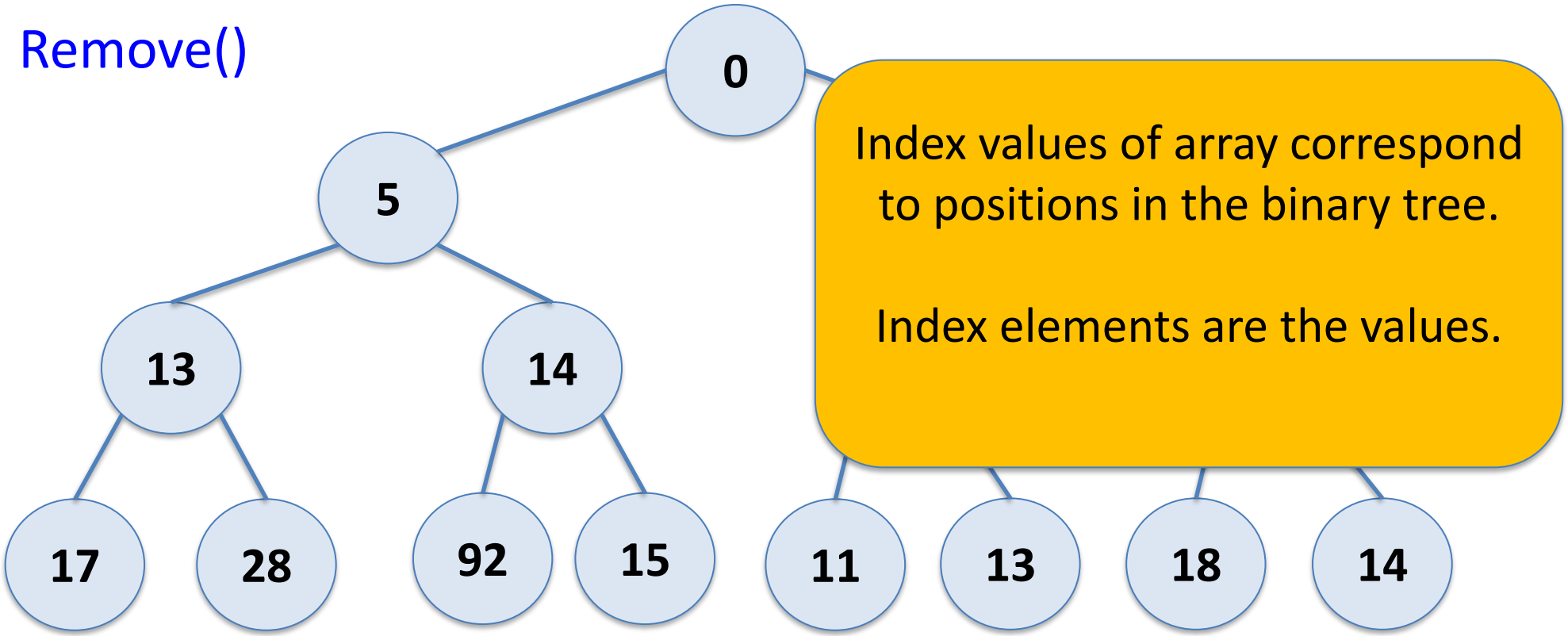
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14



Eytzinger's Method

0	5	4	13	14	6	10	17	28	92	15	11	13	18	14

Remove()



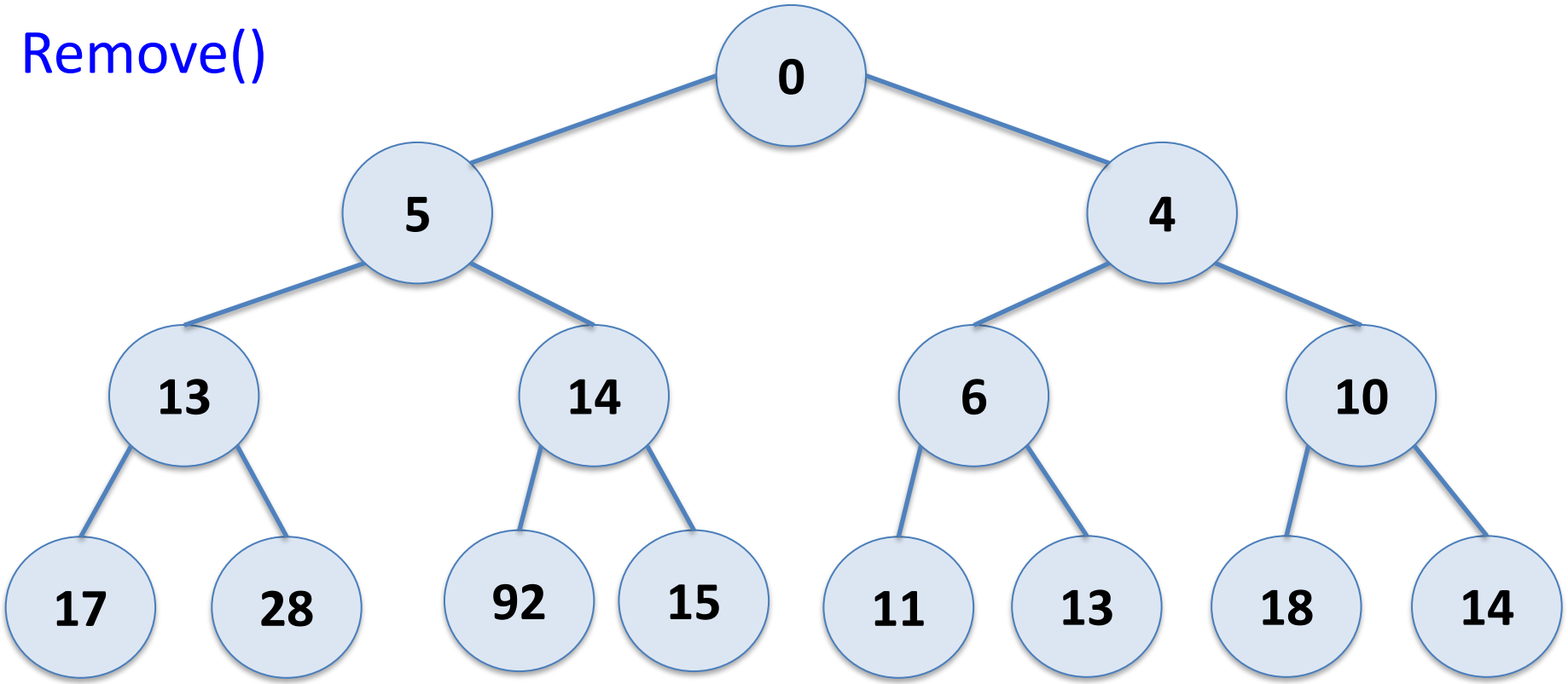
Index values of array correspond to positions in the binary tree.

Index elements are the values.

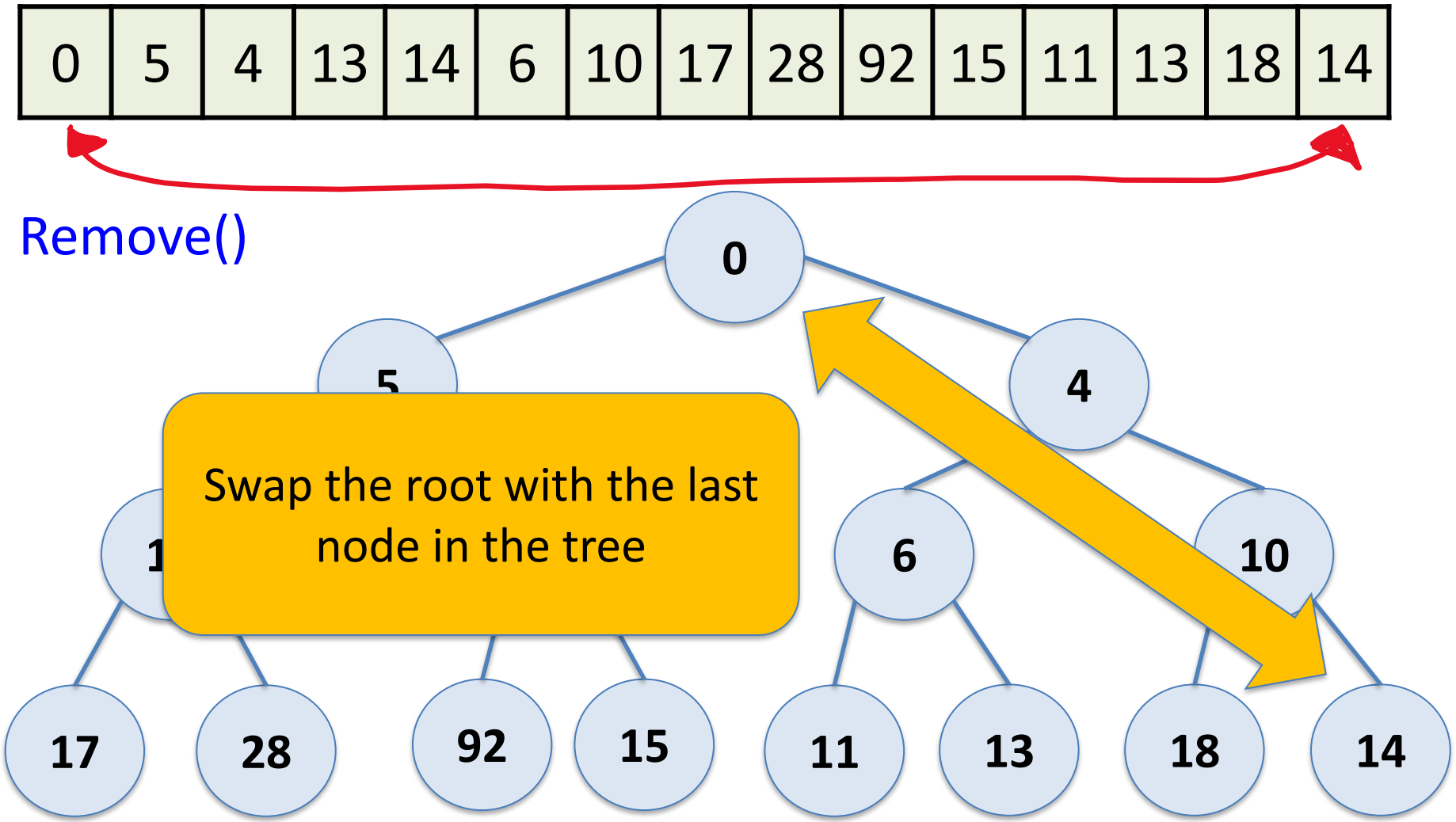
Eytzinger's Method

0	5	4	13	14	6	10	17	28	92	15	11	13	18	14

Remove()



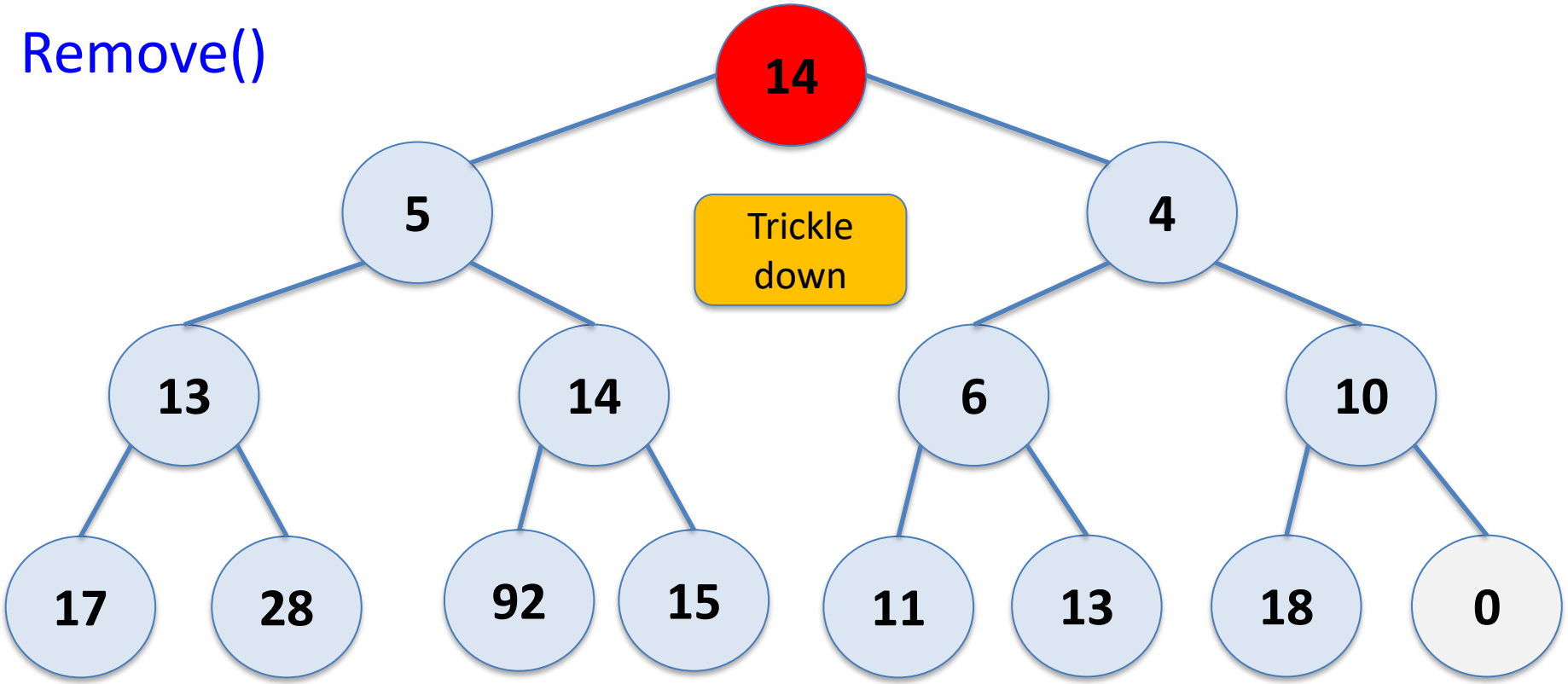
Eytzinger's Method



Eytzinger's Method

14	5	4	13	14	6	10	17	28	92	15	11	13	18	0

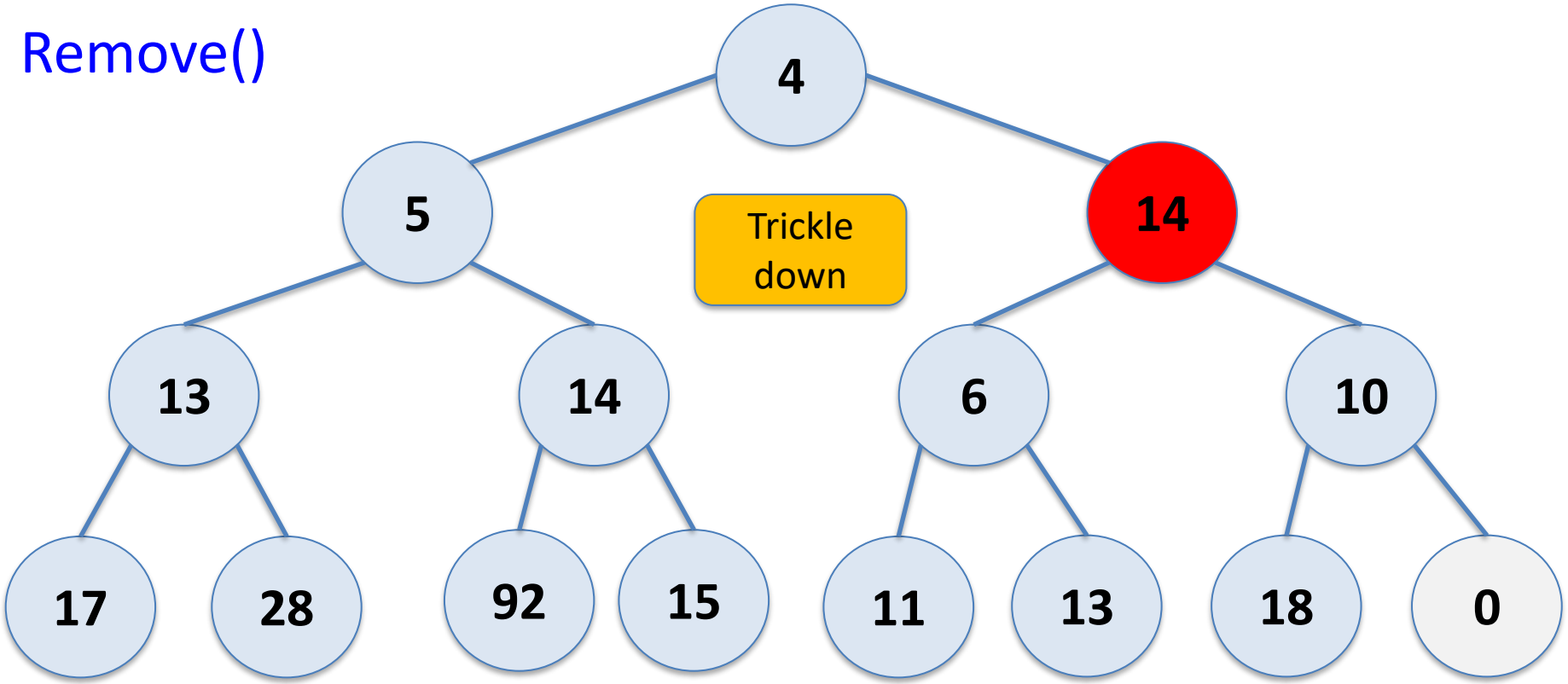
Remove()



Eytzinger's Method

4	5	14	13	14	6	10	17	28	92	15	11	13	18	0

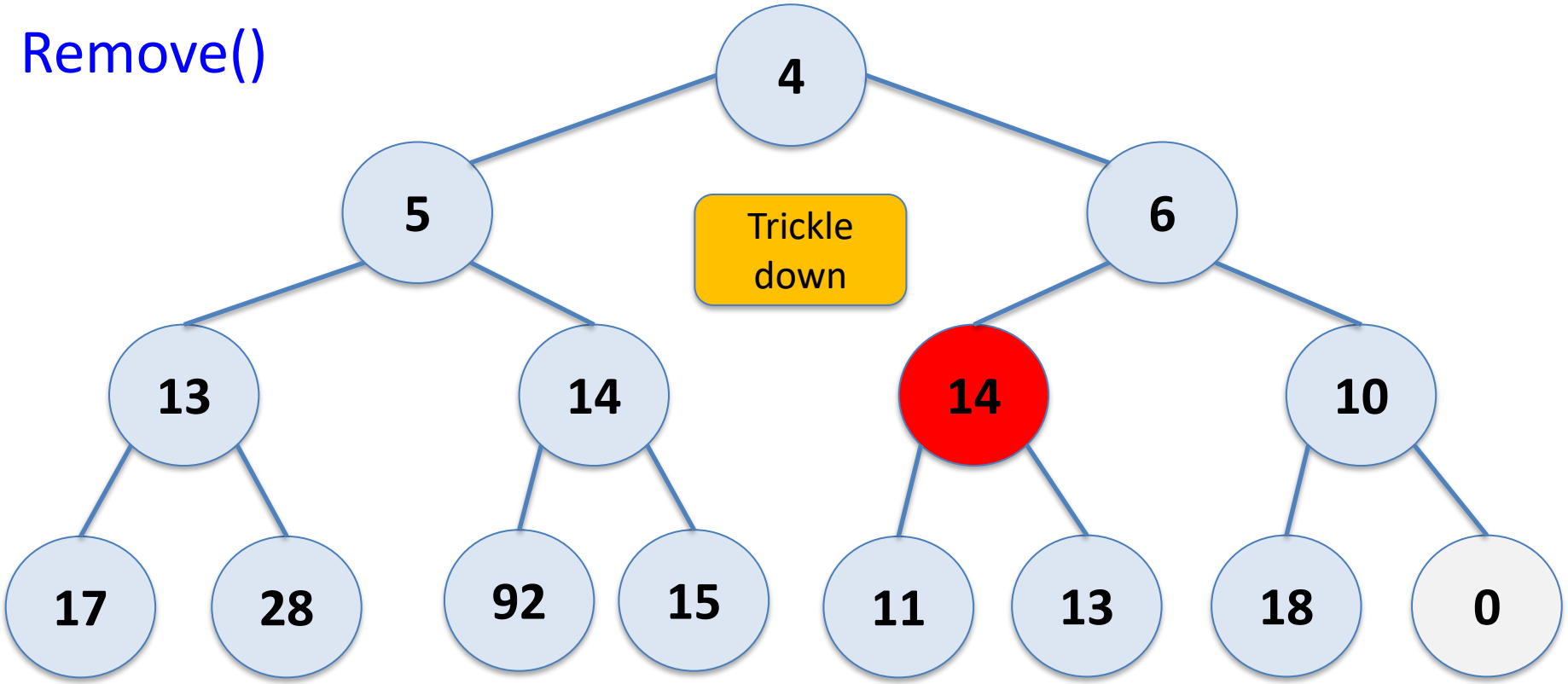
Remove()



Eytzinger's Method

4	5	6	13	14	14	10	17	28	92	15	11	13	18	0

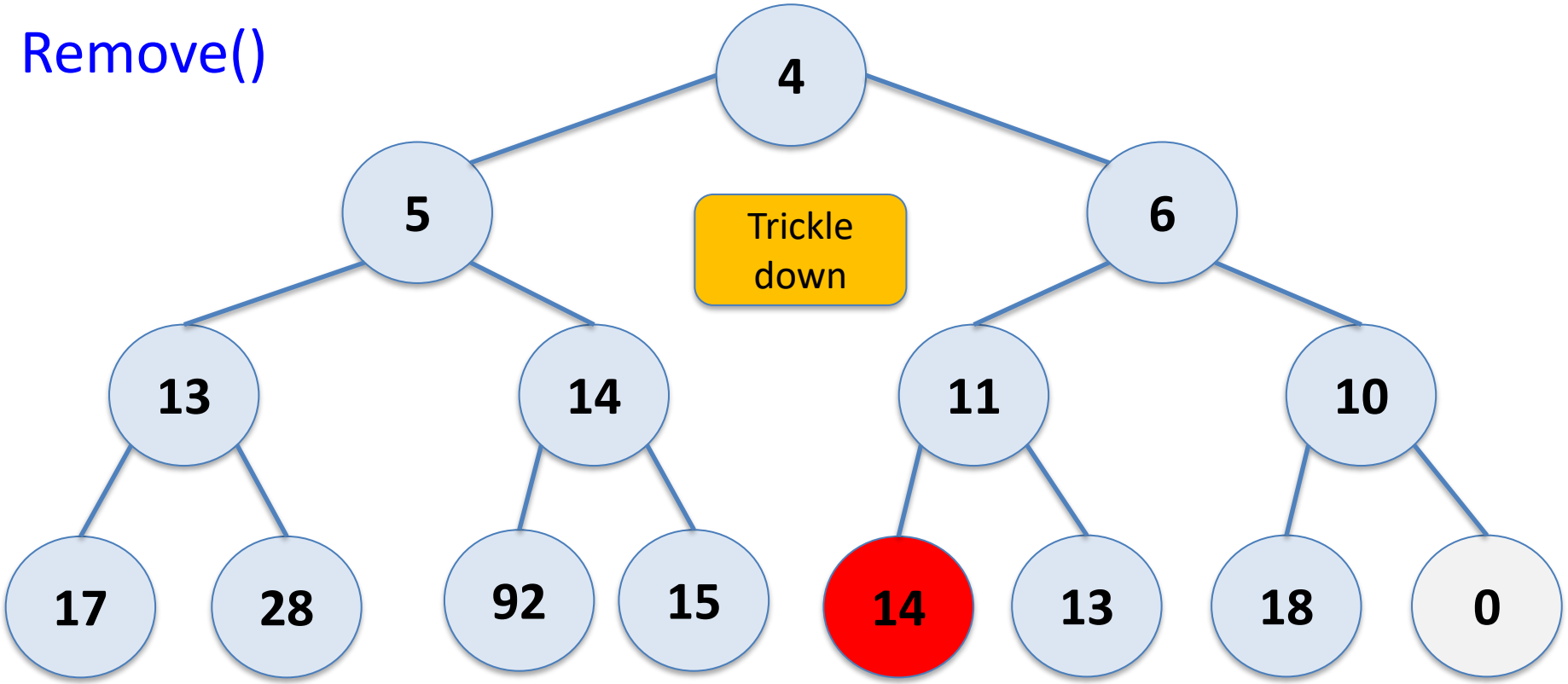
Remove()



Eytzinger's Method

4	5	6	13	14	11	10	17	28	92	15	14	13	18	0

Remove()



Eytzinger's Method

```
boolean add(T x) {
    if (n + 1 > a.length) resize();
    a[n++] = x;
    bubbleUp(n-1);
    return true;
}
void bubbleUp(int i) {
    int p = parent(i);
    while (i > 0 && compare(a[i], a[p]) < 0) {
        swap(i,p);
        i = p;
        p = parent(i);
    }
}
int parent(int i) { return (i-1)/2; }
```

Binary Heap

Theorem 10.1 A **BinaryHeap** implements the (priority) Queue interface. Ignoring the cost of calls to resize(), a BinaryHeap supports the operations add(x,p) and remove() in $O(\log n)$ time per operation.

Furthermore, beginning with an empty BinaryHeap, any sequence of m add(x,p) and remove() operations results in a total of $O(m)$ time spent during all calls to resize(). (Hence amortized constant time.)

Heapify

Question: How do you create binary heap from an unsorted array?

Heapify

Question: How do you create binary heap from an unsorted array?

1. Build the heap from the top down…

```
for j from 1 to n-1 do
  bubbleUp(j)
```

Heapify

Question: How do you create binary heap from an unsorted array?

2. Build the heap from the bottom up…

```
for j from n/2-1 downto 0 do
  trickleDown(j)
```

Heapify

Question: How do you create binary heap from an unsorted array?

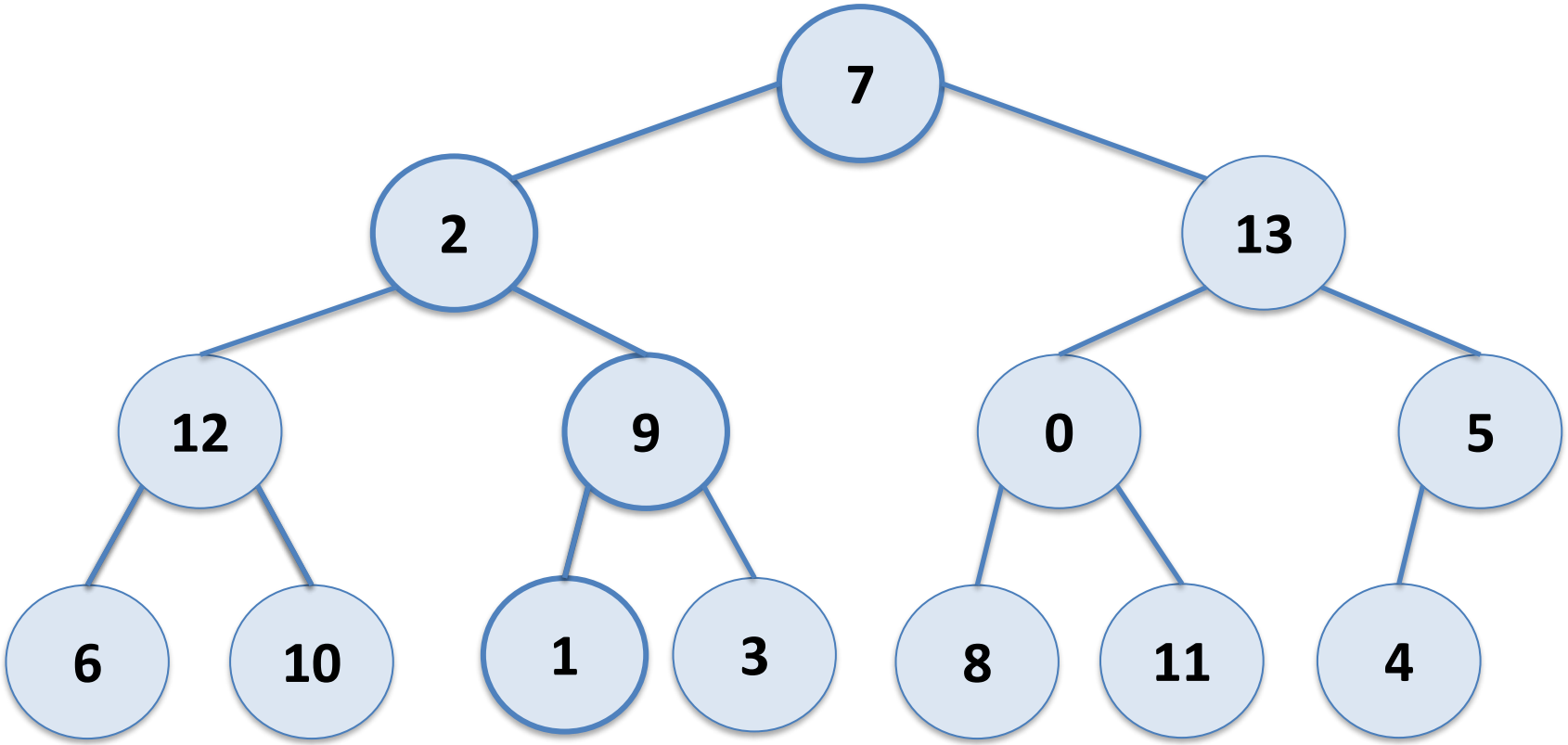
2. Build the hea

This is called the **heapify**, **fastHeapify** or **buildHeap** method

```
for j from n/2-1 downto 0 do
  trickleDown(j)
```

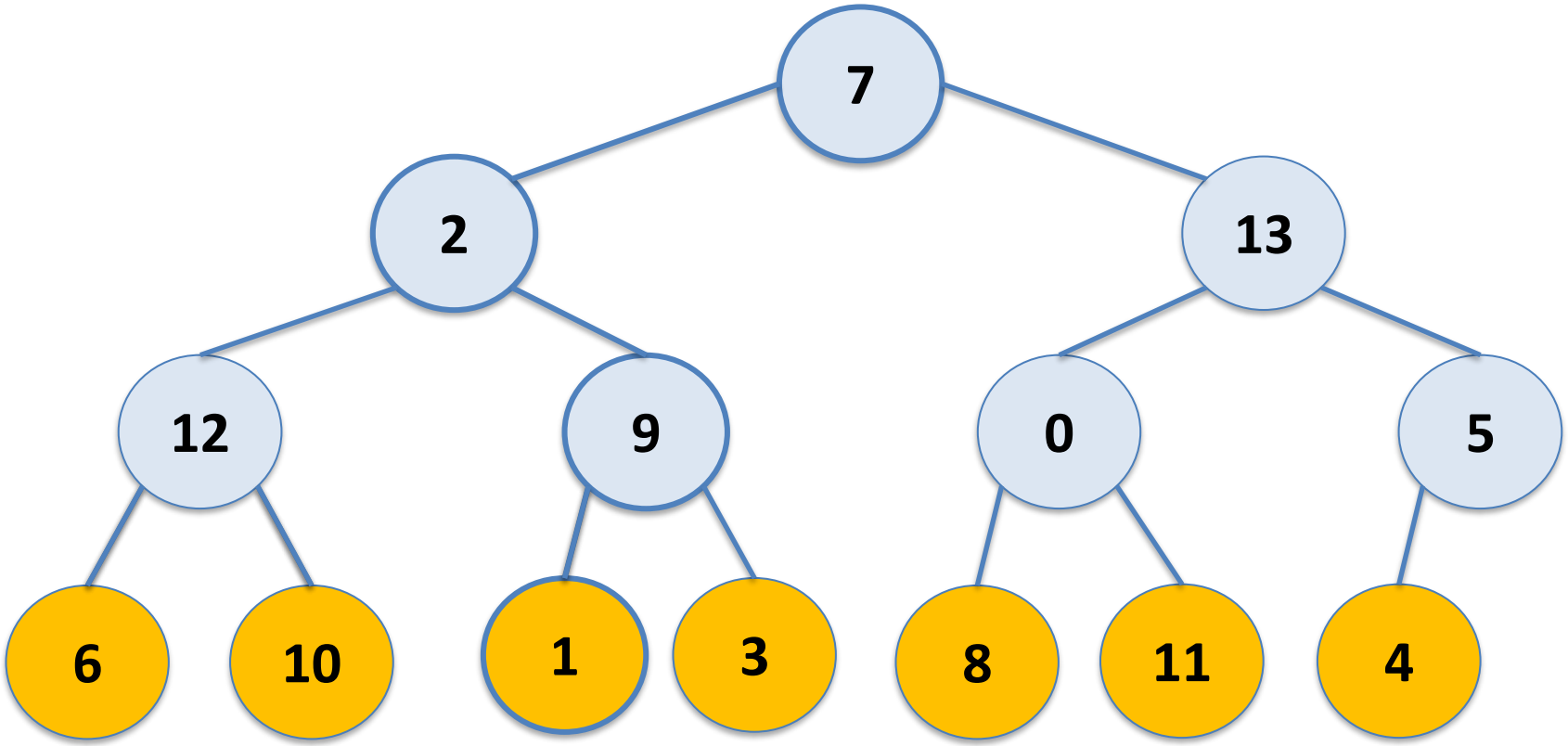
Fast Heapify

7	2	13	12	9	0	5	6	10	1	3	8	11	4	



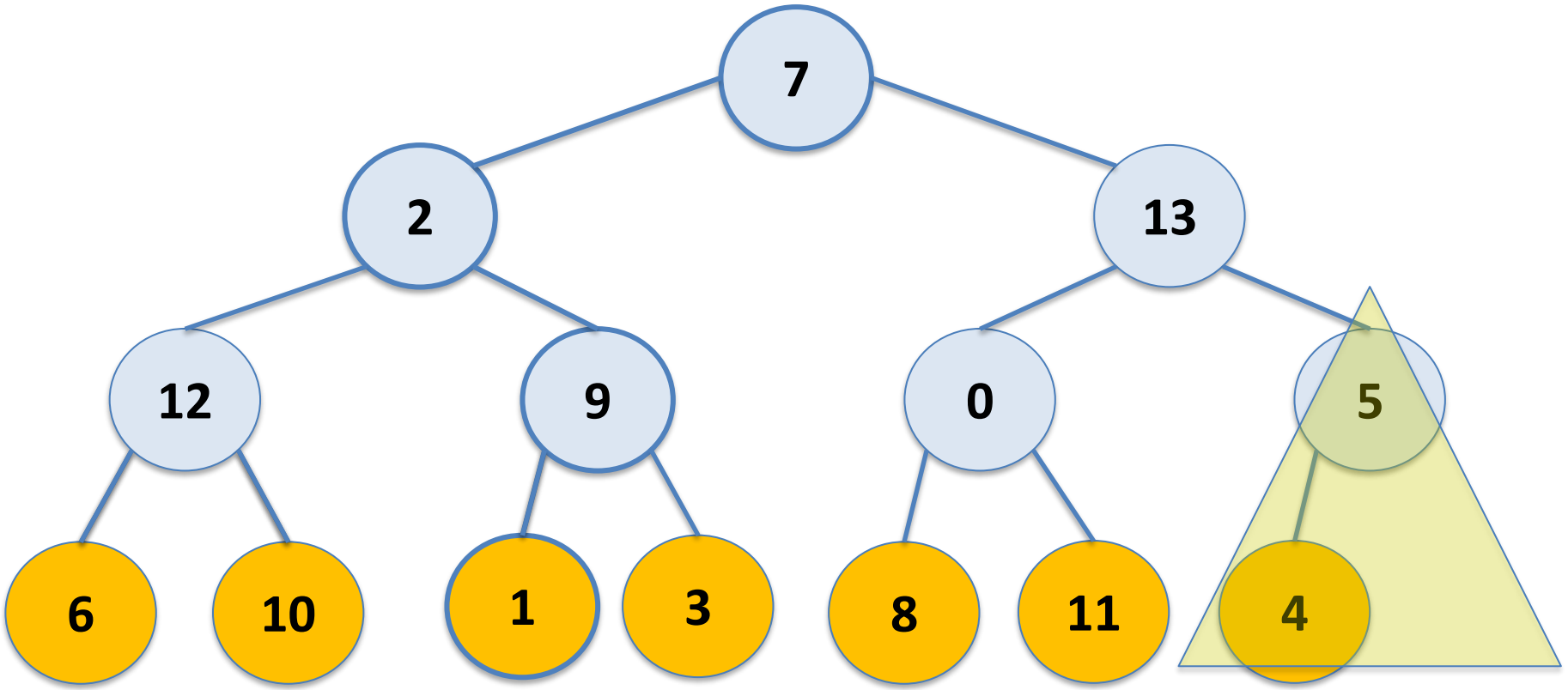
Fast Heapify

7	2	13	12	9	0	5	6	10	1	3	8	11	4	



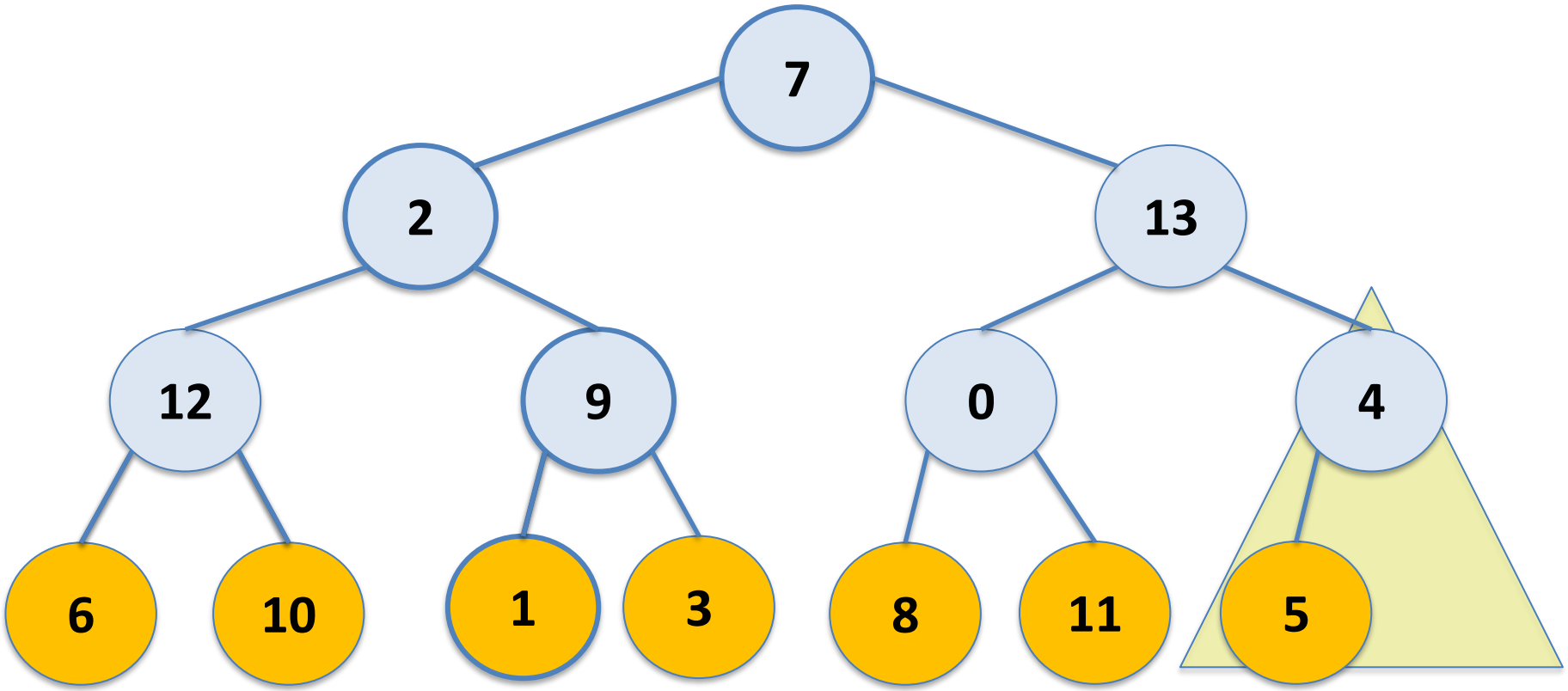
Fast Heapify

7	2	13	12	9	0	5	6	10	1	3	8	11	4	



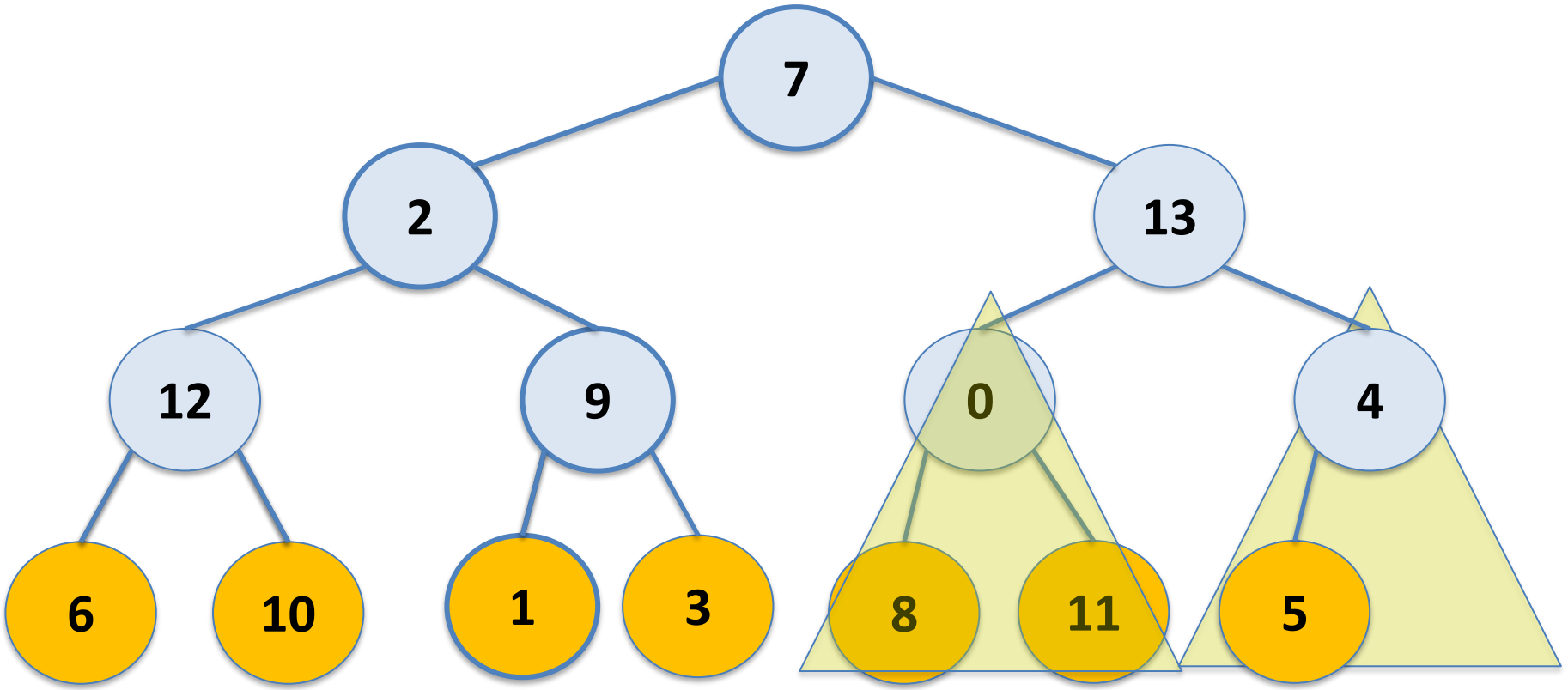
Fast Heapify

7	2	13	12	9	0	4	6	10	1	3	8	11	5	



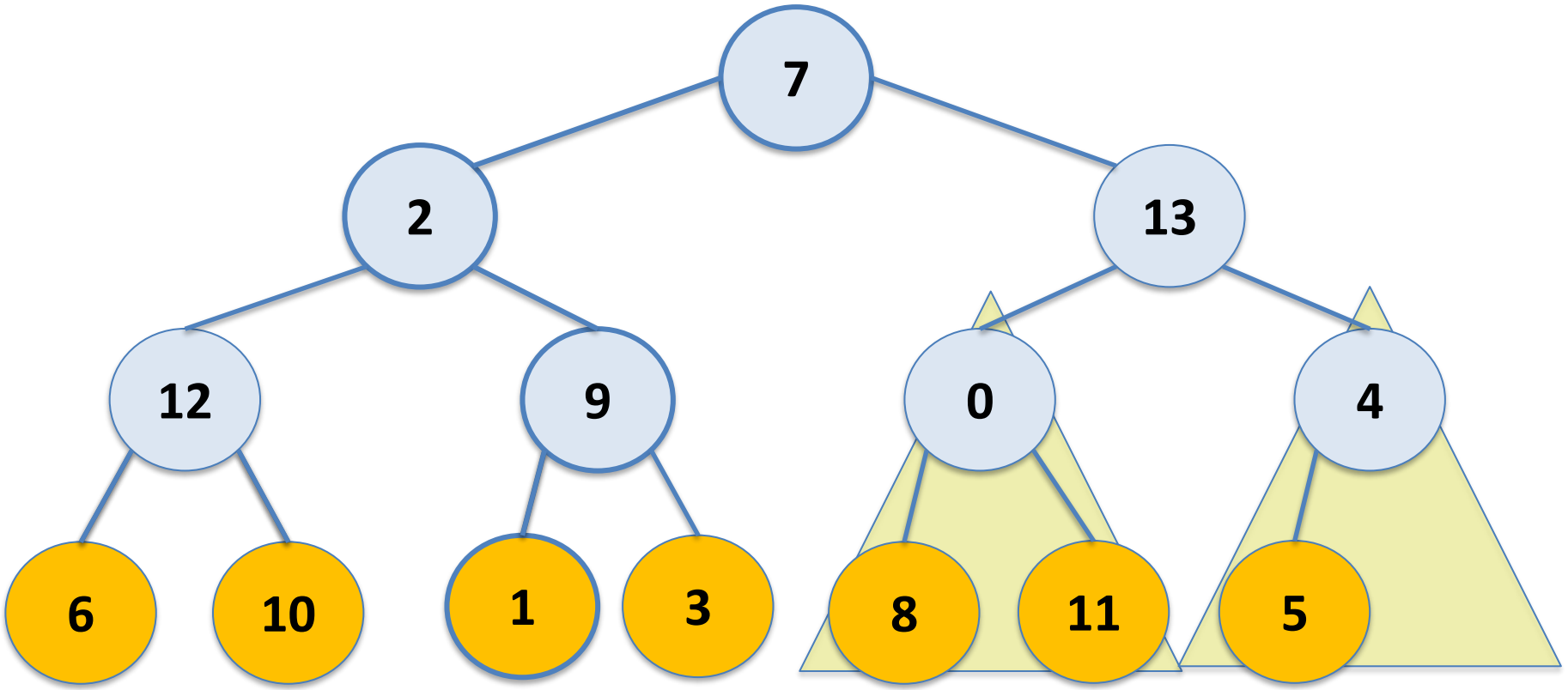
Fast Heapify

7	2	13	12	9	0	4	6	10	1	3	8	11	5	



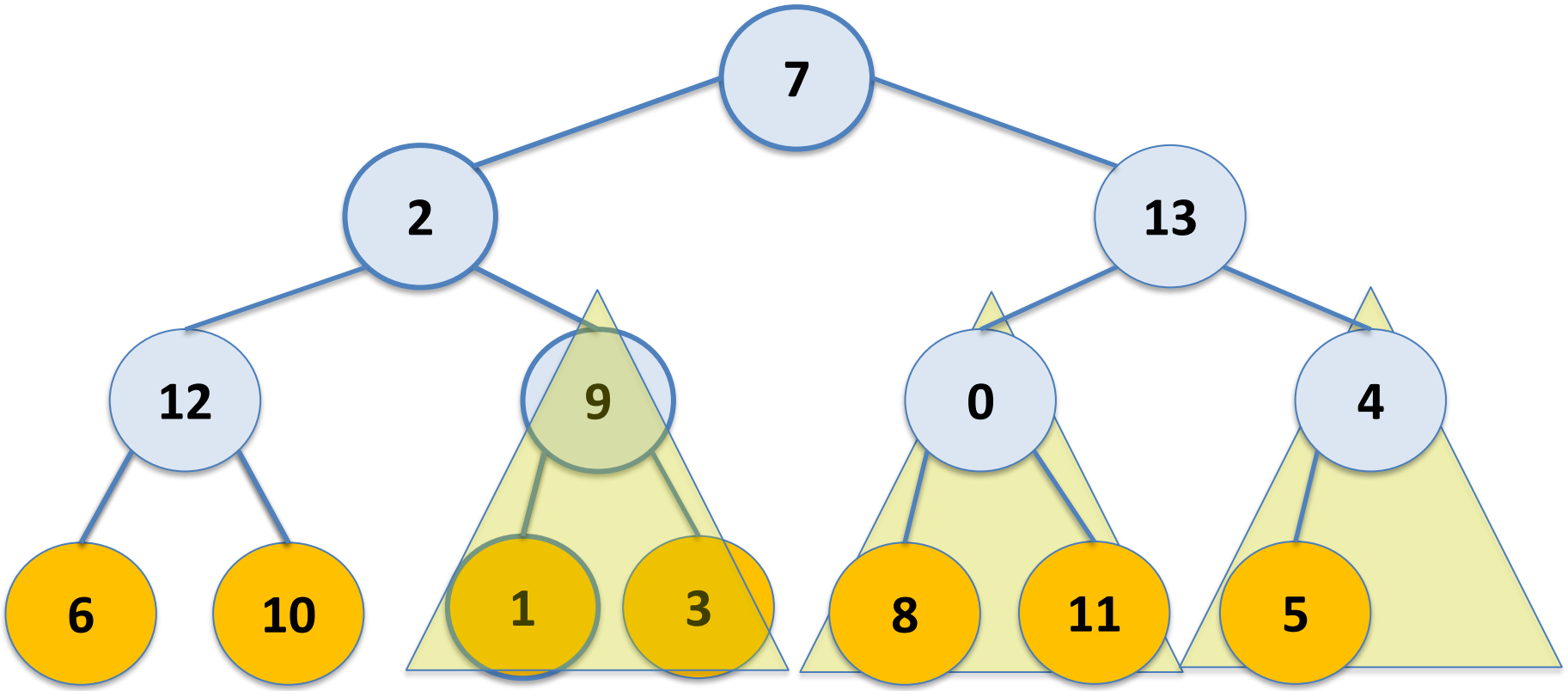
Fast Heapify

7	2	13	12	9	0	4	6	10	1	3	8	11	5	



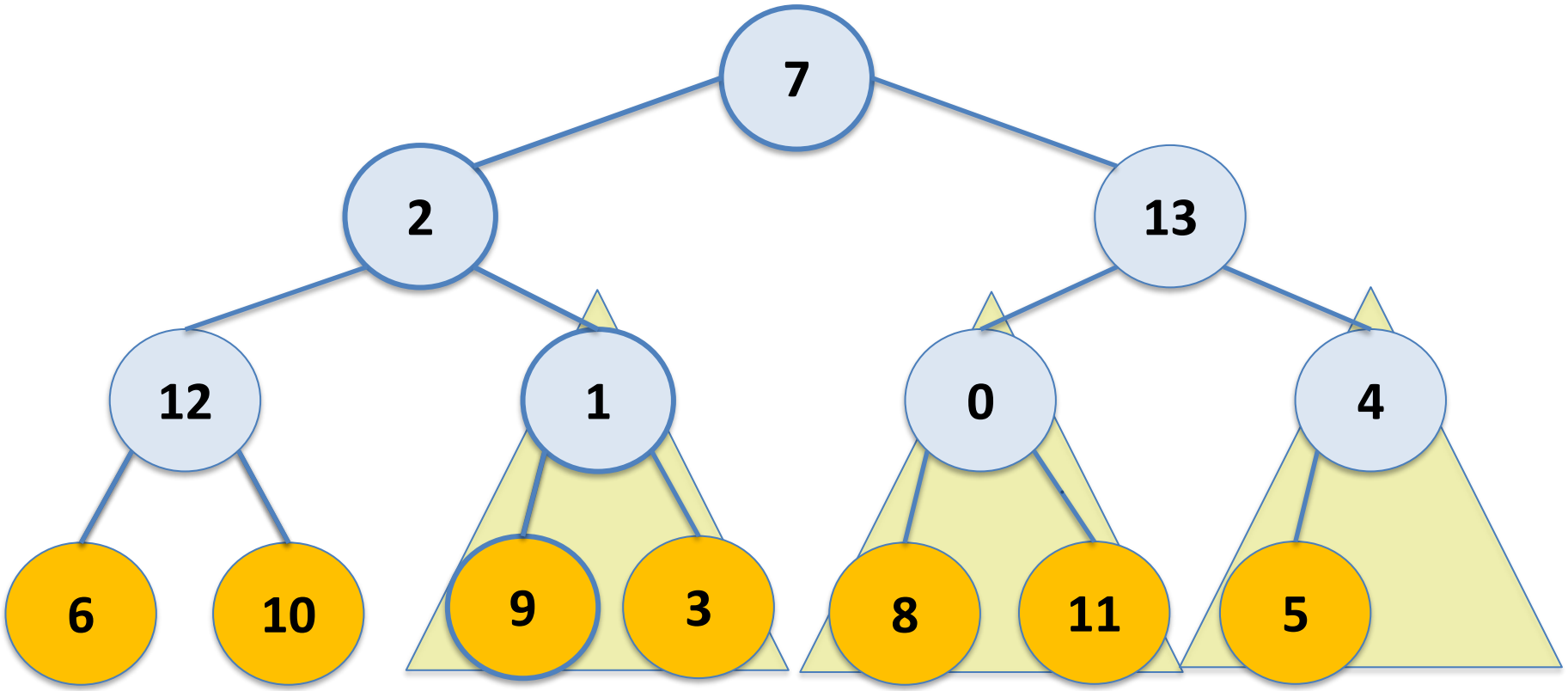
Fast Heapify

7	2	13	12	9	0	4	6	10	1	3	8	11	5	



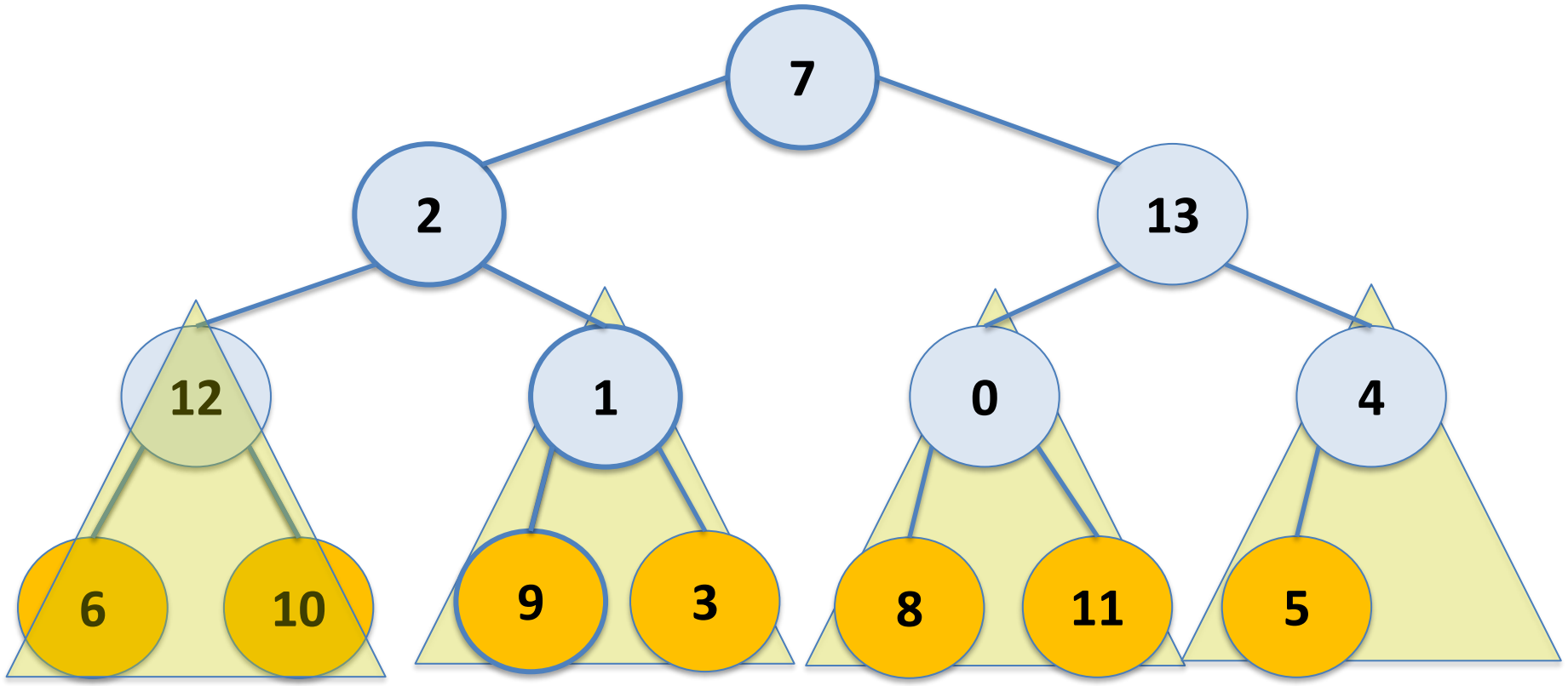
Fast Heapify

7	2	13	12	1	0	4	6	10	9	3	8	11	5	



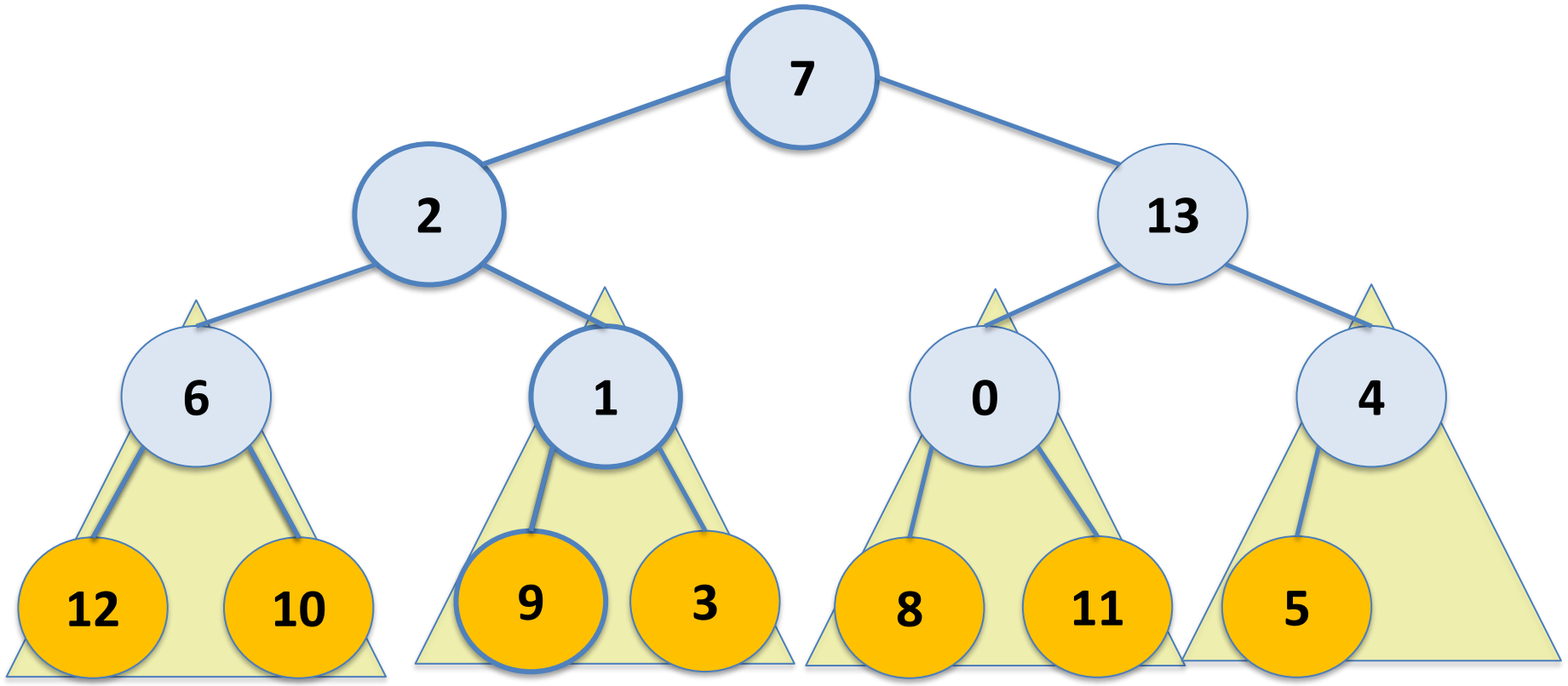
Fast Heapify

7	2	13	12	1	0	4	6	10	9	3	8	11	5	



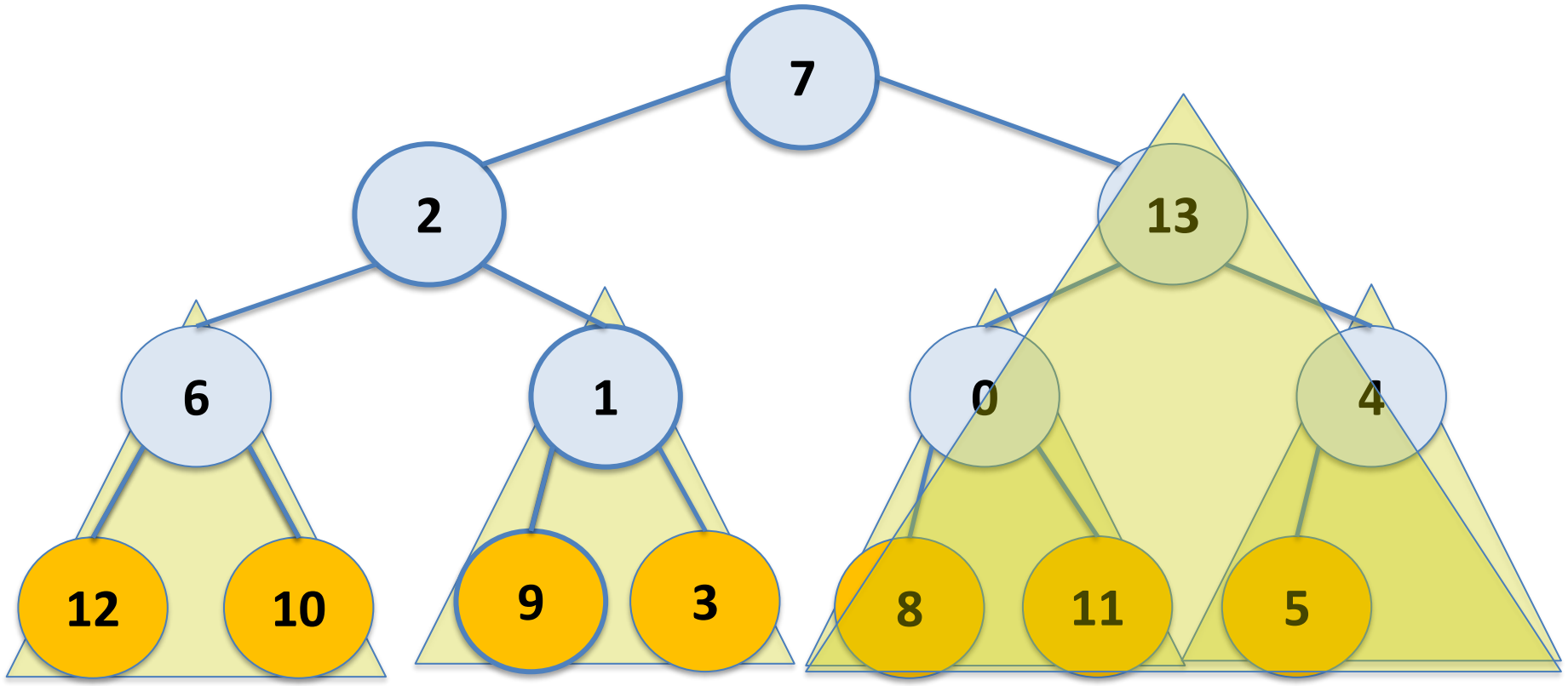
Fast Heapify

7	2	13	6	1	0	4	12	10	9	3	8	11	5	



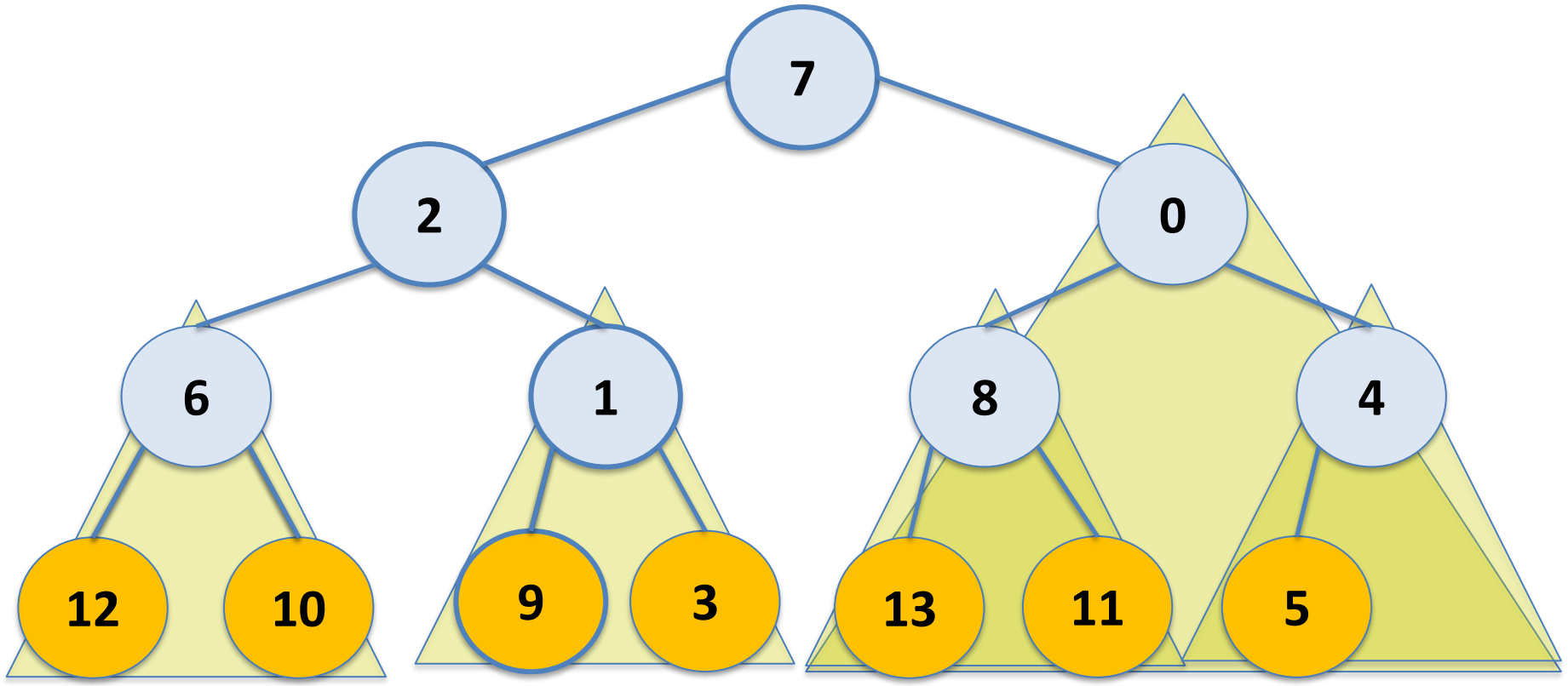
Fast Heapify

7	2	13	6	1	0	4	12	10	9	3	8	11	5	



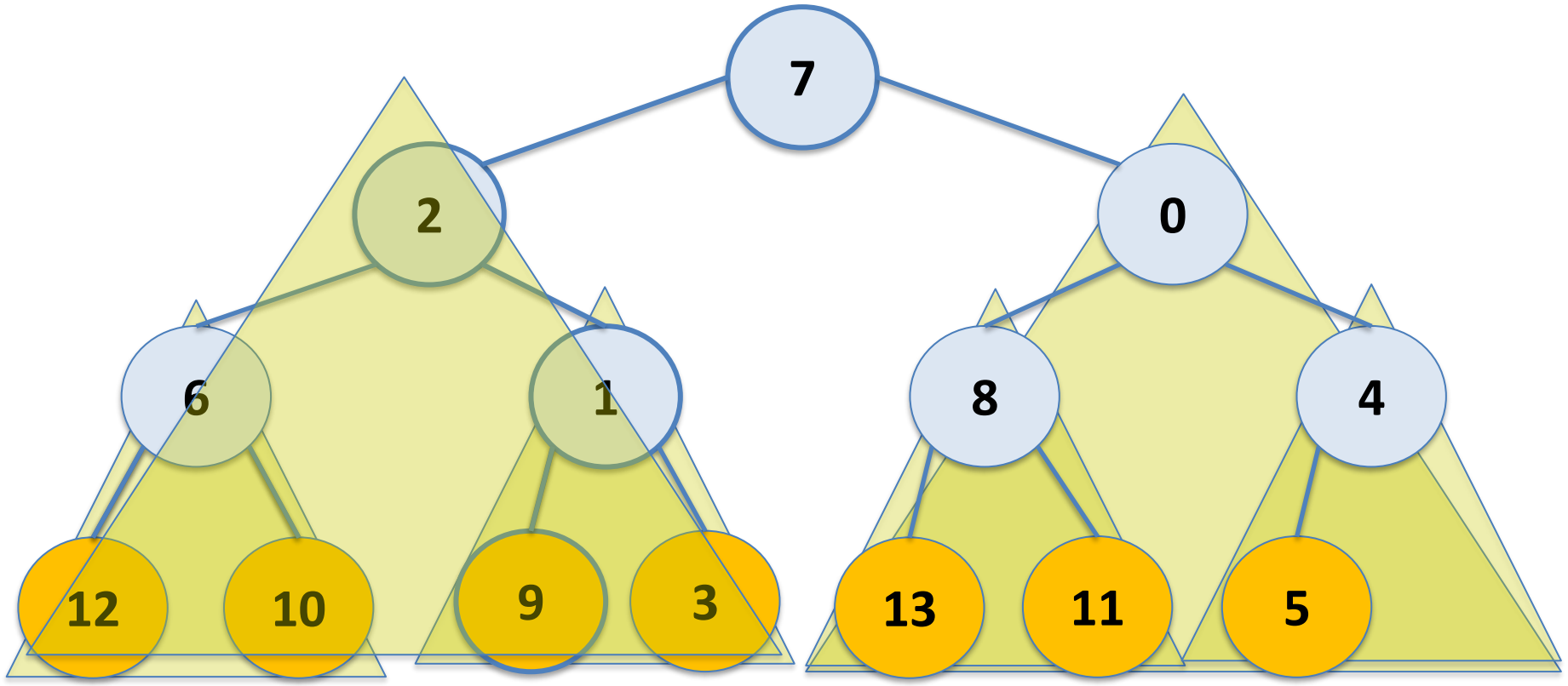
Fast Heapify

7	2	0	6	1	8	4	12	10	9	3	13	11	5	



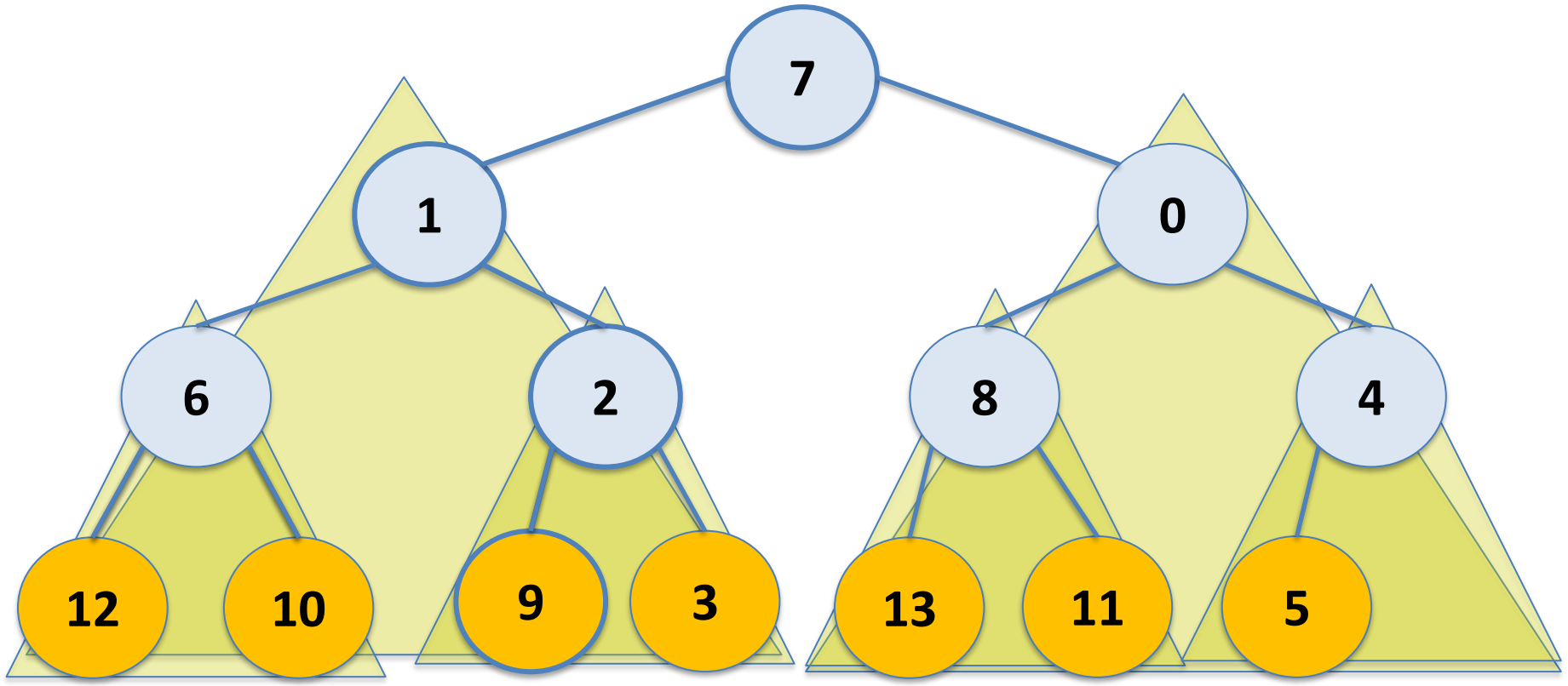
Fast Heapify

7	2	0	6	1	8	4	12	10	9	3	13	11	5	



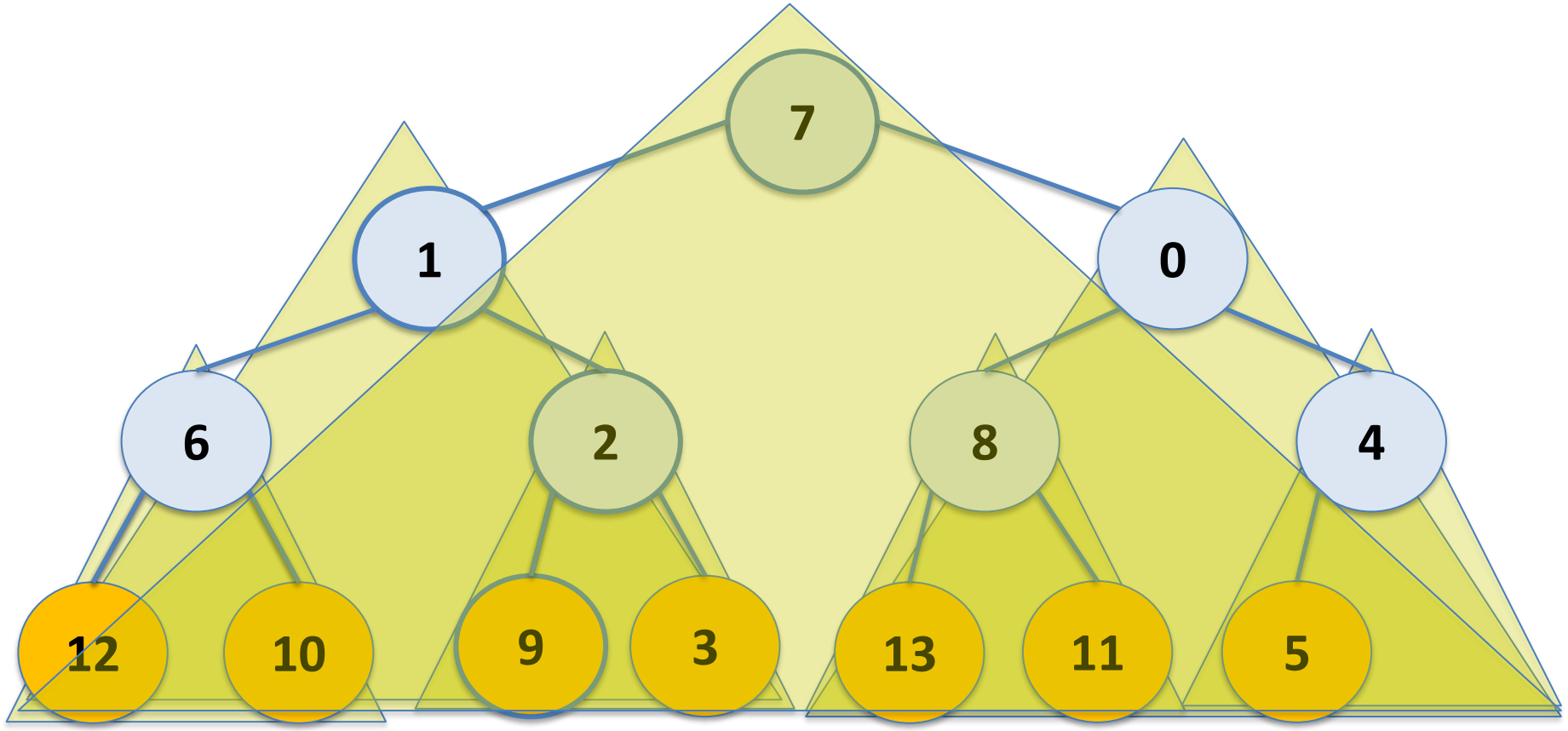
Fast Heapify

7	1	0	6	2	8	4	12	10	9	3	13	11	5	



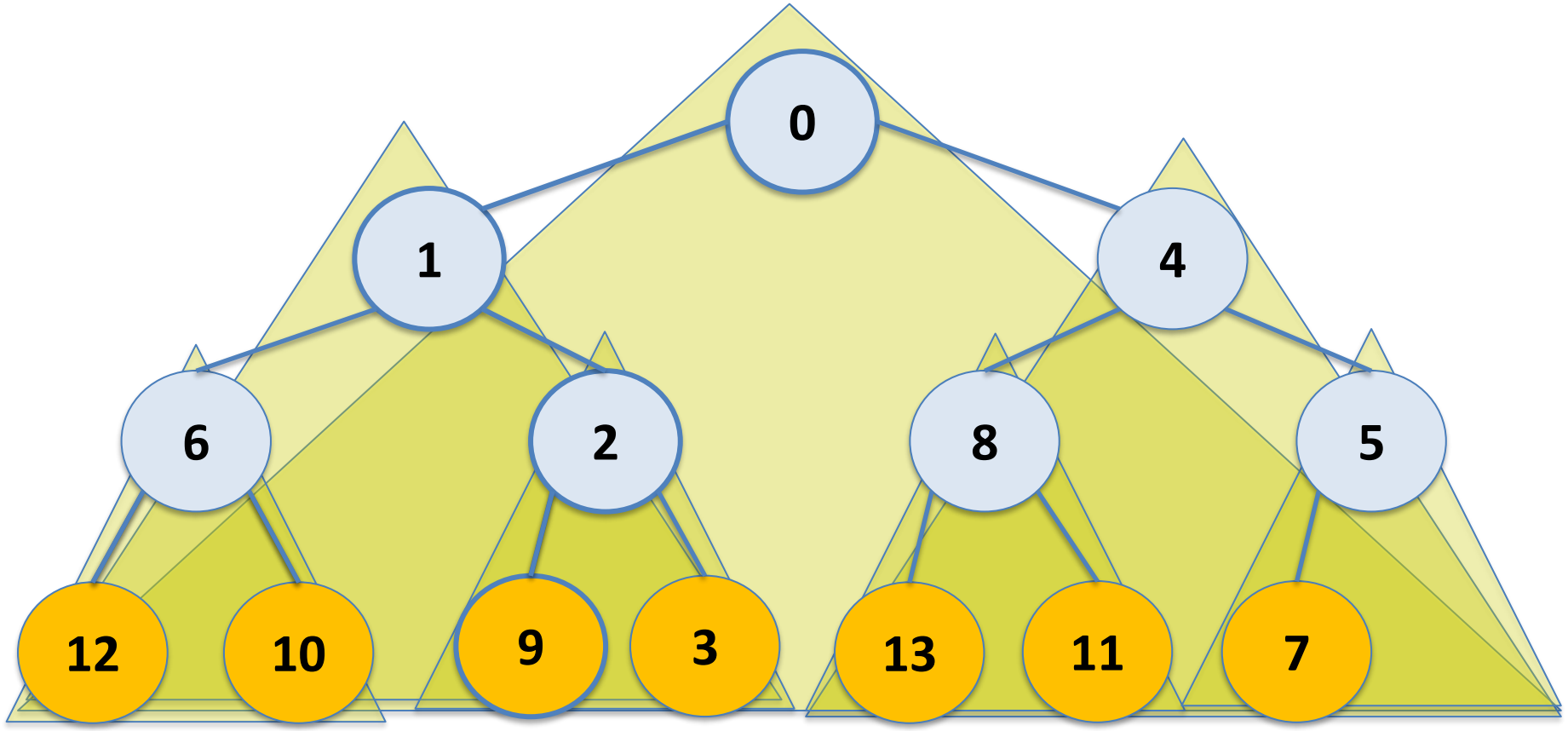
Fast Heapify

7	1	0	6	2	8	4	12	10	9	3	13	11	5	



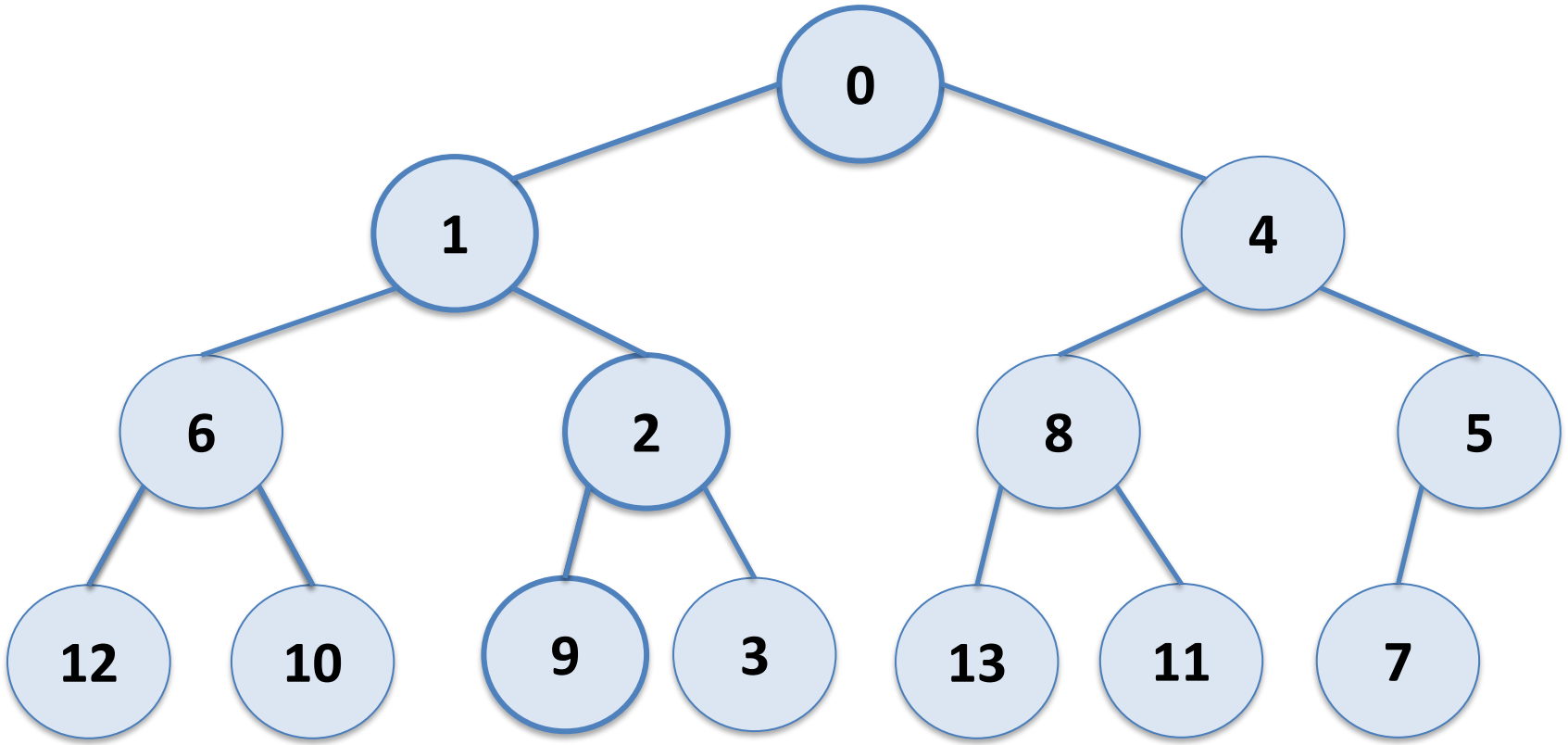
Fast Heapify

0	1	4	6	2	8	5	12	10	9	3	13	11	7	



Fast Heapify

0	1	4	6	2	8	5	12	10	9	3	13	11	7	



Fast Heapify

0	1	4	6	2	8	5	12	10	9	3	13	11	7	

n/2 elements trickle down from height 0 (do nothing)

n/4 elements trickle down from height 1

n/8 elements trickle down from height 2

⋮

1 element trickles down from height h

Fast Heapify

n/2 elements trickle down from height 0 (do nothing)

n/4 elements trickle down from height 1

n/8 elements trickle down from height 2

⋮

1 element trickles down from height h

Number of comparisons is

$$\leq 2 \times \left(0\frac{n}{2} + 1\frac{n}{4} + 2\frac{n}{8} + 3\frac{n}{16} + \cdots + h \cdot 1\right)$$

Fast Heapify

Number of comparisons is

$$\leq 2 \times \left(0\frac{n}{2} + 1\frac{n}{4} + 2\frac{n}{8} + 3\frac{n}{16} + \cdots + h\right)$$

$$\leq 2\sum_{i=0}^{h} i\frac{n}{2^{i+1}} = \frac{2n}{2}\sum_{i=0}^{\mathrm{h}} \frac{i}{2^i} < n\sum_{i=1}^{\infty} \frac{i}{2^i} = 2n \in O(n)$$

Fast Heapify

We can build a binary heap from an existing unordered array with at most $2n$ comparisons. This is done in-place (no extra memory needed).

If we started with an empty heap and added the elements one-by-one from the array to the we would need $O(n \log n)$comparisons.

Fast Heapify

Exercise: How many comparisons are needed with the top down approach?

Build the heap from the top down…

```
for j from 1 to n-1 do
  bubbleUp(j)
```

Hint: what is the cost of calling `bubbleUp(j)` for the values in the bottom level? (There are n/2 of these values…)

Heap

What if you want to remove an element from the heap that doesn't have the min priority?